

Curated File Guide: script.js

The public website runtime that renders the portfolio, search, project windows, browser history, rich content, links, and AI chat behavior.

Before The Code: File Overview

Abstract

script.js is the public website runtime. It is the JavaScript that turns the static portfolio files into an interactive recruiter-facing site. The HTML page provides stable containers and the JSON catalog provides saved content, but script.js decides what appears, how projects are filtered, how rich text renders, how files become links, how project sections open into full-window views, how browser Back and Refresh interact with those views, how search behaves like a real search tool, and how the AI chat gathers useful context before asking the backend.

The important distinction is that script.js is public and read-only. It should never expose builder-only editing controls. Its job is to present what has already been saved and parsed by the builder. That is why much of the file is about rendering, normalization, routing, search indexing, assistant context selection, and UI state rather than file writes.

Introduction

When a visitor opens the portfolio, index.html loads styles.css, electronics-search.js, and script.js. script.js immediately grabs the DOM elements it will control: the project grid, search input, filters, profile fields, resume area, contact area, AI assistant, and footer. Then it waits for portfolio data. The data can come from an embedded window.__PORTFOLIO_CATALOG__ object in previews or from projects.json on the published website. Once the catalog is loaded, script.js normalizes profile data, renders the hero and contact information, creates category filters, renders project cards, builds the search index, and restores any deep-linked project section.

This file is the main reason the same portfolio can feel alive without requiring a database-backed website. It uses static files as the source of truth, then builds an application-like experience in the browser. That makes the public website easy to host on GitHub Pages or Cloudflare, while still giving visitors search, project windows, AI questions, and responsive navigation.

Background Information

The file works with a catalog generated by the builder parser. That catalog contains categories, projects, profile fields, site content, field styles, rich text blocks, fun facts, and optional site sections. The functions in script.js assume the catalog is already safe enough to display, but they still escape HTML where user-facing strings are inserted into the DOM. Rich text rendering is intentionally structured because portfolio content can include plain text, links, formulas, images, files, code blocks, captions, and nested project subsections.

The website also has a windowing model. Project cards appear inside categories. A project section button opens a full-screen dialog. Inside that dialog, subsections can be clicked like nested folders. Browser history is updated with the project id, section index, and resource path so the browser Back button can move through the same hierarchy. This makes project content feel more like a navigable portfolio than a long flat webpage.

Search is also broader than a normal text filter. The search index contains page sections, project names, category labels, tools, languages, rich text, files, URLs, captions, uploaded text files where fetchable, and electronics keyword expansions. The goal is that a recruiter can type an electronics phrase and land near the relevant artifact, even if the phrase appears inside a project section or file caption rather than on the top card.

Because the website is static, script.js must do work that a database-backed application might normally ask a server to do. It must decide which categories exist, which projects belong to each category, which sections are

empty, which files should render as downloads, which links should open externally, and which project paths should be represented in browser history. The price of static hosting is that the browser runtime has to be disciplined. The benefit is that the public site remains easy to deploy, cache, mirror, and serve from GitHub Pages or Cloudflare without a long-running custom web server.

The file also sits between several other files. `index.html` gives it DOM anchors. `styles.css` gives visual meaning to the classes it writes. `electronics-search.js` can provide expanded electronics vocabulary. `projects.json` provides the portfolio catalog. The AI backend endpoint gives it remote intelligence when available. If one of those contracts changes, `script.js` is usually the first file that reveals the mismatch because rendering, search, or chat stops behaving correctly.

Purpose Of The File

The purpose of `script.js` is to convert portfolio data into a polished public experience. The file decides not only what to render, but also what not to render. Empty sections are hidden. Overview content appears as the first readable content when present. Links that look like websites open as websites rather than downloads. Local files become downloadable resources. Images render inline only when they are meant to be shown. AI links are only displayed when related to the question context.

Without `script.js`, `index.html` would be mostly a static shell with placeholders. There would be no dynamic project directory, no generated categories, no rich project windows, no browser-aware section routing, no searchable uploaded file text, no AI context gathering, no responsive project rendering, and no dynamic profile/contact population.

Primary Responsibilities

The first responsibility is state hydration. `loadProjectCatalog` reads the portfolio catalog, normalizes the data, stores it in module-level variables, and triggers rendering. This is the startup path for the public website.

The second responsibility is rendering. Functions such as `renderProfile`, `renderSiteContent`, `renderFunFacts`, `renderSiteSections`, `projectCard`, `parsedSection`, and `parsedNodeContent` turn structured catalog objects into actual HTML.

The third responsibility is interaction. The dialog functions create the full-window project section view, the routing functions keep browser history synchronized, the search functions build and display selectable results, and the AI functions manage the chat panel.

The fourth responsibility is interpretation. The assistant functions classify questions, gather relevant portfolio context, select public source links, decide whether a question is general or portfolio-specific, and prepare the request sent to the AI backend. This prevents the assistant from blindly dumping generic links for every question.

The fifth responsibility is restraint. Public visitors should see only finished, parsed, saved portfolio output. The file therefore hides empty content, avoids exposing builder edit controls, limits which sources are shown with AI answers, and uses normalized link behavior so a website link does not become a fake downloadable file. This restraint is what separates the public website from the local builder application.

The sixth responsibility is mobile and desktop continuity. The same project data must read cleanly on an iPhone and on a wide monitor. The responsive helper functions estimate project density and media presence, while the dialog and assistant functions keep windows and chat panels usable without letting one part of the page resize unrelated parts. `script.js` is therefore not only a renderer; it is also the public interaction policy for different devices.

Important Data Objects And Why They Matter

`categories`, `projects`, `siteSections`, `profile`, `siteContent`, `funFacts`, `fieldStyles`, and the rich variants of those fields are the hydrated catalog state. They are not hardcoded content. They are populated from `projects.json` or preview data, which is why the same script can run for Maurice's site and also for someone else's freshly installed builder output.

searchableEntries is the in-memory search index. Every item in it has a title, type, context, text, optional project id, optional section index, optional URL, and normalized text. The search functions score and display these entries.

assistantChatHistory and assistantSourceMap are the AI interface memory. assistantChatHistory keeps recent conversation so follow-up questions make sense. assistantSourceMap maps clickable source buttons in an answer back to portfolio search results so a visitor can jump to the relevant project or file.

currentSearchResults and searchResultLimit are the active search session. They remember what the latest query produced and how many items the user wants to see. Without these values, pressing Enter, selecting a result, or changing the result limit would not know which result list is current.

sectionRouteKey and sectionRouteParams are the browser-history contract for project windows. They keep the public URL connected to the currently opened project section and nested subsection. The values are small, but they are what let Refresh reopen the same project view and Back return to the previous view.

Chapter 1: Syntax, Runtime, And Framework Foundations

Runtime context: script.js runs in the public website visitor's browser. It cannot run Git or touch the local filesystem. Its power comes from the DOM, browser history, fetch, forms, dialogs, and event listeners.

Analogy: *index.html creates the switches and displays, styles.css paints them, and script.js is the wiring harness that makes clicks, searches, project windows, and chat messages do something.*

DOM anchors: The first part of the file captures DOM anchors with document.querySelector. Those anchors are not decorative variables; they are the live handles to the project grid, search box, filters, profile fields, contact area, resume panel, AI assistant, and footer. Later functions use those handles to populate the page after projects.json loads.

Opening DOM anchors and public runtime state in script.js

```
const grid = document.querySelector("#project-grid");
const searchInput = document.querySelector("#project-search");
const projectFilters = document.querySelector("#project-filters");
let filterButtons = [...document.querySelectorAll(".filter-button")];
const projectCount = document.querySelector("#project-count");
const projectTrackCount = document.querySelector("#project-track-count");
const projectTrackLabels = document.querySelector("#project-track-labels");
const year = document.querySelector("#year");
const funFactsCallout = document.querySelector("#fun-facts-callout");
const heroEyebrow = document.querySelector(".hero-content .eyebrow");
const heroTitle = document.querySelector("#hero-title");
const heroCopy = document.querySelector(".hero-copy");
const brandName = document.querySelector(".brand strong");
const brandSubtitle = document.querySelector(".brand small");
const brandIcon = document.querySelector(".brand-icon");
const brandText = document.querySelector(".omb-engraving");
const headerAvatar = document.querySelector(".header-avatar");
const headerAvatarImage = document.querySelector(".header-avatar img");
const heroImage = document.querySelector(".hero-image");
const resumeSection = document.querySelector("#resume");
const contactBand = document.querySelector("#contact");
const profilePhoto = document.querySelector(".profile-photo");
const contactTitle = document.querySelector("#contact-title");
const contactIntro = document.querySelector(".contact-intro");
const contactDetails = document.querySelector(".contact-details");
const contactLinks = document.querySelector(".contact-links");
const footerOwner = document.querySelector(".site-footer span");
const searchWrap = searchInput?.closest(".search-wrap");
const aiAssistantForm = document.querySelector("#ai-assistant-form");
const aiAssistantPanel = document.querySelector("#ask-ai");
const aiAssistantInput = document.querySelector("#ai-assistant-input");
const aiAssistantLog = document.querySelector("#ai-assistant-log");
const aiAssistantStatus = document.querySelector("#ai-assistant-status");
const aiClearChatButton = document.querySelector("#ai-clear-chat");

let categories = [];
let projects = [];
let siteSections = [];
let funFacts = [];
let funFactsRich = null;
let fieldStyles = {};
let siteContent = null;
let siteContentRich = {};
let profile = null;
let profileRich = {};
let activeFilter = "all";
let activeSectionDialogDrag = null;
let activeSectionDialogResize = null;
let sectionDialogDragEnabled = false;
let isApplyingSectionRoute = false;
let searchPanel = null;
let searchStatus = null;
let searchLimitSelect = null;
let currentSearchResults = [];
let assistantSourceCounter = 0;
let assistantSourceMap = new Map();
let assistantChatHistory = [];
let searchableEntries = [];
```

```

let searchResultLimit = 10;
let searchHighlightTimer = 0;
const searchHighlightName = "portfolio-search-highlight";

const sectionRouteKey = "portfolio-section";
const sectionRouteParams = {
  path: "portfolioPath",
  project: "portfolioProject",
  section: "portfolioSection"
};

const supportedCodeLanguages = [
  { id: "c", label: "C", aliases: ["c"] },
  { id: "cpp", label: "C++", aliases: ["cpp", "c++", "cplusplus"] },
  { id: "verilog", label: "Verilog", aliases: ["verilog", "v"] },
  { id: "systemverilog", label: "SystemVerilog", aliases: ["systemverilog", "system verilog", "sv"] },
  { id: "ltspice", label: "LTspice", aliases: ["ltspice", "spice", "cir", "net", "asc"] },
  { id: "java", label: "Java", aliases: ["java"] },
  { id: "javascript", label: "JavaScript", aliases: ["javascript", "js", "mjs"] },
  { id: "python", label: "Python", aliases: ["python", "py"] },
  { id: "html", label: "HTML", aliases: ["html", "htm"] }
];

const legacyTemplateSkins = {
  "skin-light-blue": "appearance-light-blue-red-click",
  "skin-clean-white": "appearance-white-blue-click",
  "skin-deep-navy": "appearance-deep-navy-cyan-click",
  "skin-red-warm": "appearance-warm-red-pale-click",
  "skin-emerald-instrument": "appearance-emerald-mint-click",
  "skin-amber-power": "appearance-amber-cream-click",
  "skin-violet-mixed": "appearance-violet-lilac-click",
  "skin-graphite-asic": "appearance-graphite-white-click",

```

Function Syntax Rules Used Later

Reading rule: The later chapters include function entry points and exact source blocks, but they do not re-explain the same syntax over and over. Read `async` as a marker that the function may wait for file work, network work, Git, compilers, or AI responses. Read parameters as the caller's contract with the function. Read default values after `=` as fallback behavior. Read destructured parameters in braces as one object being unpacked into named fields. Read `const`, `let`, and `var` as places where the function or file stores information while it works.

Compact syntax example used as a reference for later chapters

```

async function exampleOperation({ projectId, fileName = "main.c" }, options = {}) {
  const target = resolveInsideCompileRoot(projectId, fileName);
  return await runProcess("tool", [target], options);
}

```

How to use later chapters: After this chapter, a function section will only explain what is specific to that function: why it exists, what data it receives, what exact code implements it, what helpers it calls, and what local objects or variables carry the work.

Chapter 2: Top-Level Objects And Variables

This chapter explains the long-lived objects and variables before the function walkthrough. Think of these top-level values as the workshop benches, labeled drawers, shared measuring tools, and safety rails of the file. A function can walk into the workshop, use these shared objects, and leave without recreating the whole environment from scratch. They are explained by meaning and use, not by line number.

Dom Reference Or Ui State

grid

grid points at the project-grid container. renderProjects writes category sections and project cards into this element, making it the public project's visible mounting point. It anchors UI behavior to a specific browser element or window state. Functions that reference it include mediaGrid, renderSiteSections, parsedChildCards, projectCard, renderProjects, restoreSectionRouteAfterCatalogLoad, loadProjectCatalog.

Source excerpt for grid

```
const grid = document.querySelector("#project-grid");
```

searchInput

searchInput points at the project search box. Search handlers read its value, build dropdown results, apply highlights, and decide which projects remain visible. It anchors UI behavior to a specific browser element or window state. Functions that reference it include queueSearchHighlights, ensureSearchPanel, renderSearchResults, updateSearchDropdown, showSearchPanelIfNeeded, renderProjects, restoreSectionRouteAfterCatalogLoad.

Source excerpt for searchInput

```
const searchInput = document.querySelector("#project-search");
```

filterButtons

filterButtons stores the current filter button elements. It is updated after dynamic filters are rendered because the original NodeList would not include newly created buttons. It anchors UI behavior to a specific browser element or window state. Functions that reference it include setActiveFilter, bindFilterButtons.

Source excerpt for filterButtons

```
let filterButtons = [...document.querySelectorAll(".filter-button")];
```

aiClearChatButton

aiClearChatButton is the explicit Clear chat control. It resets conversation history, source maps, and the visible log. It anchors UI behavior to a specific browser element or window state. Functions that reference it include updateAssistantPanelGrowth, restoreSectionRouteAfterCatalogLoad.

Source excerpt for aiClearChatButton

```
const aiClearChatButton = document.querySelector("#ai-clear-chat");
```

Portfolio Data State

projectFilters

projectFilters points at the category filter container. renderCategoryFilters fills it with filter buttons derived from the actual catalog categories. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderCategoryFilters.

Source excerpt for projectFilters

```
const projectFilters = document.querySelector("#project-filters");
```

projectCount

projectCount displays the number of visible or total projects. It is updated after catalog load and render operations so the summary band matches the data. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderProjects.

Source excerpt for projectCount

```
const projectCount = document.querySelector("#project-count");
```

projectTrackCount

projectTrackCount displays how many project categories/tracks exist. It is derived from the saved categories rather than hardcoded. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderCategoryFilters.

Source excerpt for projectTrackCount

```
const projectTrackCount = document.querySelector("#project-track-count");
```

projectTrackLabels

projectTrackLabels displays the category names. It helps the top summary reflect Analog, Digital, Embedded, or any custom categories the builder creates. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderCategoryFilters.

Source excerpt for projectTrackLabels

```
const projectTrackLabels = document.querySelector("#project-track-labels");
```

resumeSection

resumeSection is the public resume panel. renderProfile hides or shows it depending on whether a resume URL exists. Functions that reference it include renderProfile.

Source excerpt for resumeSection

```
const resumeSection = document.querySelector("#resume");
```

profilePhoto

profilePhoto is the large contact-section profile image. It is separate from the header avatar so desktop/mobile layouts can size them differently. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderProfile.

Source excerpt for profilePhoto

```
const profilePhoto = document.querySelector(".profile-photo");
```

projects

projects is the runtime project array. Rendering, search, assistant context, and project windows all read from it. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include addSiteSectionSearchEntries, rebuildSearchIndex, assistantNamedProjectIds, assistantQuestionHasPortfolioIntent, assistantQuestionTokens, assistantNamedProjectTopicTokens, assistantContextForQuestion, assistantFallbackResults, assistantLocalAnswer, sectionRouteUrl.

Source excerpt for projects

```
let projects = [];
```

siteSections

siteSections stores custom main-page sections such as personal profile, hobbies, sports, social media, or professional material. Functions that reference it include addSiteSectionSearchEntries, assistantPublicProfileLinks, assistantContextForQuestion, renderSiteSections, loadProjectCatalog.

Source excerpt for siteSections

```
let siteSections = [];
```

profile

profile stores owner identity: name, email, phone, images, resume, GitHub, LinkedIn, and other public contact details. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderProfile, isWebsiteLinkItem, projectResponsiveClass, responsiveStyleValues, addSiteSectionSearchEntries, assistantUrlsFromTextValue, assistantPublicProfileLinks, assistantQuestionAllowsPublicLookup, assistantContextForQuestion, assistantFallbackResults.

Source excerpt for profile

```
let profile = null;
```

profileRich

profileRich stores rich versions of profile/contact fields so styled text can survive from builder to website. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Functions that reference it include renderProfile, loadProjectCatalog.

Source excerpt for profileRich

```
let profileRich = {};
```

activeSectionDialogDrag

activeSectionDialogDrag stores the drag state while a project window is being moved. It anchors UI behavior to a specific browser element or window state. Functions that reference it include beginSectionDialogDrag, moveSectionDialogDrag, endSectionDialogDrag.

Source excerpt for activeSectionDialogDrag

```
let activeSectionDialogDrag = null;
```

activeSectionDialogResize

activeSectionDialogResize stores the resize state while a project window edge is being dragged. It anchors UI behavior to a specific browser element or window state. Functions that reference it include beginSectionDialogResize, moveSectionDialogResize, endSectionDialogResize.

Source excerpt for activeSectionDialogResize

```
let activeSectionDialogResize = null;
```

sectionDialogDragEnabled

sectionDialogDragEnabled prevents drag handlers from being installed more than once. It anchors UI behavior to a specific browser element or window state. Functions that reference it include enableSectionDialogDrag.

Source excerpt for sectionDialogDragEnabled

```
let sectionDialogDragEnabled = false;
```

isApplyingSectionRoute

isApplyingSectionRoute prevents browser history updates from recursively triggering themselves while a saved route is being restored. Functions that reference it include syncSectionRouteToHistory, clearSectionRouteInHistory, applySectionRoute.

Source excerpt for isApplyingSectionRoute

```
let isApplyingSectionRoute = false;
```

sectionRouteKey

sectionRouteKey identifies portfolio section entries in browser history state. Functions that reference it include sectionRouteState, sectionRouteFromState.

Source excerpt for sectionRouteKey

```
const sectionRouteKey = "portfolio-section";
```

sectionRouteParams

sectionRouteParams defines the URL parameter names for project id, section index, and nested subsection path. Visible object fields include path, project, section. Those fields form the contract consumed by related functions. Functions that reference it include sectionBaseUrl, sectionRouteUrl, sectionRouteFromLocation.

Source excerpt for sectionRouteParams

```
const sectionRouteParams = {
  path: "portfolioPath",
  project: "portfolioProject",
  section: "portfolioSection"
};
```

defaultProfile

defaultProfile is the empty-owner fallback. It defines all expected profile fields so renderProfile can run even before the user enters contact details. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Visible object fields include brandImage, brandText, contactIntro, displayName, email, githubUrl, heroImage, linkedinUrl, phone, portfolioLabel, profileImage, resumeUrl, websiteUrl. Those fields form the contract consumed by related functions. Functions that reference it include normalizeProfile.

Source excerpt for defaultProfile

```
const defaultProfile = {
  brandImage: "",
  brandText: "Portfolio",
  contactIntro: "",
  displayName: "",
  email: "",
  githubUrl: "",
  heroImage: "",
  linkedinUrl: "",
  phone: "",
  portfolioLabel: "Portfolio",
  profileImage: "",
  resumeUrl: "",
  websiteUrl: ""
};
```

Temporary Calculation

year

year points at the footer year span. Startup code can set it from the current date so the footer remains current without manual HTML edits. Functions that reference it include renderProfile.

Source excerpt for year

```
const year = document.querySelector("#year");
```

funFactsCallout

funFactsCallout is the near-top personality callout. renderFunFacts only reveals it when saved fun facts exist. Functions that reference it include renderFunFacts.

Source excerpt for funFactsCallout

```
const funFactsCallout = document.querySelector("#fun-facts-callout");
```

heroEyebrow

heroEyebrow is the small label above the hero title. renderSiteContent can replace it from the builder's front page settings. Functions that reference it include normalizeSiteContent, renderSiteContent, addSiteSectionSearchEntries.

Source excerpt for heroEyebrow

```
const heroEyebrow = document.querySelector(".hero-content .eyebrow");
```

heroTitle

heroTitle is the main public headline. It receives the owner-controlled front-page title instead of forcing a fixed sentence into every portfolio. Functions that reference it include normalizeSiteContent, renderSiteContent, addSiteSectionSearchEntries.

Source excerpt for heroTitle

```
const heroTitle = document.querySelector("#hero-title");
```

heroCopy

heroCopy is the supporting paragraph in the hero section. It is one of the fields affected by rich text and plain text styling choices. Functions that reference it include normalizeSiteContent, renderSiteContent, addSiteSectionSearchEntries.

Source excerpt for heroCopy

```
const heroCopy = document.querySelector(".hero-copy");
```

brandName

brandName is the bold brand text in the header. renderProfile updates it to match the portfolio owner or site title. Functions that reference it include renderProfile.

Source excerpt for brandName

```
const brandName = document.querySelector(".brand strong");
```

brandSubtitle

brandSubtitle is the small subtitle beside the brand. It usually communicates local builder/public portfolio context or a tagline. Functions that reference it include renderProfile.

Source excerpt for brandSubtitle

```
const brandSubtitle = document.querySelector(".brand small");
```

brandIcon

brandIcon is the image element for the OMB visual mark. Profile settings can change it when the builder is generalized for other users. Functions that reference it include renderProfile.

Source excerpt for brandIcon

```
const brandIcon = document.querySelector(".brand-icon");
```

brandText

brandText is the OMB engraving text in the header logo area. It carries the small trademark-like label. Functions that reference it include normalizeProfile, renderProfile.

Source excerpt for brandText

```
const brandText = document.querySelector(".omb-engraving");
```

headerAvatar

headerAvatar is the clickable top-right profile image container. renderProfile reveals it only when a profile image exists. Functions that reference it include renderProfile.

Source excerpt for headerAvatar

```
const headerAvatar = document.querySelector(".header-avatar");
```

headerAvatarImage

headerAvatarImage is the actual img element inside the avatar link. It receives the saved profile picture URL. Functions that reference it include renderProfile.

Source excerpt for headerAvatarImage

```
const headerAvatarImage = document.querySelector(".header-avatar img");
```

heroImage

heroImage is the main hero/background image element. Site content settings decide whether it is displayed. Functions that reference it include normalizeProfile, renderProfile.

Source excerpt for heroImage

```
const heroImage = document.querySelector(".hero-image");
```

contactBand

contactBand is the final dark contact area. It also contains the AI assistant panel, so layout code must keep chat growth from distorting contact information. Functions that reference it include renderProfile.

Source excerpt for contactBand

```
const contactBand = document.querySelector("#contact");
```

contactTitle

contactTitle is the heading for the contact panel. renderProfile updates it from the owner identity. Functions that reference it include renderProfile.

Source excerpt for contactTitle

```
const contactTitle = document.querySelector("#contact-title");
```

contactIntro

contactIntro is the contact-section introduction text. It can be empty, plain text, or rich content depending on saved profile data. Functions that reference it include normalizeProfile, renderProfile, addSiteSectionSearchEntries.

Source excerpt for contactIntro

```
const contactIntro = document.querySelector(".contact-intro");
```

contactDetails

contactDetails is the list of direct contact methods such as email and phone. renderProfile builds clickable mailto and tel links inside it. Functions that reference it include renderProfile.

Source excerpt for contactDetails

```
const contactDetails = document.querySelector(".contact-details");
```

contactLinks

contactLinks is the container for external profile links such as GitHub, LinkedIn, resume, or custom websites. Functions that reference it include renderProfile.

Source excerpt for contactLinks

```
const contactLinks = document.querySelector(".contact-links");
```

footerOwner

footerOwner is the footer ownership text. It lets the footer follow the current profile owner rather than remaining generic. Functions that reference it include renderProfile.

Source excerpt for footerOwner

```
const footerOwner = document.querySelector(".site-footer span");
```

searchWrap

searchWrap is the parent around the search box. The search dropdown attaches near it so results appear where the user is typing. Functions that reference it include ensureSearchPanel.

Source excerpt for searchWrap

```
const searchWrap = searchInput?.closest(".search-wrap");
```

categories

categories is the runtime copy of project categories loaded from the catalog. It can include more than the original Analog, Digital, and Embedded groups. Functions that reference it include slugLabel, setActiveFilter, renderCategoryFilters, addSiteSectionSearchEntries, assistantContextForQuestion, assistantFallbackResults, projectCard, renderProjects, openParsedSection, loadProjectCatalog.

Source excerpt for categories

```
let categories = [];
```

funFacts

funFacts stores simple fun fact strings. renderFunFacts turns at most a few lines into a near-top callout. Functions that reference it include renderFunFacts, loadProjectCatalog.

Source excerpt for funFacts

```
let funFacts = [];
```

funFactsRich

funFactsRich stores rich-format fun facts when the builder saves styled content instead of plain strings. Functions that reference it include renderFunFacts, loadProjectCatalog.

Source excerpt for funFactsRich

```
let funFactsRich = null;
```

siteContent

siteContent stores front-page text and site-level settings such as hero copy and titles. Functions that reference it include renderSiteContent, loadProjectCatalog.

Source excerpt for siteContent

```
let siteContent = null;
```

siteContentRich

siteContentRich stores rich versions of site-level text fields. Functions that reference it include renderSiteContent, loadProjectCatalog.

Source excerpt for siteContentRich

```
let siteContentRich = {};
```

activeFilter

activeFilter stores the currently selected project category filter. Functions that reference it include projectVisible, setActiveFilter, renderCategoryFilters, renderProjects.

Source excerpt for activeFilter

```
let activeFilter = "all";
```

searchPanel

searchPanel stores the dropdown result panel created for search. Functions that reference it include ensureSearchPanel, renderSearchResults, goToSearchResult, restoreSectionRouteAfterCatalogLoad.

Source excerpt for searchPanel

```
let searchPanel = null;
```

searchStatus

searchStatus stores the status element inside the search dropdown. Functions that reference it include ensureSearchPanel, setSearchMessage.

Source excerpt for searchStatus

```
let searchStatus = null;
```

searchLimitSelect

searchLimitSelect stores the dropdown control that lets the visitor choose 5, 10, 15, or 20 results. Functions that reference it include ensureSearchPanel.

Source excerpt for searchLimitSelect

```
let searchLimitSelect = null;
```

currentSearchResults

currentSearchResults is the current ranked search-result array. It lets click handlers navigate results without recalculating the search. Functions that reference it include ensureSearchPanel, renderSearchResults, restoreSectionRouteAfterCatalogLoad.

Source excerpt for currentSearchResults

```
let currentSearchResults = [];
```

searchableEntries

searchableEntries is the full in-browser search index. It contains project titles, sections, files, captions, page sections, uploaded text, and electronics keyword expansions. Functions that reference it include rebuildSearchIndex, searchResultsFor.

Source excerpt for searchableEntries

```
let searchableEntries = [];
```

searchResultLimit

searchResultLimit stores how many dropdown results to display. Functions that reference it include ensureSearchPanel, renderSearchResults.

Source excerpt for searchResultLimit

```
let searchResultLimit = 10;
```

searchHighlightTimer

searchHighlightTimer debounces highlight updates so typing does not repaint the page too aggressively. Functions that reference it include queueSearchHighlights.

Source excerpt for searchHighlightTimer

```
let searchHighlightTimer = 0;
```

searchHighlightName

searchHighlightName is the CSS class applied to highlighted search matches. Functions that reference it include clearSearchHighlights, applySearchHighlights.

Source excerpt for searchHighlightName

```
const searchHighlightName = "portfolio-search-highlight";
```

defaultSiteContent

defaultSiteContent is the fallback front-page copy for a blank install. It keeps the site readable before the builder user customizes hero text and site messaging. Visible object fields include heroCopy, heroEyebrow, heroTitle. Those fields form the contract consumed by related functions. Functions that reference it include normalizeSiteContent.

Source excerpt for defaultSiteContent

```
const defaultSiteContent = {  
  heroCopy: "Add your projects, documents, diagrams, source code, images, profile details, and links.\nSave drafts locally, preview the site, then publish when your target repository is ready.",  
  heroEyebrow: "Engineering portfolio",  
  heroTitle: "Build a portfolio that presents your work clearly."  
};
```

Ai Configuration Or Conversation State

aiAssistantForm

aiAssistantForm is the chat form. Its submit handler starts answerAssistantQuestion. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include updateAssistantPanelGrowth, setAssistantBusy, restoreSectionRouteAfterCatalogLoad.

Source excerpt for aiAssistantForm

```
const aiAssistantForm = document.querySelector("#ai-assistant-form");
```

aiAssistantPanel

aiAssistantPanel is the whole Ask My Portfolio panel. Growth and layout code use it to keep the assistant right-justified and scrollable. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include assistantDesktopDockedHeight, updateAssistantPanelGrowth.

Source excerpt for aiAssistantPanel

```
const aiAssistantPanel = document.querySelector("#ask-ai");
```

aiAssistantInput

aiAssistantInput is the question field. It is cleared/focused by chat handlers and intentionally does not keep long instructional text inside the typed value. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include setAssistantBusy, clearAssistantChat, answerAssistantQuestion, restoreSectionRouteAfterCatalogLoad.

Source excerpt for aiAssistantInput

```
const aiAssistantInput = document.querySelector("#ai-assistant-input");
```

aiAssistantLog

aiAssistantLog is the scrollable message history. appendAssistantMessage and replaceAssistantMessage write user and assistant messages into it. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include updateAssistantPanelGrowth, appendAssistantMessage, scrollAssistantAnswerToStart, clearAssistantChat, restoreSectionRouteAfterCatalogLoad.

Source excerpt for aiAssistantLog

```
const aiAssistantLog = document.querySelector("#ai-assistant-log");
```

aiAssistantStatus

aiAssistantStatus is the small status line used for ready/thinking/error states. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include setAssistantStatus.

Source excerpt for aiAssistantStatus

```
const aiAssistantStatus = document.querySelector("#ai-assistant-status");
```

assistantSourceCounter

assistantSourceCounter creates stable ids for assistant source buttons. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include assistantSourcesMarkup, clearAssistantChat.

Source excerpt for assistantSourceCounter

```
let assistantSourceCounter = 0;
```

assistantSourceMap

assistantSourceMap connects source button ids to source records so clicking a source opens the right place. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or

context selection. Functions that reference it include `assistantSourcesMarkup`, `clearAssistantChat`, `restoreSectionRouteAfterCatalogLoad`.

Source excerpt for `assistantSourceMap`

```
let assistantSourceMap = new Map();
```

Appearance Configuration

`fieldStyles`

`fieldStyles` stores plain-field font, size, and color choices. It lets front-page, profile, and contact fields render consistently with builder formatting. Functions that reference it include `applyPlainFieldStyle`, `plainFieldStyleAttribute`, `loadProjectCatalog`.

Source excerpt for `fieldStyles`

```
let fieldStyles = {};
```

`legacyTemplateSkins`

`legacyTemplateSkins` maps older template ids to the newer appearance-only template ids. It keeps older saved projects rendering after template behavior changed. Visible object fields include `skin-light-blue`, `skin-clean-white`, `skin-deep-navy`, `skin-red-warm`, `skin-emerald-instrument`, `skin-amber-power`, `skin-violet-mixed`, `skin-graphite-asic`, `analog-opamp-topology`, `analog-power-charger`, `analog-filter-front-end`, `analog-sensor-interface`, `analog-mixed-signal-timing`, `digital-fpga-pipeline`, `digital-asic-block`, `digital-verification-suite`, `digital-interface-controller`, `digital-signal-processing`. Those fields form the contract consumed by related functions. Functions that reference it include `canonicalTemplateId`.

Source excerpt for `legacyTemplateSkins`

```
const legacyTemplateSkins = {
  "skin-light-blue": "appearance-light-blue-red-click",
  "skin-clean-white": "appearance-white-blue-click",
  "skin-deep-navy": "appearance-deep-navy-cyan-click",
  "skin-red-warm": "appearance-warm-red-pale-click",
  "skin-emerald-instrument": "appearance-emerald-mint-click",
  "skin-amber-power": "appearance-amber-cream-click",
  "skin-violet-mixed": "appearance-violet-lilac-click",
  "skin-graphite-asic": "appearance-graphite-white-click",
  "analog-opamp-topology": "appearance-light-blue-red-click",
  "analog-power-charger": "appearance-amber-cream-click",
  "analog-filter-front-end": "appearance-light-blue-red-click",
  "analog-sensor-interface": "appearance-emerald-mint-click",
  "analog-mixed-signal-timing": "appearance-violet-lilac-click",
  "digital-fpga-pipeline": "appearance-deep-navy-cyan-click",
  "digital-asic-block": "appearance-graphite-white-click",
  "digital-verification-suite": "appearance-graphite-white-click",
  "digital-interface-controller": "appearance-deep-navy-cyan-click",
  "digital-signal-processing": "appearance-violet-lilac-click",
  "embedded-stm32-sensor": "appearance-emerald-mint-click",
  "embedded-rtos-control": "appearance-graphite-white-click",
  "embedded-power-monitor": "appearance-amber-cream-click",
  "embedded-wireless-node": "appearance-deep-navy-cyan-click",
  "embedded-motor-control": "appearance-violet-lilac-click"
};
```

Runtime State Or Cache

`assistantChatHistory`

`assistantChatHistory` stores recent user and assistant messages. It gives the chatbot conversation memory. It influences assistant behavior: conversation memory, source display, model prompts, backend routing, or context selection. Functions that reference it include `assistantRemember`, `clearAssistantChat`, `answerAssistantQuestion`.

Source excerpt for `assistantChatHistory`

```
let assistantChatHistory = [];
```

Compiler Or Language Configuration

supportedCodeLanguages

supportedCodeLanguages is the public language list for code rendering and detection. It names C, C++, Verilog/SystemVerilog, Java, JavaScript, Python, HTML, and related aliases. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Functions that reference it include normalizeCodeLanguage, codeLanguageLabel.

Source excerpt for supportedCodeLanguages

```
const supportedCodeLanguages = [
  { id: "c", label: "C", aliases: ["c"] },
  { id: "cpp", label: "C++", aliases: ["cpp", "c++", "cplusplus"] },
  { id: "verilog", label: "Verilog", aliases: ["verilog", "v"] },
  { id: "systemverilog", label: "SystemVerilog", aliases: ["systemverilog", "system verilog", "sv"] },
  { id: "ltspice", label: "LTspice", aliases: ["ltspice", "spice", "cir", "net", "asc"] },
  { id: "java", label: "Java", aliases: ["java"] },
  { id: "javascript", label: "JavaScript", aliases: ["javascript", "js", "mjs"] },
  { id: "python", label: "Python", aliases: ["python", "py"] },
  { id: "html", label: "HTML", aliases: ["html", "htm"] }
];
```

Chapter 3: Function Walkthrough With Source Blocks

This walkthrough is function-by-function. Each section now follows a textbook rhythm: first the idea of the function, then the syntax, then how to read that syntax, then the implementation and the local objects that make the implementation work. Line numbers are deliberately avoided because the source will keep changing.

Public runtime utility

The functions in this group all support public runtime utility. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function normalize(value)

Role: To understand normalize, start with the problem it owns. normalize standardizes value data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalize(value) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for normalize

```
function normalize(value) {
  return String(value || "").toLowerCase();
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `normalize` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `normalize`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `normalize`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `sourceLooksCpp(source = "")`

Role: Begin with the role of `sourceLooksCpp`. `sourceLooksCpp` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sourceLooksCpp(source = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `source` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `sourceLooksCpp`

```
function sourceLooksCpp(source = "") {  
  const text = String(source || "");  
  return /#include\s*<(?:iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|  
variant)>/.test(text) ||  
  /\b(std::|cout|cin|cerr|namespace|template\s*<|class\s+\w+|new\s+\w+|delete\s+|constexpr|nullptr|  
using\s+namespace)\b/.test(text);  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `sourceLooksCpp` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `sourceLooksCpp`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside `sourceLooksCpp` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

text: `text` stores a computed value for temporary calculation. It is initialized from `String(source || "")`. Within the function it is passed into `test`; assigned or updated later in the function.

function sourceLooksC(source = "")

Role: Begin with the role of sourceLooksC. sourceLooksC belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sourceLooksC(source = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. source is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for sourceLooksC

```
function sourceLooksC(source = "") {  
  const text = String(source || "");  
  return /#include\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) ||  
    /\b(printf|scanf|malloc|calloc|realloc|free|struct\s+\w+|typedef\s+struct)\b/.test(text);  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means sourceLooksC performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without sourceLooksC, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside sourceLooksC explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

text: text stores a computed value for temporary calculation. It is initialized from String(source || ""). Within the function it is passed into test; assigned or updated later in the function.

function textBlocksFromPlainText(text)

Role: Begin with the role of textBlocksFromPlainText. textBlocksFromPlainText belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function textBlocksFromPlainText(text) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. text is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for textBlocksFromPlainText

```
function textBlocksFromPlainText(text) {  
  return [{  
    align: "left",  
    fontFamily: "Arial",  
    fontSize: "normal",  
    text: String(text || ""),  
    type: "paragraph"  
  }];  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means textBlocksFromPlainText performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without textBlocksFromPlainText, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside textBlocksFromPlainText, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function unwrapFormula(value)

Role: The best entry point for unwrapFormula is its responsibility in the surrounding workflow. unwrapFormula belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function unwrapFormula(value) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for unwrapFormula

```
function unwrapFormula(value) {  
  const text = String(value || "").trim();  
  const wrappers = [  
    [/^$\$(\[s\S]+\)\$\$/ , 1],  
    [/^\\\[([s\S]+)\\\[$/ , 1],  
    [/^\\\[([s\S]+)\\\[$/ , 1],  
    [/^$\$([s\S]+)\$/ , 1]  
  ];  
  for (const [pattern, group] of wrappers) {  
    const match = text.match(pattern);  
    if (match) return match[group].trim();  
  }  
  return text;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means unwrapFormula performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without unwrapFormula, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The easiest way to follow the body of unwrapFormula is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

text: text stores a computed value for temporary calculation. It is initialized from `String(value || "").trim()`. Within the function it is returned to the caller; accesses properties/methods `match`; assigned or updated later in the function.

wrappers: wrappers stores a list for temporary calculation. It is initialized from `[]`. Within the function it is assigned or updated later in the function.

destructured [pattern, group]: destructured `[pattern, group]` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. It unpacks API handles from another object, giving the rest of the file short direct names for those APIs.

match: match stores a computed value for temporary calculation. It is initialized from `text.match(pattern)`. Within the function it is returned to the caller; assigned or updated later in the function.

function looksLikeBareWebAddress(value = "")

Role: The best entry point for looksLikeBareWebAddress is its responsibility in the surrounding workflow. looksLikeBareWebAddress belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function looksLikeBareWebAddress(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for looksLikeBareWebAddress

```
function looksLikeBareWebAddress(value = "") {  
  const clean = String(value || "").trim();  
  if (!clean || /\s/.test(clean) || /^(\.?\.\.?\|\/|#|mailto:|tel:)/i.test(clean)) return false;  
  if (/^[a-z][a-z0-9+.-]*:/i.test(clean)) return false;  
  const host = clean.split(/[/?#]/)[0].toLowerCase();  
  if (!/^[a-z0-9-]+\.[a-z0-9-]+\.[a-z0-9-]+\.[a-z0-9-]+$/i.test(host)) return false;  
  return !/\.(pdf|docx?|xlsx?|pptx?|zip|7z|rar|png|jpe?g|gif|svg|webp|txt|md|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|v|sv|vhd|?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb)$/i.test(host);  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `looksLikeBareWebAddress` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `looksLikeBareWebAddress`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The easiest way to follow the body of `looksLikeBareWebAddress` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(value || "").trim()`. Within the function it is accessed; properties/methods `split`; passed into `test`; assigned or updated later in the function.

host: `host` stores a computed value for temporary calculation. It is initialized from `clean.split(/[/?#]/)[0].toLowerCase()`. Within the function it is passed into `test`; assigned or updated later in the function.

function displayTitle(value, fallback = "Untitled item")

Role: Begin with the role of `displayTitle`. `displayTitle` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function displayTitle(value, fallback = "Untitled item") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `fallback` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for displayTitle

```
function displayTitle(value, fallback = "Untitled item") {  
  const title = String(value || fallback).trim();  
  const replacements = {  
    "Brief": "Overview",  
    "Project Brief": "Overview",  
    "Design Brief": "Design Overview",  
    "Simulation Brief": "Simulation Overview"  
  };  
  return replacements[title] || title;  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `displayTitle` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `displayTitle`, the specific rule it owns would disappear from the named workflow. Callers would either skip

that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside `displayTitle` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

title: `title` stores a computed value for temporary calculation. It is initialized from `String(value || fallback).trim()`. Within the function it is assigned or updated later in the function.

replacements: `replacements` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; assigned or updated later in the function.

function slugLabel(value)

Role: The best entry point for `slugLabel` is its responsibility in the surrounding workflow. `slugLabel` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function slugLabel(value) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for slugLabel

```
function slugLabel(value) {  
  const category = categories.find((item) => item.id === value);  
  return category ? category.label : value;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `slugLabel` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `slugLabel`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The easiest way to follow the body of `slugLabel` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

category: `category` stores a function or callback value for portfolio data state. It is initialized from `categories.find((item) => item.id === value)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is returned to the caller; accesses properties/methods `label`; assigned or updated later in the function.

function parsedItemTerms(item)

Role: To understand `parsedItemTerms`, start with the problem it owns. `parsedItemTerms` interprets `dltemTerms` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Function entry point:

```
function parsedItemTerms(item) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `parsedItemTerms`

```
function parsedItemTerms(item) {  
  return [  
    item?.title,  
    item?.description,  
    item?.meta,  
    item?.url,  
    item?.kind,  
    ...(item?.rich?.blocks || []).flatMap((block) => [block.text, block.formula, block.title, block.caption]),  
    ...(item?.children || []).flatMap(parsedItemTerms)  
  ];  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `parsedItemTerms` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `parsedItemTerms`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `parsedItemTerms`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function pillList(items, className = "")`

Role: To understand `pillList`, start with the problem it owns. `pillList` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function pillList(items, className = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `items` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `className` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for pillList

```
function pillList(items, className = "") {  
  return (items || []).  
    .map((item) => {  
      const label = typeof item === "string" ? item : item.name || item.title || item.label || "";  
      return label ? `    })  
    .join("");  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means pillList performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without pillList, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: After the main flow, the working values inside pillList reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

label: label stores a computed value for temporary calculation. It is initialized from `typeof item === "string" ? item : item.name || item.title || item.label || ""`. Within the function it is returned to the caller; assigned or updated later in the function.

function evidenceList(items, renderItem, emptyMessage)

Role: The best entry point for evidenceList is its responsibility in the surrounding workflow. evidenceList belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function evidenceList(items, renderItem, emptyMessage) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. items is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. renderItem is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. emptyMessage is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for evidenceList

```
function evidenceList(items, renderItem, emptyMessage) {  
  if (!items || !items.length) {  
    return `  }  
  
  return `}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and

returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means evidenceList performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without evidenceList, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside evidenceList, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function detailBlock(title, className, content)

Role: Read detailBlock first as a named idea, not as syntax. detailBlock belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function detailBlock(title, className, content) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. title is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. className is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. content is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for detailBlock

```
function detailBlock(title, className, content) {  
  return  
    <details class="{className}">  
      <summary>{title}</summary>  
      {content}  
    </details>  
  ;  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means detailBlock performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without detailBlock, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside detailBlock, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function mediaGrid(items)

Role: Read mediaGrid first as a named idea, not as syntax. mediaGrid belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function mediaGrid(items) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. items is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for mediaGrid

```
function mediaGrid(items) {  
  if (!items || !items.length) {  
    return `

No project images have been added yet.

`;  
  }  
  
  return `  
    <div class="media-grid">  
      ${items.map((item) => `  
        <figure>  
          <a href="${escapeHtml(normalizeLinkTarget(item.url, { assumeWeb: isWebsiteLinkItem(item, item.url) })))}"$  
{linkAttributes(item.url, item)}>  
              
          </a>  
          <figcaption>  
            <strong>${escapeHtml(item.title)}</strong>  
            <span>${escapeHtml(item.caption || "")}</span>  
          </figcaption>  
        </figure>  
      `).join("")}  
    </div>  
  `;  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The function also has to be read through the helpers it calls. It works with escapeHtml, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes. Those calls show that mediaGrid is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without mediaGrid, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside mediaGrid, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function itemLabel(item = {}, fallback = "Download file")

Role: The best entry point for itemLabel is its responsibility in the surrounding workflow. itemLabel belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function itemLabel(item = {}, fallback = "Download file") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `fallback` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `itemLabel`

```
function itemLabel(item = {}, fallback = "Download file") {  
  const url = itemUrl(item);  
  return item.title || item.label || item.name || item.caption || fileNameFromUrl(url) || fallback;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `itemUrl`, `fileNameFromUrl`. Those calls show that `itemLabel` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `itemLabel`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The easiest way to follow the body of `itemLabel` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

url: `url` stores a computed value for network or routing value. It is initialized from `itemUrl(item)`. Within the function it is passed into `fileNameFromUrl`; assigned or updated later in the function.

`function parsedChildCards(children, projectId, sectionIndex, basePath = [])`

Role: Begin with the role of `parsedChildCards`. `parsedChildCards` interprets `dChildCards` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Function entry point:

```
function parsedChildCards(children, projectId, sectionIndex, basePath = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `children` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `basePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or

malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `parsedChildCards`

```
function parsedChildCards(children, projectId, sectionIndex, basePath = []) {  
  const visibleChildren = children.filter((child) => nodeHasRenderableContent(child) && !child.url && !  
    nodeIsOverview(child));  
  if (!visibleChildren.length) return "";  
  return `  
    <div class="subsection-grid section-content-grid">  
      ${children.map((child, index) => nodeHasRenderableContent(child) && !child.url && !nodeIsOverview(child) ?  
        parsedNodeCard(child, projectId, sectionIndex, [...basePath, index]) : "").join("")}  
    </div>  
  `;  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `nodeHasRenderableContent`, `nodeIsOverview`, `parsedNodeCard`. Those calls show that `parsedChildCards` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `parsedChildCards`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside `parsedChildCards` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

visibleChildren: `visibleChildren` stores a function or callback value for temporary calculation. It is initialized from `children.filter((child) => nodeHasRenderableContent(child) && !child.url && !nodeIsOverview(child))`. Within the function it is accesses properties/methods `length`; assigned or updated later in the function.

`function escapeRegExp(value = "")`

Role: Begin with the role of `escapeRegExp`. `escapeRegExp` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function escapeRegExp(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `escapeRegExp`

```
function escapeRegExp(value = "") {  
  return String(value).replace(/[\.\*\?\^\$\{\}\|\[\]\\\]/g, "\\$&");  
}
```

```
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `escapeRegExp` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `escapeRegExp`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `escapeRegExp`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `scrollToDomTarget(id)`

Role: To understand `scrollToDomTarget`, start with the problem it owns. `scrollToDomTarget` belongs to public runtime utility. This helper supports the public site runtime by normalizing values or preparing data for another renderer. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function scrollToDomTarget(id) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `id` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `scrollToDomTarget`

```
function scrollToDomTarget(id) {  
  const target = id ? document.getElementById(id) : null;  
  if (!target) return false;  
  closeSectionDialogForRoute();  
  target.scrollIntoView({ behavior: "smooth", block: "start" });  
  try {  
    history.replaceState(history.state || { appWindow: "portfolio-base" }, "", `#${id}`);  
  } catch {  
    // Scrolling is still the important behavior.  
  }  
  queueSearchHighlights();  
  return true;  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser route/history. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: updates browser history so Back, Forward, and refresh can follow the same window state.

Collaborators: Next, trace the helper calls that surround the function. It works with `closeSectionDialogForRoute`, `queueSearchHighlights`. Those calls show that `scrollToDomTarget` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `scrollToDomTarget`, the specific rule it owns would disappear from the named workflow. Callers would either skip

that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: After the main flow, the working values inside `scrollToDomTarget` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

target: target stores a DOM element reference. It is initialized from `id ? document.getElementById(id) : null`. Within the function it is accesses properties/methods `scrollIntoView`; assigned or updated later in the function.

Profile and site content

The functions in this group all support profile and site content. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function normalizeFunFacts(value)

Role: The best entry point for `normalizeFunFacts` is its responsibility in the surrounding workflow. `normalizeFunFacts` standardizes FunFacts data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeFunFacts(value) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for normalizeFunFacts

```
function normalizeFunFacts(value) {  
  const items = Array.isArray(value) ? value : String(value || "").split(/\r?\n/);  
  return items  
    .map((item) => String(item || "").replace(/\s+/g, " ").trim())  
    .filter(Boolean)  
    .slice(0, 10);  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `normalizeFunFacts` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `normalizeFunFacts`, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Local values: The easiest way to follow the body of `normalizeFunFacts` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

items: items stores a computed value for temporary calculation. It is initialized from `Array.isArray(value) ? value : String(value || "").split(/\r?\n/)`. Within the function it is returned to the caller; assigned or updated later in the function.

function normalizeSiteContent(content = {})

Role: To understand normalizeSiteContent, start with the problem it owns. normalizeSiteContent standardizes SiteContent data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeSiteContent(content = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. content is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for normalizeSiteContent

```
function normalizeSiteContent(content = {}) {  
  return {  
    heroCopy: String(content.heroCopy || "").trim() || defaultSiteContent.heroCopy,  
    heroEyebrow: String(content.heroEyebrow || "").trim() || defaultSiteContent.heroEyebrow,  
    heroTitle: String(content.heroTitle || "").trim() || defaultSiteContent.heroTitle  
  };  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means normalizeSiteContent performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without normalizeSiteContent, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Local values: Inside normalizeSiteContent, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function normalizeProfile(profileValue = {})

Role: Read normalizeProfile first as a named idea, not as syntax. normalizeProfile standardizes Profile data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeProfile(profileValue = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. profileValue points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string

data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for normalizeProfile

```
function normalizeProfile(profileValue = {}) {  
  return {  
    brandImage: String(profileValue.brandImage || "").trim(),  
    brandText: String(profileValue.brandText || "").trim() || String(profileValue.displayName || "").trim() ||  
    defaultProfile.brandText,  
    contactIntro: String(profileValue.contactIntro || "").trim(),  
    displayName: String(profileValue.displayName || "").trim(),  
    email: String(profileValue.email || "").trim(),  
    githubUrl: String(profileValue.githubUrl || "").trim(),  
    heroImage: String(profileValue.heroImage || "").trim(),  
    linkedinUrl: String(profileValue.linkedinUrl || "").trim(),  
    phone: String(profileValue.phone || "").trim(),  
    portfolioLabel: String(profileValue.portfolioLabel || "").trim() || defaultProfile.portfolioLabel,  
    profileImage: String(profileValue.profileImage || "").trim(),  
    resumeUrl: String(profileValue.resumeUrl || "").trim(),  
    websiteUrl: String(profileValue.websiteUrl || "").trim()  
  };  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means normalizeProfile performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without normalizeProfile, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Local values: Inside normalizeProfile, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function mailComposeLink(email = "")

Role: The best entry point for mailComposeLink is its responsibility in the surrounding workflow. mailComposeLink belongs to profile and site content. This function renders or normalizes owner identity, front-page text, contact links, fun facts, or custom main-page sections. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function mailComposeLink(email = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. email is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for mailComposeLink

```
function mailComposeLink(email = "") {  
  const clean = String(email || "").trim();  
  return clean ? `https://mail.google.com/mail/?view=cm&fs=1&to=${encodeURIComponent(clean)}` : "";  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and

returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means mailComposeLink performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without mailComposeLink, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Local values: The easiest way to follow the body of mailComposeLink is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(email || "").trim()`.

Within the function it is returned to the caller; passed into `encodeURIComponent`; assigned or updated later in the function.

function phoneLink(phone = "")

Role: Begin with the role of phoneLink. phoneLink belongs to profile and site content. This function renders or normalizes owner identity, front-page text, contact links, fun facts, or custom main-page sections. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function phoneLink(phone = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. phone is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for phoneLink

```
function phoneLink(phone = "") {  
  const clean = String(phone || "").trim();  
  return clean ? `tel:${clean.replace(/[\^d+]/g, "")}` : "";  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means phoneLink performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without phoneLink, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Local values: The local state inside phoneLink explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(phone || "").trim()`.

Within the function it is returned to the caller; accesses properties/methods `replace`; assigned or updated later in the function.

function looksLikeWebOrContactText(value = "")

Role: To understand looksLikeWebOrContactText, start with the problem it owns.

looksLikeWebOrContactText belongs to profile and site content. This function renders or normalizes owner identity, front-page text, contact links, fun facts, or custom main-page sections. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function looksLikeWebOrContactText(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for looksLikeWebOrContactText

```
function looksLikeWebOrContactText(value = "") {  
  const clean = String(value || "").trim();  
  if (!clean) return false;  
  if (/^b(?:https?:\/\|www\.)[^<"]+/i.test(clean)) return true;  
  if (/^b[a-z0-9_%-]+\@[a-z0-9-]+\.[a-z]{2,}\b/i.test(clean)) return true;  
  if (/^b(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com\/[^<"]+/i.test(clean)) return true;  
  if (looksLikeBareWebAddress(clean)) return true;  
  return clean.split(/\s+/).some((word) => looksLikeBareWebAddress(word.replace(/^[('"]+|([)]"'.,;:!?]+$/g, "")));  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with looksLikeBareWebAddress. Those calls show that looksLikeWebOrContactText is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without looksLikeWebOrContactText, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Local values: After the main flow, the working values inside looksLikeWebOrContactText reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from String(value || "").trim().

Within the function it is returned to the caller; accesses properties/methods split; passed into test; assigned or updated later in the function.

function isWebsiteLinkItem(item = {}, target = "")

Role: Begin with the role of isWebsiteLinkItem. isWebsiteLinkItem is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

Function entry point:

```
function isWebsiteLinkItem(item = {}, target = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `target` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `isWebsiteLinkItem`

```
function isWebsiteLinkItem(item = {}, target = "") {
  const clean = String(target || "").trim();
  if (/^(https?:)?\:\/\/i.test(clean) || /^www\./i.test(clean)) return true;
  const typeText = [item.kind, item.type, item.status, item.meta, item.label, item.title]
    .filter(Boolean)
    .join(" ")
    .toLowerCase();
  return /\b(link|website|web link|url|external|github|linkedin|drive|social|profile)\b/.test(typeText);
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `isWebsiteLinkItem` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `isWebsiteLinkItem`, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Local values: The local state inside `isWebsiteLinkItem` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(target || "").trim()`. Within the function it is passed into `test`; assigned or updated later in the function.

typeText: `typeText` stores a list for temporary calculation. It is initialized from `[item.kind, item.type, item.status, item.meta, item.label, item.title]`. Within the function it is passed into `test`; assigned or updated later in the function.

function `inferResponsiveProfile(project)`

Role: The best entry point for `inferResponsiveProfile` is its responsibility in the surrounding workflow. `inferResponsiveProfile` makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

Function entry point:

```
function inferResponsiveProfile(project) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string*

data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for inferResponsiveProfile

```
function inferResponsiveProfile(project) {
  const sections = project?.portfolioView?.sections || [];
  const visibleSections = sections.filter((section) => section?.id !== "brief" &&
sectionHasRenderableContent(section));
  const itemCount = visibleSections.reduce(
    (count, section) => count + (section.items || []).reduce((sum, item) => sum + responsiveNodeCount(item), 0),
    0
  );
  const maxDepth = visibleSections.reduce(
    (depth, section) => Math.max(depth, ...(section.items || []).map(responsiveNodeDepth), 1),
    1
  );
  const hasMedia = visibleSections.some((section) => responsiveNodeHasMedia(section));
  const density = itemCount >= 18 || visibleSections.length >= 6 || maxDepth >= 4
    ? "dense"
    : itemCount <= 6 && visibleSections.length <= 2
    ? "simple"
    : "balanced";
  return {
    version: 1,
    breakpoints: {
      mobile: 640,
      tablet: 900,
      desktop: 1180
    },
    cardLayout: hasPublicTemplate(project) ? "single" : "split",
    contentMax: hasMedia || maxDepth >= 3 ? "1180px" : "980px",
    density,
    mediaMax: hasMedia ? "860px" : "720px",
    sectionCardMin: density === "dense" ? "210px" : density === "simple" ? "280px" : "240px",
    touchTarget: "44px",
    windowMode: "full-screen"
  };
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `sectionHasRenderableContent`, `responsiveNodeCount`, `responsiveNodeHasMedia`, `hasPublicTemplate`. Those calls show that `inferResponsiveProfile` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `inferResponsiveProfile`, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Local values: The easiest way to follow the body of `inferResponsiveProfile` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

sections: `sections` stores a computed value for portfolio data state. It is initialized from `project?.portfolioView?.sections || []`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`; assigned or updated later in the function.

visibleSections: `visibleSections` stores a function or callback value for portfolio data state. It is initialized from `sections.filter((section) => section?.id !== "brief" && sectionHasRenderableContent(section))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `reduce`, `some`; assigned or updated later in the function.

itemCount: `itemCount` stores a computed value for temporary calculation. It is initialized from `visibleSections.reduce()`. Within the function it is assigned or updated later in the function.

maxDepth: maxDepth stores a computed value for temporary calculation. It is initialized from visibleSections.reduce(. Within the function it is assigned or updated later in the function.

hasMedia: hasMedia stores a function or callback value for temporary calculation. It is initialized from visibleSections.some((section) => responsiveNodeHasMedia(section)). Within the function it is assigned or updated later in the function.

density: density stores a computed value for temporary calculation. It is initialized from itemCount >= 18 || visibleSections.length >= 6 || maxDepth >= 4. Within the function it is assigned or updated later in the function.

function projectResponsiveProfile(project)

Role: Read projectResponsiveProfile first as a named idea, not as syntax. projectResponsiveProfile handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Function entry point:

```
function projectResponsiveProfile(project) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for projectResponsiveProfile

```
function projectResponsiveProfile(project) {  
  const savedProfile = project?.portfolioView?.responsive;  
  if (savedProfile?.version) return { ...inferResponsiveProfile(project), ...savedProfile };  
  return inferResponsiveProfile(project);  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: The function also has to be read through the helpers it calls. It works with inferResponsiveProfile. Those calls show that projectResponsiveProfile is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without projectResponsiveProfile, owner-specific content would remain generic or malformed. Contact links, profile text, fun facts, and custom sections would not render with the intended public shape.

Local values: The local objects and variables inside projectResponsiveProfile show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

savedProfile: savedProfile stores a computed value for portfolio data state. It is initialized from project?.portfolioView?.responsive. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

Rich content rendering

The functions in this group all support rich content rendering. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function renderFunFacts()

Role: Read renderFunFacts first as a named idea, not as syntax. renderFunFacts turns saved data for FunFacts into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Function entry point:

```
function renderFunFacts() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for renderFunFacts

```
function renderFunFacts() {
  if (!funFactsCallout) return;
  const facts = normalizeFunFacts(funFacts);
  const hasRichFacts = Boolean(funFactsRich?.blocks?.length);
  funFactsCallout.hidden = !facts.length && !hasRichFacts;
  funFactsCallout.innerHTML = facts.length || hasRichFacts ? `
    <div class="fun-facts-window">
      <span class="fun-facts-label">Fun fact:</span>
      <div class="fun-facts-lines">
        ${hasRichFacts
          ? renderRichContent({ blocks: funFactsRich.blocks.slice(0, 3) }, facts.join("\n"))
          : facts.map((fact) => `<p class="fun-fact-line">${renderInlineMath(fact)}</p>`).join("")}
        </div>
      </div>
    ` : "";
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Collaborators: The function also has to be read through the helpers it calls. It works with normalizeFunFacts, renderRichContent, renderInlineMath. Those calls show that renderFunFacts is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without renderFunFacts, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: The local objects and variables inside renderFunFacts show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

facts: facts stores a computed value for temporary calculation. It is initialized from normalizeFunFacts(funFacts). Within the function it is iterated or transformed as a collection; accesses properties/methods join, length, map; assigned or updated later in the function.

hasRichFacts: hasRichFacts stores a computed value for temporary calculation. It is initialized from Boolean(funFactsRich?.blocks?.length). Within the function it is assigned or updated later in the function.

function escapeHtml(value)

Role: Read escapeHtml first as a named idea, not as syntax. escapeHtml belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function escapeHtml(value) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for escapeHtml

```
function escapeHtml(value) {  
  return String(value || "")  
    .replaceAll("&", "&amp;")  
    .replaceAll("<", "&lt;")  
    .replaceAll(">", "&gt;")  
    .replaceAll("'", "&quot;")  
    .replaceAll('"', "&#039;");  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means escapeHtml performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without escapeHtml, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: Inside escapeHtml, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function normalizeCodeLanguage(value = "")

Role: Begin with the role of normalizeCodeLanguage. normalizeCodeLanguage standardizes CodeLanguage data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeCodeLanguage(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for normalizeCodeLanguage

```
function normalizeCodeLanguage(value = "") {  
  const clean = String(value || "").trim().toLowerCase().replace(/[_-]+/g, " ");  
  const match = supportedCodeLanguages.find((language) =>  
    language.id === clean || language.aliases.includes(clean)  
  );  
  return match?.id || "javascript";  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means normalizeCodeLanguage performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without normalizeCodeLanguage, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: The local state inside normalizeCodeLanguage explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from String(value || "").trim().toLowerCase().replace(/[_-]+/g, " "). Within the function it is passed into includes; assigned or updated later in the function.

match: match stores a function or callback value for temporary calculation. It is initialized from supportedCodeLanguages.find((language) => . Within the function it is returned to the caller; assigned or updated later in the function.

function codeLanguageLabel(value = "")

Role: The best entry point for codeLanguageLabel is its responsibility in the surrounding workflow. codeLanguageLabel belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function codeLanguageLabel(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for codeLanguageLabel

```
function codeLanguageLabel(value = "") {  
  return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value))?.label || "Code";  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `normalizeCodeLanguage`. Those calls show that `codeLanguageLabel` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `codeLanguageLabel`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: Inside `codeLanguageLabel`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function normalizeCodePasteMode(value = "")

Role: Read `normalizeCodePasteMode` first as a named idea, not as syntax. `normalizeCodePasteMode` standardizes `CodePasteMode` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeCodePasteMode(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `normalizeCodePasteMode`

```
function normalizeCodePasteMode(value = "") {  
  return value === "source" ? "source" : "basic";  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `normalizeCodePasteMode` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `normalizeCodePasteMode`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: Inside `normalizeCodePasteMode`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function normalizeCodeText(value = "", pasteMode = "source")

Role: To understand `normalizeCodeText`, start with the problem it owns. `normalizeCodeText` standardizes `CodeText` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeCodeText(value = "", pasteMode = "source") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `pasteMode` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `normalizeCodeText`

```
function normalizeCodeText(value = "", pasteMode = "source") {
  const normalized = String(value || "").replace(/\r\n?/g, "\n").replace(/\u00a0/g, " ");
  if (normalizeCodePasteMode(pasteMode) === "source") return normalized.replace(/\t/g, " ").replace(/\s+$/g, "");
  return normalized
    .split("\n")
    .map((line) => line.trimEnd())
    .join("\n")
    .replace(/\n{4,}/g, "\n\n\n\n")
    .trim();
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalizeCodePasteMode`. Those calls show that `normalizeCodeText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `normalizeCodeText`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: After the main flow, the working values inside `normalizeCodeText` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

normalized: `normalized` stores a computed value for temporary calculation. It is initialized from `String(value || "").replace(/\r\n?/g, "\n").replace(/\u00a0/g, " ")`. Within the function it is returned to the caller; accesses properties/methods `replace`; assigned or updated later in the function.

`function detectCodeLanguage(value = "")`

Role: Begin with the role of `detectCodeLanguage`. `detectCodeLanguage` makes an educated classification from incomplete evidence. It looks at names, extensions, source text, repository state, or runtime state and returns the app's best decision so the next operation can choose the correct path.

Function entry point:

```
function detectCodeLanguage(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string

data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for detectCodeLanguage

```
function detectCodeLanguage(value = "") {
  const code = String(value || "");
  if (/<\/?[a-z][\s\S]*?>/i.test(code) || /<!doctype\s+html/i.test(code)) return "html";
  if (/b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|class\s+
  \w+)\b/.test(code)) return "systemverilog";
  if (/b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b/.test(code)) return "verilog";
  if (/^\s*\.(?!(tran|ac|dc|op|model|subckt|ends|param)\b/im.test(code) || /\bV\w+\s+\w+\s+\w+\s+(\?:DC|SIN|
  PULSE)?/i.test(code)) return "ltspice";
  if (/b(def|elif|import\s+\w+|from\s+\w+\s+import|self|None|True|False)\b/.test(code)) return "python";
  if (/b(public\s+class|private\s+|protected\s+|static\s+void\s+main|System\.out)\b/.test(code)) return "java";
  if (sourceLooksCpp(code) || sourceLooksC(code) || /\b(#include|main\s*\(|puts\s*\(|fprintf|sizeof\s*\(|
  \b/.test(code)) {
    return sourceLooksCpp(code) ? "cpp" : "c";
  }
  if (/b(const|let|var|function|=>|console\.log|document\.|window\.)\b/.test(code)) return "javascript";
  return "javascript";
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `sourceLooksC`, `sourceLooksCpp`. Those calls show that `detectCodeLanguage` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `detectCodeLanguage`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: The local state inside `detectCodeLanguage` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

code: code stores a computed value for temporary calculation. It is initialized from `String(value || "")`. Within the function it is passed into `sourceLooksC`, `sourceLooksCpp`, `test`; assigned or updated later in the function.

function tokenizedCodeHtml(code = "", language = "javascript")

Role: To understand `tokenizedCodeHtml`, start with the problem it owns. `tokenizedCodeHtml` is the lightweight syntax highlighter used by the public site. It escapes the code first, then wraps comments, strings, numbers, keywords, preprocessor directives, and other recognized tokens in spans with CSS classes. It supports the major languages that the builder exposes, including C, C++, SystemVerilog, LTspice-oriented text, Java, JavaScript, Python, and HTML-like code. The function exists so code added to a project reads like code rather than plain paragraph text. Without it, source examples in the portfolio would be visually flat, harder to scan, and less convincing to technical recruiters.

Function entry point:

```
function tokenizedCodeHtml(code = "", language = "javascript") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `code` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `language` names the programming or hardware-description language. The function uses it to choose file extensions, highlighting rules, compilers, simulators, labels, or output behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string

data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for tokenizedCodeHtml

```
function tokenizeHtmlCode(code = "", language = "javascript") {
  let html = escapeHtml(code);
  const tokens = [];
  const protect = (pattern, className) => {
    html = html.replace(pattern, (match) => {
      const token = `@@CODE_TOKEN_${tokens.length}@@`;
      tokens.push(`>${code}</span>`);
  html = html.replace(/@@CODE_TOKEN_(\\d+)@@/g, (_, match, index) => tokens[Number(index)] || "");
  return html;
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `escapeHtml`, `normalizeCodeLanguage`. Those calls show that `tokenizedCodeHtml` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `tokenizedCodeHtml`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: After the main flow, the working values inside `tokenizedCodeHtml` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

html: `html` stores a computed value for temporary calculation. It is initialized from `escapeHtml(code)`. Within the function it is returned to the caller; accesses properties/methods `replace`; assigned or updated later in the function.

tokens: `tokens` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `length`, `push`; assigned or updated later in the function.

protect: `protect` stores a function or callback value for temporary calculation. It is initialized from `(pattern, className) => {}`. Within the function it is assigned or updated later in the function.

token: `token` stores a string value for temporary calculation. It is initialized from ``@@CODE_TOKEN_${tokens.length}@@``. Within the function it is returned to the caller; passed into `protect`; assigned or updated later in the function.

commentPattern: `commentPattern` stores a list for temporary calculation. It is initialized from `["python", "Itspice"].includes(normalizeCodeLanguage(language))`. Within the function it is passed into `protect`; assigned or updated later in the function.

keywordMap: `keywordMap` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is accesses properties/methods `javascript`; assigned or updated later in the function.

keywords: `keywords` stores a computed value for lookup collection. It is initialized from `keywordMap[normalizeCodeLanguage(language)] || keywordMap.javascript`. Within the function it is passed into `replace`; assigned or updated later in the function.

function renderRichCodeBlock(block = {})

Role: Begin with the role of `renderRichCodeBlock`. `renderRichCodeBlock` turns saved data for `RichCodeBlock` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Function entry point:

```
function renderRichCodeBlock(block = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `block` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for renderRichCodeBlock

```
function renderRichCodeBlock(block = {}) {
```

```

const language = normalizeCodeLanguage(block.language || detectCodeLanguage(block.code || ""));
const code = normalizeCodeText(block.code || "", block.pasteMode || "source");
return `
<figure class="rich-code-block rich-code-language-${language}">
  <figcaption><span>&lt;/span> ${escapeHtml(codeLanguageLabel(language))}</figcaption>
  <pre><code>${tokenizedCodeHtml(code, language)}</code></pre>
</figure>
`;
}

```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalizeCodeLanguage`, `detectCodeLanguage`, `normalizeCodeText`, `escapeHtml`, `codeLanguageLabel`, `tokenizedCodeHtml`. Those calls show that `renderRichCodeBlock` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `renderRichCodeBlock`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: The local state inside `renderRichCodeBlock` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

language: `language` stores a computed value for compiler or language configuration. It is initialized from `normalizeCodeLanguage(block.language || detectCodeLanguage(block.code || ""))`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `escapeHtml`, `normalizeCodeLanguage`, `tokenizedCodeHtml`; assigned or updated later in the function.

code: `code` stores a computed value for temporary calculation. It is initialized from `normalizeCodeText(block.code || "", block.pasteMode || "source")`. Within the function it is passed into `normalizeCodeLanguage`, `normalizeCodeText`, `tokenizedCodeHtml`; assigned or updated later in the function.

function renderPlainMultiline(value = "")

Role: The best entry point for `renderPlainMultiline` is its responsibility in the surrounding workflow. `renderPlainMultiline` turns saved data for `PlainMultiline` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Function entry point:

```
function renderPlainMultiline(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for renderPlainMultiline

```

function renderPlainMultiline(value = "") {
  return escapeHtml(value).replace(/\r\n?|\n/g, "<br>");
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and

returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `escapeHtml`. Those calls show that `renderPlainMultiline` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `renderPlainMultiline`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: Inside `renderPlainMultiline`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderSiteContent()

Role: Begin with the role of `renderSiteContent`. `renderSiteContent` turns saved data for `SiteContent` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Function entry point:

```
function renderSiteContent() { ... }
```

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for renderSiteContent

```
function renderSiteContent() {  
  const content = normalizeSiteContent(siteContent || {});  
  if (heroEyebrow) heroEyebrow.innerHTML = renderRichFieldContent(siteContentRich.heroEyebrow, content.heroEyebrow);  
  if (heroTitle) heroTitle.innerHTML = renderRichFieldContent(siteContentRich.heroTitle, content.heroTitle);  
  if (heroCopy) heroCopy.innerHTML = renderRichFieldContent(siteContentRich.heroCopy, content.heroCopy);  
  applyPlainFieldStyle(heroEyebrow, "site-hero-eyebrow");  
  applyPlainFieldStyle(heroTitle, "site-hero-title");  
  applyPlainFieldStyle(heroCopy, "site-hero-copy");  
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalizeSiteContent`, `renderRichFieldContent`, `applyPlainFieldStyle`. Those calls show that `renderSiteContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `renderSiteContent`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: The local state inside `renderSiteContent` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

content: `content` stores a computed value for temporary calculation. It is initialized from `normalizeSiteContent(siteContent || {})`. Within the function it is accesses properties/methods `heroCopy`, `heroEyebrow`, `heroTitle`; passed into `renderRichFieldContent`; assigned or updated later in the function.

function renderProfile()

Role: To understand renderProfile, start with the problem it owns. renderProfile applies owner identity to the public page. It updates the displayed name, tagline, profile photo, brand/title text, email link, phone link, GitHub/resume links, contact cards, resume viewer, metadata hints, and social/contact links. It is the bridge between saved profile data and the visible recruiter-facing contact section. The function is important because the same website shell can serve Maurice's portfolio or a fresh user's portfolio. Profile data cannot be hardcoded if the builder is intended to be reusable. Without renderProfile, the page would either remain generic or require manual HTML editing for every owner.

Function entry point:

```
function renderProfile() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for renderProfile

```
function renderProfile() {
  const current = normalizeProfile(profile || {});
  const displayName = current.displayName || "Portfolio";
  const label = current.portfolioLabel || "Portfolio";
  const emailLink = mailComposeLink(current.email);
  const callLink = phoneLink(current.phone);
  const links = [
    current.email ? { label: "Email", url: emailLink, external: true } : null,
    current.phone ? { label: "Call", url: callLink } : null,
    current.githubUrl ? { label: "GitHub", url: normalizeLinkTarget(current.githubUrl, { assumeWeb: true }),
external: true } : null,
    current.linkedinUrl ? { label: "LinkedIn", url: normalizeLinkTarget(current.linkedinUrl, { assumeWeb: true }),
external: true } : null,
    current.websiteUrl ? { label: "Website", url: normalizeLinkTarget(current.websiteUrl, { assumeWeb: true }),
external: true } : null,
    current.resumeUrl ? { label: "Resume", url: current.resumeUrl, external: false } : null
  ].filter(Boolean);

  document.title = `${displayName} | ${label}`;
  if (brandName) brandName.innerHTML = renderRichFieldContent(profileRich.displayName, displayName);
  if (brandSubtitle) brandSubtitle.innerHTML = renderRichFieldContent(profileRich.portfolioLabel, label);
  applyPlainFieldStyle(brandName, "profile-display-name");
  applyPlainFieldStyle(brandSubtitle, "profile-portfolio-label");
  if (brandText) brandText.textContent = current.brandText || displayName.slice(0, 3).toUpperCase() || "PORT";
  if (brandIcon) {
    if (current.brandImage) {
      brandIcon.src = current.brandImage;
      brandIcon.hidden = false;
    } else {
      brandIcon.hidden = true;
    }
  }

  if (headerAvatar && headerAvatarImage) {
    headerAvatar.hidden = !current.profileImage;
    if (current.profileImage) {
      headerAvatarImage.src = current.profileImage;
      headerAvatarImage.alt = displayName;
      headerAvatar.setAttribute("aria-label", `Go to ${displayName} contact information`);
    }
  }

  if (heroImage) {
    if (current.heroImage) {
      heroImage.hidden = false;
      heroImage.src = current.heroImage;
      heroImage.alt = `${displayName} portfolio background`;
    } else {
      heroImage.hidden = true;
    }
  }
}
```

```

document.querySelectorAll('a[href="#resume"]').forEach((link) => {
  link.hidden = !current.resumeUrl;
});
const heroGithubLink = [...document.querySelectorAll(".hero-actions a")]
  .find((link) => /github/i.test(link.textContent || ""));
if (heroGithubLink) {
  heroGithubLink.hidden = !current.githubUrl;
  if (current.githubUrl) heroGithubLink.href = normalizeLinkTarget(current.githubUrl, { assumeWeb: true });
}

if (resumeSection) {
  resumeSection.hidden = !current.resumeUrl;
  resumeSection.querySelectorAll("a").forEach((link) => {
    link.href = current.resumeUrl || "#";
  });
  const resumeObject = resumeSection.querySelector("object");
  if (resumeObject) resumeObject.data = current.resumeUrl || "";
  const fallbackLink = resumeSection.querySelector("object a");
  if (fallbackLink) fallbackLink.href = current.resumeUrl || "#";
}

if (profilePhoto) {
  profilePhoto.hidden = !current.profileImage;
  if (current.profileImage) {
    profilePhoto.src = current.profileImage;
    profilePhoto.alt = `Portrait of ${displayName}`;
  }
}
if (contactTitle) contactTitle.innerHTML = renderRichFieldContent(profileRich.displayName, displayName);
applyPlainFieldStyle(contactTitle, "profile-display-name");
if (contactIntro) {
  contactIntro.innerHTML = renderRichFieldContent(
    profileRich.contactIntro,
    current.contactIntro || (current.displayName ? `${displayName}'s contact information and public links.` : "Add contact details in the builder.")
  );
}
applyPlainFieldStyle(contactIntro, "profile-contact-intro");
if (contactDetails) {
  contactDetails.innerHTML = [
    current.email ? `` : "",
    current.phone ? `` : ""
  \\].filter\\(Boolean\\).join\\(""\\);
}
if \\(contactLinks\\) {
  contactLinks.innerHTML = links.map\\(\\(link\\) => {
    const styleId = link.label === "GitHub"
      ? "profile-github"
      : link.label === "LinkedIn"
      ? "profile-linkedin"
      : link.label === "Website"
      ? "profile-website"
      : link.label === "Email"
      ? "profile-email"
      : link.label === "Call"
      ? "profile-phone"
      : "";
    return `

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalizeProfile`, `mailComposeLink`, `phoneLink`, `normalizeLinkTarget`, `renderRichFieldContent`, `applyPlainFieldStyle`, `plainFieldStyleAttribute`. Those calls show that `renderProfile` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `renderProfile`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: After the main flow, the working values inside `renderProfile` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

current: `current` stores a computed value for temporary calculation. It is initialized from `normalizeProfile(profile || {})`. Within the function it is accesses properties/methods `brandImage`, `brandText`, `contactIntro`, `displayName`, `email`, `githubUrl`, `heroImage`, `linkedinUrl`; passed into `mailComposeLink`, `normalizeLinkTarget`, `phoneLink`, `renderRichFieldContent`; assigned or updated later in the function.

displayName: `displayName` stores a computed value for temporary calculation. It is initialized from `current.displayName || "Portfolio"`. Within the function it is accesses properties/methods `slice`; passed into `renderRichFieldContent`, `setAttribute`; assigned or updated later in the function.

label: `label` stores a computed value for temporary calculation. It is initialized from `current.portfolioLabel || "Portfolio"`. Within the function it is passed into `applyPlainFieldStyle`, `renderRichFieldContent`, `setAttribute`; assigned or updated later in the function.

emailLink: `emailLink` stores a computed value for temporary calculation. It is initialized from `mailComposeLink(current.email)`. Within the function it is assigned or updated later in the function.

callLink: `callLink` stores a computed value for temporary calculation. It is initialized from `phoneLink(current.phone)`. Within the function it is assigned or updated later in the function.

links: `links` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; passed into `renderRichFieldContent`; assigned or updated later in the function.

heroGithubLink: `heroGithubLink` stores a list for Git publishing and authorization state. It is initialized from `[...document.querySelectorAll(".hero-actions a")]`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is accesses properties/methods `hidden`, `href`; assigned or updated later in the function.

resumeObject: `resumeObject` stores a DOM element reference. It is initialized from `resumeSection.querySelector("object")`. Within the function it is accesses properties/methods `data`; assigned or updated later in the function.

fallbackLink: `fallbackLink` stores a DOM element reference. It is initialized from `resumeSection.querySelector("object a")`. Within the function it is accesses properties/methods `href`; assigned or updated later in the function.

styleId: `styleId` stores a computed value for appearance configuration. It is initialized from `link.label === "GitHub"`. Within the function it is passed into `plainFieldStyleAttribute`; assigned or updated later in the function.

function `renderInlineMath(text)`

Role: To understand `renderInlineMath`, start with the problem it owns. `renderInlineMath` turns saved data for `InlineMath` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Function entry point:

```
function renderInlineMath(text) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `renderInlineMath`

```
function renderInlineMath(text) {  
  const escaped = escapeHtml(text);  
  const withMath = escaped.replace(/\$([\^$]+)\$/g, '<span class="rich-inline-formula">$1</span>');  
  return withMath.replace(/\b(https?:\/\/[\^<]+|www\.[\^<]+|(? :github|linkedin|gitlab|bitbucket|drive|docs)  
  \.com\/[\^<]+)\/gi, (match) => {  
    const trailing = match.match(/[/,.,:;!]+$/)?.[0] || "";  
    const clean = trailing ? match.slice(0, -trailing.length) : match;  
    const href = normalizeLinkTarget(clean, { assumeWeb: true });  
    return `<a href="${href}" target="_blank" rel="noreferrer">${clean}</a>${trailing}`;  
  });  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Next, trace the helper calls that surround the function. It works with `escapeHtml`, `normalizeLinkTarget`. Those calls show that `renderInlineMath` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `renderInlineMath`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: After the main flow, the working values inside `renderInlineMath` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

escaped: `escaped` stores a computed value for temporary calculation. It is initialized from `escapeHtml(text)`. Within the function it is accesses properties/methods `replace`; assigned or updated later in the function.

withMath: `withMath` stores a computed value for temporary calculation. It is initialized from `escaped.replace(/\$([\^$]+)\$/g, '<span class="rich-inline-formula">$1</span>')`. Within the function it is returned to the caller; accesses properties/methods `replace`; assigned or updated later in the function.

trailing: `trailing` stores a computed value for temporary calculation. It is initialized from `match.match(/[/,.,:;!]+$/)`. Within the function it is accesses properties/methods `length`; passed into `slice`; assigned or updated later in the function.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `trailing ? match.slice(0, -trailing.length) : match`. Within the function it is passed into `normalizeLinkTarget`; assigned or updated later in the function.

href: `href` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(clean, { assumeWeb: true })`. Within the function it is assigned or updated later in the function.

function `renderMultilineInlineText(text = "")`

Role: The best entry point for `renderMultilineInlineText` is its responsibility in the surrounding workflow. `renderMultilineInlineText` turns saved data for `MultilineInlineText` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Function entry point:

```
function renderMultilineInlineText(text = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `renderMultilineInlineText`

```
function renderMultilineInlineText(text = "") {  
  return renderInlineMath(text).replace(/\\r\\n?|\\n/g, "<br>");  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `renderInlineMath`. Those calls show that `renderMultilineInlineText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `renderMultilineInlineText`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: Inside `renderMultilineInlineText`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function sanitizeRichInlineHtml(value = "")`

Role: To understand `sanitizeRichInlineHtml`, start with the problem it owns. `sanitizeRichInlineHtml` is the safety filter for rich inline HTML. Builder editors may store spans, links, bold text, colors, and other inline markup, but the public website should not blindly inject arbitrary HTML. This function removes unsafe tags and attributes while preserving the formatting the portfolio actually needs. The function prevents a content field from becoming a script injection surface. It works with `renderRichContent` and `renderRichFieldContent`, which need formatted HTML but should only receive controlled markup.

Function entry point:

```
function sanitizeRichInlineHtml(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `sanitizeRichInlineHtml`

```
function sanitizeRichInlineHtml(value = "") {  
  const template = document.createElement("template");
```

```

template.innerHTML = String(value || "");
const allowedTags = new Set(["A", "B", "BR", "EM", "I", "SPAN", "STRONG", "U"]);

[...template.content.querySelectorAll("*")].forEach((node) => {
  if (!allowedTags.has(node.tagName)) {
    node.replaceWith(...node.childNodes);
    return;
  }

  const href = node.tagName === "A" ? String(node.getAttribute("href") || "").trim() : "";
  const normalizedHref = normalizeLinkTarget(href, { assumeWeb: true });
  const safeHref = /^(https?:|mailto:|tel:|#|\/)/i.test(normalizedHref) ? normalizedHref : "";
  const fontFamily = cleanFontFamily(node.style.fontFamily || "");
  const fontPx = normalizeFontPx(parseFloat(node.style.fontSize || ""));
  const color = normalizeTextColor(node.style.color || "");
  const rawFontWeight = node.style.fontWeight || "";
  const fontWeight = /^(bold|[6-9]00)$/i.test(rawFontWeight)
    ? "700"
    : /^(normal|[1-5]00)$/i.test(rawFontWeight)
    ? "400"
    : "";
  const fontStyle = node.style.fontStyle === "italic" ? "italic" : "";
  const textDecoration = node.style.textDecoration.includes("underline") ? "underline" : "";

  [...node.attributes].forEach((attribute) => node.removeAttribute(attribute.name));

  if (node.tagName === "A" && safeHref) {
    node.setAttribute("href", safeHref);
    node.setAttribute("target", "_blank");
    node.setAttribute("rel", "noreferrer");
  }
  if (fontFamily) node.style.fontFamily = fontFamily;
  if (fontPx) node.style.fontSize = `${fontPx}px`;
  if (color) node.style.color = color;
  if (fontWeight) node.style.fontWeight = fontWeight;
  if (fontStyle) node.style.fontStyle = fontStyle;
  if (textDecoration) node.style.textDecoration = textDecoration;
});

linkifyRichTextNodes(template.content);
return template.innerHTML;
}

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: creates DOM elements instead of relying only on static HTML; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalizeLinkTarget`, `cleanFontFamily`, `normalizeFontPx`, `normalizeTextColor`, `linkifyRichTextNodes`. Those calls show that `sanitizeRichInlineHtml` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `sanitizeRichInlineHtml`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: After the main flow, the working values inside `sanitizeRichInlineHtml` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

template: `template` stores a DOM element reference. It is initialized from `document.createElement("template")`. Within the function it is returned to the caller; accesses properties/methods `content`, `innerHTML`; passed into `createElement`, `linkifyRichTextNodes`; assigned or updated later in the function.

allowedTags: `allowedTags` stores a constructed object for lookup collection. It is initialized from `new Set(["A", "B", "BR", "EM", "I", "SPAN", "STRONG", "U"])`. Within the function it is accesses properties/methods `has`; assigned or updated later in the function.

href: href stores a computed value for temporary calculation. It is initialized from `node.tagName === "A" ? String(node.getAttribute("href") || "").trim() : ""`. Within the function it is passed into `String`, `normalizeLinkTarget`, `setAttribute`; assigned or updated later in the function.

normalizedHref: normalizedHref stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(href, { assumeWeb: true })`. Within the function it is passed into `test`; assigned or updated later in the function.

safeHref: safeHref stores a computed value for temporary calculation. It is initialized from `/^(https?:|mailto:|tel:|#|V)/i.test(normalizedHref) ? normalizedHref : ""`. Within the function it is passed into `setAttribute`; assigned or updated later in the function.

fontFamily: fontFamily stores a computed value for temporary calculation. It is initialized from `cleanFontFamily(node.style.fontFamily || "")`. Within the function it is passed into `cleanFontFamily`; assigned or updated later in the function.

fontPx: fontPx stores a computed value for temporary calculation. It is initialized from `normalizeFontPx(parseFloat(node.style.fontSize || ""))`. Within the function it is assigned or updated later in the function.

color: color stores a computed value for appearance configuration. It is initialized from `normalizeTextColor(node.style.color || "")`. Within the function it is passed into `normalizeTextColor`; assigned or updated later in the function.

rawFontWeight: rawFontWeight stores a computed value for temporary calculation. It is initialized from `node.style.fontWeight || ""`. Within the function it is passed into `test`; assigned or updated later in the function.

fontWeight: fontWeight stores a computed value for temporary calculation. It is initialized from `/^(bold|[6-9]00)$/i.test(rawFontWeight)`. Within the function it is assigned or updated later in the function.

fontStyle: fontStyle stores a computed value for appearance configuration. It is initialized from `node.style.fontStyle === "italic" ? "italic" : ""`. Within the function it is assigned or updated later in the function.

textDecoration: textDecoration stores a computed value for temporary calculation. It is initialized from `node.style.textDecoration.includes("underline") ? "underline" : ""`. Within the function it is accesses properties/methods `includes`; assigned or updated later in the function.

function richTextStyle(block = {})

Role: Begin with the role of `richTextStyle`. `richTextStyle` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function richTextStyle(block = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `block` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for richTextStyle

```
function richTextStyle(block = {}) {  
  const styles = [];  
  const fontFamily = cleanFontFamily(block.fontFamily || "Arial") || "Arial";  
  const fontPx = normalizeFontPx(block.fontPx);  
  const color = normalizeTextColor(block.color || "");
```

```

    if (fontFamily) styles.push(`font-family: ${fontFamily}`);
    if (fontPx) styles.push(`font-size: ${fontPx}px`);
    if (color) styles.push(`color: ${color}`);
    if (block.bold) styles.push("font-weight: 700");
    if (block.italic) styles.push("font-style: italic");
    if (block.underline) styles.push("text-decoration: underline");

    return styles.length ? ` style="${escapeHtml(styles.join("; "))}"` : "";
}

```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `cleanFontFamily`, `normalizeFontPx`, `normalizeTextColor`, `escapeHtml`. Those calls show that `richTextStyle` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `richTextStyle`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: The local state inside `richTextStyle` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

styles: `styles` stores a list for appearance configuration. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `join`, `length`, `push`; passed into `escapeHtml`; assigned or updated later in the function.

fontFamily: `fontFamily` stores a computed value for temporary calculation. It is initialized from `cleanFontFamily(block.fontFamily || "Arial") || "Arial"`. Within the function it is passed into `cleanFontFamily`, `push`; assigned or updated later in the function.

fontPx: `fontPx` stores a computed value for temporary calculation. It is initialized from `normalizeFontPx(block.fontPx)`. Within the function it is passed into `normalizeFontPx`, `push`; assigned or updated later in the function.

color: `color` stores a computed value for appearance configuration. It is initialized from `normalizeTextColor(block.color || "")`. Within the function it is passed into `normalizeTextColor`, `push`; assigned or updated later in the function.

function richImageCropStyle(block = {})

Role: Read `richImageCropStyle` first as a named idea, not as syntax. `richImageCropStyle` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function richImageCropStyle(block = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `block` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for richImageCropStyle

```
function richImageCropStyle(block = {}) {
  const aspect = normalizeCropAspect(block.cropAspect);
  const styles = [
    `--crop-zoom: ${normalizeCropZoom(block.cropZoom)}`,
    `--crop-x: ${normalizeCropPosition(block.cropX)}%`,
    `--crop-y: ${normalizeCropPosition(block.cropY)}%`
  ];
  if (aspect !== "original") styles.push(`--crop-aspect: ${aspect}`);
  return `style="${escapeHtml(styles.join("; "))}"`;
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The function also has to be read through the helpers it calls. It works with `normalizeCropAspect`, `normalizeCropZoom`, `normalizeCropPosition`, `escapeHtml`. Those calls show that `richImageCropStyle` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `richImageCropStyle`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: The local objects and variables inside `richImageCropStyle` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

aspect: `aspect` stores a computed value for temporary calculation. It is initialized from `normalizeCropAspect(block.cropAspect)`. Within the function it is passed into `push`; assigned or updated later in the function.

styles: `styles` stores a list for appearance configuration. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; passed into `escapeHtml`; assigned or updated later in the function.

function richImageDownloadLink(block = {})

Role: The best entry point for `richImageDownloadLink` is its responsibility in the surrounding workflow. `richImageDownloadLink` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function richImageDownloadLink(block = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `block` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for richImageDownloadLink

```
function richImageDownloadLink(block = {}) {
  const label = escapeHtml(cleanRichImageTitle(block) || block.caption || "Image file");
  const url = block.url || "#";
  const target = normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(block, url) });
```

```

return `
<p class="rich-download-only">
  <a
    class="resource-link"
    href="${escapeHtml(target)}"
    ${linkAttributes(target, block)}
    ${downloadAttribute(target, block)}
  >
    ${label}
  </a>
</p>
`;
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `escapeHtml`, `cleanRichImageTitle`, `normalizeLinkTarget`, `isWebsiteLinkItem`, `linkAttributes`, `downloadAttribute`. Those calls show that `richImageDownloadLink` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `richImageDownloadLink`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: The easiest way to follow the body of `richImageDownloadLink` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

label: label stores a computed value for temporary calculation. It is initialized from `escapeHtml(cleanRichImageTitle(block) || block.caption || "Image file")`. Within the function it is assigned or updated later in the function.

url: url stores a computed value for network or routing value. It is initialized from `block.url || "#"`. Within the function it is passed into `normalizeLinkTarget`; assigned or updated later in the function.

target: target stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(block, url) })`. Within the function it is passed into `downloadAttribute`, `escapeHtml`, `linkAttributes`; assigned or updated later in the function.

function cleanRichImageTitle(block = {})

Role: Read `cleanRichImageTitle` first as a named idea, not as syntax. `cleanRichImageTitle` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function cleanRichImageTitle(block = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `block` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for cleanRichImageTitle

```

function cleanRichImageTitle(block = {}) {
  const title = String(block.title || "").trim();
  return /^pasted image\b/i.test(title) ? "" : title;
}

```

```
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `cleanRichImageTitle` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `cleanRichImageTitle`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: The local objects and variables inside `cleanRichImageTitle` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

title: title stores a computed value for temporary calculation. It is initialized from `String(block.title || "").trim()`. Within the function it is passed into `String, test`; assigned or updated later in the function.

function renderRichContent(rich, fallbackText = "")

Role: To understand `renderRichContent`, start with the problem it owns. `renderRichContent` is the renderer for builder-created rich content. It accepts blocks representing paragraphs, images, formulas, files, links, code blocks, captions, and styled text, then converts those blocks into safe public HTML. It decides whether an image should appear inline, whether a file should be downloadable, how captions attach, and how text styles survive into the website. This function matters because the builder is not just saving plain text. It supports formatted explanations, pasted images, formulas, code, and files. Without `renderRichContent`, much of what the builder collects would be flattened or lost when the portfolio is published.

Function entry point:

```
function renderRichContent(rich, fallbackText = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `rich` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `fallbackText` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for renderRichContent

```
function renderRichContent(rich, fallbackText = "") {
  const blocks = rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText);
  return `
    <div class="rich-content">
      ${blocks.map((block) => {
        const align = ["left", "center", "right"].includes(block.align) ? block.align : "left";
        if (block.type === "image") {
          if (block.display === "download") return richImageDownloadLink(block);
          const title = cleanRichImageTitle(block);
          const imageSrc = normalizeLinkTarget(block.url, { assumeWeb: true });
          return `
            <figure class="rich-image justify-${align}">
              <span class="rich-image-viewport crop-${normalizeCropAspect(block.cropAspect) === "original" ?
"original" : "active"}"${richImageCropStyle(block)}>
                
              </span>
            `
          }
        }
      })}
    </div>
  `
}
```

```

    `${(title || block.caption) ? `<figcaption>${title ? `<strong>${escapeHtml(title)}</strong>` : ""}${
    {block.caption ? `<span>${escapeHtml(block.caption)}</span>` : ""}</figcaption>` : ""}
    </figcaption>
    `;
  }
  if (block.type === "formula") {
    const formula = unwrapFormula(block.formula);
    if (looksLikeWebOrContactText(formula)) {
      return `<p class="rich-paragraph rich-text-normal text-${align}">${renderInlineMath(formula)}</p>`;
    }
    return `<div class="rich-formula justify-${align}">${escapeHtml(formula)}</div>`;
  }
  if (block.type === "code") return renderRichCodeBlock(block);
  const size = ["small", "normal", "large"].includes(block.fontSize) ? block.fontSize : "normal";
  const content = block.html
    ? sanitizeRichInlineHtml(block.html)
    : renderInlineMath(block.text || "");
  return `<p class="rich-paragraph rich-text-${size} text-${align}"${richTextStyle(block)}>${content}</p>`;
}).join("")
</div>
`;
}

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Next, trace the helper calls that surround the function. It works with `textBlocksFromPlainText`, `richImageDownloadLink`, `cleanRichImageTitle`, `normalizeLinkTarget`, `normalizeCropAspect`, `richImageCropStyle`, `escapeHtml`, `unwrapFormula`. Those calls show that `renderRichContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `renderRichContent`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: After the main flow, the working values inside `renderRichContent` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

blocks: `blocks` stores a computed value for temporary calculation. It is initialized from `rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText)`. Within the function it is iterated or transformed as a collection; accesses properties/methods map; assigned or updated later in the function.

align: `align` stores a list for temporary calculation. It is initialized from `["left", "center", "right"].includes(block.align) ? block.align : "left"`. Within the function it is passed into `includes`; assigned or updated later in the function.

title: `title` stores a computed value for temporary calculation. It is initialized from `cleanRichImageTitle(block)`. Within the function it is passed into `escapeHtml`; assigned or updated later in the function.

imageSrc: `imageSrc` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(block.url, { assumeWeb: true })`. Within the function it is passed into `escapeHtml`; assigned or updated later in the function.

formula: `formula` stores a computed value for temporary calculation. It is initialized from `unwrapFormula(block.formula)`. Within the function it is passed into `escapeHtml`, `renderInlineMath`, `unwrapFormula`; assigned or updated later in the function.

size: `size` stores a list for temporary calculation. It is initialized from `["small", "normal", "large"].includes(block.fontSize) ? block.fontSize : "normal"`. Within the function it is assigned or updated later in the function.

content: content stores a computed value for temporary calculation. It is initialized from block.html. Within the function it is assigned or updated later in the function.

function renderRichFieldContent(rich, fallbackText = "")

Role: To understand renderRichFieldContent, start with the problem it owns. renderRichFieldContent turns saved data for RichFieldContent into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Function entry point:

```
function renderRichFieldContent(rich, fallbackText = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. rich is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. fallbackText is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for renderRichFieldContent

```
function renderRichFieldContent(rich, fallbackText = "") {  
  const blocks = rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText);  
  return blocks.map((block) => {  
    if (block.type !== "paragraph") return "";  
    const align = ["left", "center", "right"].includes(block.align) ? block.align : "left";  
    const content = block.html  
      ? sanitizeRichInlineHtml(block.html)  
      : renderInlineMath(block.text || "");  
    return `  }).filter(Boolean).join("");  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Next, trace the helper calls that surround the function. It works with textBlocksFromPlainText, sanitizeRichInlineHtml, renderInlineMath, richTextStyle. Those calls show that renderRichFieldContent is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without renderRichFieldContent, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: After the main flow, the working values inside renderRichFieldContent reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

blocks: blocks stores a computed value for temporary calculation. It is initialized from rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText). Within the function it is returned to the caller; iterated or transformed as a collection; accesses properties/methods map; assigned or updated later in the function.

align: align stores a list for temporary calculation. It is initialized from ["left", "center", "right"].includes(block.align) ? block.align : "left". Within the function it is passed into includes; assigned or updated later in the function.

content: content stores a computed value for temporary calculation. It is initialized from block.html. Within the function it is assigned or updated later in the function.

function richTextTerms(rich)

Role: Read richTextTerms first as a named idea, not as syntax. richTextTerms belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function richTextTerms(rich) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. rich is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for richTextTerms

```
function richTextTerms(rich) {  
  return (rich?.blocks || []).flatMap((block) => [  
    block.text,  
    block.html,  
    block.formula,  
    block.code,  
    block.language,  
    block.title,  
    block.caption,  
    block.url  
  ]);  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means richTextTerms performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without richTextTerms, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: Inside richTextTerms, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderDynamicLinks(items = [])

Role: Read renderDynamicLinks first as a named idea, not as syntax. renderDynamicLinks turns saved data for DynamicLinks into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Function entry point:

```
function renderDynamicLinks(items = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. items is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more

predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for renderDynamicLinks

```
function renderDynamicLinks(items = []) {  
  const links = items.filter((item) => item?.url && (item.label || item.title));  
  if (!links.length) return "";  
  return `item.title || "Open")).join("")}</div>`;  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The function also has to be read through the helpers it calls. It works with resourceLink. Those calls show that renderDynamicLinks is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without renderDynamicLinks, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: The local objects and variables inside renderDynamicLinks show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

links: links stores a function or callback value for temporary calculation. It is initialized from items.filter((item) => item?.url && (item.label || item.title)). Within the function it is iterated or transformed as a collection; accesses properties/methods length, map; assigned or updated later in the function.

function collectRichDownloads(rich, fallbackType = "File", output = [])

Role: The best entry point for collectRichDownloads is its responsibility in the surrounding workflow. collectRichDownloads gathers scattered nested data into a shape that another function can loop over. This is important because portfolio data is hierarchical while rendering, search, and AI context often need a flat list.

Function entry point:

```
function collectRichDownloads(rich, fallbackType = "File", output = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. rich is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. fallbackType is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. output is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for collectRichDownloads

```
function collectRichDownloads(rich, fallbackType = "File", output = []) {
  (rich?.blocks || []).forEach((block) => {
    if (!block?.url) return;

    output.push({
      title: cleanRichImageTitle(block) || block.title || block.caption || fileNameFromUrl(block.url),
      url: block.url,
      status: "uploaded",
      type: block.type === "image" ? "Image" : fallbackType,
      description: block.caption || ""
    });
  });

  return output;
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `cleanRichImageTitle`, `fileNameFromUrl`. Those calls show that `collectRichDownloads` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `collectRichDownloads`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: Inside `collectRichDownloads`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderDownloadBlock(title, items = [])

Role: The best entry point for `renderDownloadBlock` is its responsibility in the surrounding workflow. `renderDownloadBlock` turns saved data for `DownloadBlock` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Function entry point:

```
function renderDownloadBlock(title, items = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `title` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `items` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for renderDownloadBlock

```
function renderDownloadBlock(title, items = []) {
  const downloads = uniqueDownloads(items);
  if (!downloads.length) return "";

  return detailBlock(escapeHtml(title), "evidence-block evidence-wide download-evidence-block", `
<ul class="download-list">
  ${downloads.map((item) => `
    <li>
      ${resourceLink(item, item.title)}
      <span>${escapeHtml(item.type || "File")} &middot; ${escapeHtml(item.status || "uploaded")}</span>
    </li>
  `)}
</ul>`);
}
```

```

    ${item.description ? `

${renderMultilineInlineText(item.description)}</p>` : ""}
  </li>
  `).join("")}
</ul>
`);
}


```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `uniqueDownloads`, `detailBlock`, `escapeHtml`, `resourceLink`, `renderMultilineInlineText`. Those calls show that `renderDownloadBlock` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `renderDownloadBlock`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: The easiest way to follow the body of `renderDownloadBlock` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

downloads: `downloads` stores a computed value for temporary calculation. It is initialized from `uniqueDownloads(items)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; assigned or updated later in the function.

function `renderParsedBriefBlock(section, fallbackSummary = "")`

Role: Read `renderParsedBriefBlock` first as a named idea, not as syntax. `renderParsedBriefBlock` turns saved data for `ParsedBriefBlock` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Function entry point:

```
function renderParsedBriefBlock(section, fallbackSummary = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `fallbackSummary` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `renderParsedBriefBlock`

```

function renderParsedBriefBlock(section, fallbackSummary = "") {
  const briefItem = section?.items?.[0] || {};
  const briefText = briefItem.description || fallbackSummary || "";
  if (!briefItem.rich?.blocks?.length && !briefText) return "";
  return `
    <details class="project-brief-default parsed-summary" open>
      <summary>Overview</summary>
      ${renderRichContent(briefItem.rich, briefText)}
    </details>
  `;
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores

them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The function also has to be read through the helpers it calls. It works with `renderRichContent`. Those calls show that `renderParsedBriefBlock` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `renderParsedBriefBlock`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: The local objects and variables inside `renderParsedBriefBlock` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

briefItem: `briefItem` stores a computed value for temporary calculation. It is initialized from `section?.items?.[0] || {}`. Within the function it is accesses properties/methods `description`, `rich`; passed into `renderRichContent`; assigned or updated later in the function.

briefText: `briefText` stores a computed value for temporary calculation. It is initialized from `briefItem.description || fallbackSummary || ""`. Within the function it is passed into `renderRichContent`; assigned or updated later in the function.

function richHasRenderableContent(rich)

Role: To understand `richHasRenderableContent`, start with the problem it owns. `richHasRenderableContent` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function richHasRenderableContent(rich) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `rich` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for richHasRenderableContent

```
function richHasRenderableContent(rich) {  
  return Boolean(rich?.blocks?.some((block) =>  
    block.type === "image" && block.url ||  
    block.type === "formula" && block.formula ||  
    block.type === "code" && block.code ||  
    block.text ||  
    block.title ||  
    block.caption  
  ));  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `richHasRenderableContent` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `richHasRenderableContent`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: Inside `richHasRenderableContent`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function richContainsMedia(rich)

Role: To understand `richContainsMedia`, start with the problem it owns. `richContainsMedia` belongs to rich content rendering. This function turns saved builder content into public HTML: styled text, formulas, code blocks, images, files, links, and safe markup. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function richContainsMedia(rich) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `rich` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for richContainsMedia

```
function richContainsMedia(rich) {  
  return Boolean(rich?.blocks?.some((block) => block.type === "image" && block.url));  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `richContainsMedia` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `richContainsMedia`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: Inside `richContainsMedia`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderProjects()

Role: Read `renderProjects` first as a named idea, not as syntax. `renderProjects` rebuilds the visible project directory from current state. It checks the active category filter and search query, groups visible projects by category, creates the category sections, and updates the summary count. It is called after catalog load, filter changes, and search changes. The function is the public directory refresh mechanism. Without it, filters and searches might update state internally but the page would not visually change.

Function entry point:

```
function renderProjects() { ... }
```

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for renderProjects

```
function renderProjects() {
  const query = normalize(searchInput.value).trim();
  const visible = projects.filter((project) => projectVisible(project, query));

  projectCount.textContent = projects.length;

  if (!visible.length) {
    grid.innerHTML = `
      <div class="empty-state">
        <h3>No projects match that view.</h3>
        <p>Search covers categories, project names, documents, tests, PCB builds, tools, languages, and links.</p>
      </div>
    `;
    queueSearchHighlights();
    return;
  }

  grid.innerHTML = categories
    .map((category) => {
      const visibleProjects = visible.filter((project) => project.category === category.id);
      const shouldShowEmptyCategory = !query && (activeFilter === "all" || activeFilter === category.id);
      return visibleProjects.length || shouldShowEmptyCategory ? categorySection(category, visibleProjects) : "";
    })
    .join("");
  queueSearchHighlights();
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Collaborators: The function also has to be read through the helpers it calls. It works with `normalize`, `projectVisible`, `queueSearchHighlights`, `categorySection`. Those calls show that `renderProjects` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `renderProjects`, builder-created content would not survive cleanly into the website. Formatting, images, formulas, links, code blocks, or sanitized HTML could be lost or rendered unsafely.

Local values: The local objects and variables inside `renderProjects` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

query: `query` stores a computed value for temporary calculation. It is initialized from `normalize(searchInput.value).trim()`. Within the function it is passed into `projectVisible`; assigned or updated later in the function.

visible: `visible` stores a function or callback value for temporary calculation. It is initialized from `projects.filter((project) => projectVisible(project, query))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `length`; assigned or updated later in the function.

visibleProjects: `visibleProjects` stores a function or callback value for portfolio data state. It is initialized from `visible.filter((project) => project.category === category.id)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is returned to the caller; accesses properties/methods `length`; passed into `categorySection`; assigned or updated later in the function.

shouldShowEmptyCategory: `shouldShowEmptyCategory` stores a computed value for portfolio data state. It is initialized from `!query && (activeFilter === "all" || activeFilter === category.id)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

Project windows and navigation

The functions in this group all support project windows and navigation. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function clampSectionDialogPosition(left, top, dialog)

Role: The best entry point for clampSectionDialogPosition is its responsibility in the surrounding workflow. clampSectionDialogPosition belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function clampSectionDialogPosition(left, top, dialog) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. left is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. top is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. dialog is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for clampSectionDialogPosition

```
function clampSectionDialogPosition(left, top, dialog) {  
  const rect = dialog.getBoundingClientRect();  
  const margin = 12;  
  const maxLeft = Math.max(margin, window.innerWidth - rect.width - margin);  
  const maxTop = Math.max(margin, window.innerHeight - rect.height - margin);  
  return {  
    left: Math.min(Math.max(left, margin), maxLeft),  
    top: Math.min(Math.max(top, margin), maxTop)  
  };  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means clampSectionDialogPosition performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without clampSectionDialogPosition, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The easiest way to follow the body of clampSectionDialogPosition is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

rect: rect stores a computed value for temporary calculation. It is initialized from dialog.getBoundingClientRect(). Within the function it is accesses properties/methods height, width; passed into max; assigned or updated later in the function.

margin: margin stores a numeric value for temporary calculation. It is initialized from 12. Within the function it is passed into max, min; assigned or updated later in the function.

maxLeft: maxLeft stores a computed value for temporary calculation. It is initialized from Math.max(margin, window.innerWidth - rect.width - margin). Within the function it is assigned or updated later in the function.

maxTop: maxTop stores a computed value for temporary calculation. It is initialized from Math.max(margin, window.innerHeight - rect.height - margin). Within the function it is assigned or updated later in the function.

function anchorSectionDialog(dialog)

Role: Begin with the role of anchorSectionDialog. anchorSectionDialog belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function anchorSectionDialog(dialog) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. dialog is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for anchorSectionDialog

```
function anchorSectionDialog(dialog) {  
  const rect = dialog.getBoundingClientRect();  
  const position = clampSectionDialogPosition(rect.left, rect.top, dialog);  
  dialog.classList.add("is-draggable-dialog");  
  dialog.style.left = `${position.left}px`;  
  dialog.style.top = `${position.top}px`;  
  dialog.style.right = "auto";  
  dialog.style.bottom = "auto";  
  dialog.style.margin = "0";  
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with clampSectionDialogPosition. Those calls show that anchorSectionDialog is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without anchorSectionDialog, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local state inside anchorSectionDialog explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

rect: rect stores a computed value for temporary calculation. It is initialized from dialog.getBoundingClientRect(). Within the function it is accesses properties/methods left, top; passed into clampSectionDialogPosition; assigned or updated later in the function.

position: position stores a computed value for temporary calculation. It is initialized from clampSectionDialogPosition(rect.left, rect.top, dialog). Within the function it is accesses properties/methods left, top; assigned or updated later in the function.

function ensureSectionResizeHandles(dialog)

Role: Begin with the role of ensureSectionResizeHandles. ensureSectionResizeHandles creates or repairs something only when it is needed. This pattern avoids duplicate DOM nodes, duplicate repositories, duplicate handlers, or missing folders while still letting later code assume the resource exists.

Function entry point:

```
function ensureSectionResizeHandles(dialog) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. dialog is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for ensureSectionResizeHandles

```
function ensureSectionResizeHandles(dialog) {  
  if (dialog.querySelector("[data-section-dialog-resize-handle]")) return;  
  ["n", "e", "s", "w", "ne", "nw", "se", "sw"].forEach((direction) => {  
    const handle = document.createElement("div");  
    handle.className = `dialog-resize-handle resize-${direction}`;  
    handle.dataset.sectionDialogResizeHandle = direction;  
    handle.setAttribute("aria-hidden", "true");  
    dialog.append(handle);  
  });  
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: creates DOM elements instead of relying only on static HTML.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means ensureSectionResizeHandles performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without ensureSectionResizeHandles, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local state inside ensureSectionResizeHandles explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

handle: handle stores a DOM element reference. It is initialized from document.createElement("div"). Within the function it is accesses properties/methods className, dataset, setAttribute; passed into append; assigned or updated later in the function.

function anchorSectionDialogForResize(dialog)

Role: To understand anchorSectionDialogForResize, start with the problem it owns. anchorSectionDialogForResize belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function anchorSectionDialogForResize(dialog) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `anchorSectionDialogForResize`

```
function anchorSectionDialogForResize(dialog) {  
  anchorSectionDialog(dialog);  
  const rect = dialog.getBoundingClientRect();  
  dialog.classList.add("is-resized-dialog");  
  dialog.style.width = `${rect.width}px`;  
  dialog.style.height = `${rect.height}px`;  
  dialog.style.maxHeight = "none";  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

Collaborators: Next, trace the helper calls that surround the function. It works with `anchorSectionDialog`. Those calls show that `anchorSectionDialogForResize` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `anchorSectionDialogForResize`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: After the main flow, the working values inside `anchorSectionDialogForResize` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

rect: `rect` stores a computed value for temporary calculation. It is initialized from `dialog.getBoundingClientRect()`. Within the function it is accesses properties/methods `height`, `width`; assigned or updated later in the function.

function `sectionDialogResizeBounds(state, event)`

Role: Read `sectionDialogResizeBounds` first as a named idea, not as syntax. `sectionDialogResizeBounds` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sectionDialogResizeBounds(state, event) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `state` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `event` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `sectionDialogResizeBounds`

```
function sectionDialogResizeBounds(state, event) {
  const margin = 12;
  const minWidth = Math.min(320, Math.max(180, window.innerWidth - margin * 2));
  const minHeight = Math.min(170, Math.max(120, window.innerHeight - margin * 2));
  const direction = state.direction;
  let { left, top, width, height } = state.rect;
  const dx = event.clientX - state.startX;
  const dy = event.clientY - state.startY;

  if (direction.includes("e")) {
    width = Math.min(Math.max(state.rect.width + dx, minWidth), window.innerWidth - left - margin);
  }
  if (direction.includes("s")) {
    height = Math.min(Math.max(state.rect.height + dy, minHeight), window.innerHeight - top - margin);
  }
  if (direction.includes("w")) {
    const right = state.rect.left + state.rect.width;
    left = Math.min(Math.max(state.rect.left + dx, margin), right - minWidth);
    width = right - left;
  }
  if (direction.includes("n")) {
    const bottom = state.rect.top + state.rect.height;
    top = Math.min(Math.max(state.rect.top + dy, margin), bottom - minHeight);
    height = bottom - top;
  }

  return { height, left, top, width };
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `sectionDialogResizeBounds` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `sectionDialogResizeBounds`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local objects and variables inside `sectionDialogResizeBounds` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

margin: `margin` stores a numeric value for temporary calculation. It is initialized from 12. Within the function it is passed into `min`; assigned or updated later in the function.

minWidth: `minWidth` stores a computed value for temporary calculation. It is initialized from `Math.min(320, Math.max(180, window.innerWidth - margin * 2))`. Within the function it is passed into `min`; assigned or updated later in the function.

minHeight: `minHeight` stores a computed value for temporary calculation. It is initialized from `Math.min(170, Math.max(120, window.innerHeight - margin * 2))`. Within the function it is passed into `min`; assigned or updated later in the function.

direction: `direction` stores a computed value for filesystem location. It is initialized from `state.direction`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods `includes`; assigned or updated later in the function.

destructured { left, top, width, height }: `destructured { left, top, width, height }` stores a computed value for imported or unpacked API handles. It is initialized from `state.rect`. It unpacks API handles from another object, giving the rest of the file short direct names for those APIs.

dx: dx stores a computed value for temporary calculation. It is initialized from event.clientX - state.startX. Within the function it is passed into min; assigned or updated later in the function.

dy: dy stores a computed value for temporary calculation. It is initialized from event.clientY - state.startY. Within the function it is passed into min; assigned or updated later in the function.

right: right stores a computed value for temporary calculation. It is initialized from state.rect.left + state.rect.width. Within the function it is assigned or updated later in the function.

bottom: bottom stores a computed value for temporary calculation. It is initialized from state.rect.top + state.rect.height. Within the function it is assigned or updated later in the function.

function sectionDialogFromDragEvent(event)

Role: To understand sectionDialogFromDragEvent, start with the problem it owns.

sectionDialogFromDragEvent belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sectionDialogFromDragEvent(event) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. event is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for sectionDialogFromDragEvent

```
function sectionDialogFromDragEvent(event) {  
  if (event.target.closest("[data-section-dialog-resize-handle]")) return null;  
  const handle = event.target.closest("[data-section-dialog-drag='true']");  
  if (!handle) return null;  
  if (event.target.closest("button, a, input, textarea, select, label, summary")) return null;  
  const dialog = handle.closest("dialog");  
  return dialog?.open ? dialog : null;  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means sectionDialogFromDragEvent performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without sectionDialogFromDragEvent, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: After the main flow, the working values inside sectionDialogFromDragEvent reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

handle: handle stores a computed value for temporary calculation. It is initialized from event.target.closest("[data-section-dialog-drag='true']"). Within the function it is accesses properties/methods closest; assigned or updated later in the function.

dialog: dialog stores a computed value for DOM reference or UI state. It is initialized from `handle.closest("dialog")`. It anchors UI behavior to a specific browser element or window state. Within the function it is returned to the caller; passed into `closest`; assigned or updated later in the function.

function beginSectionDialogDrag(event)

Role: Begin with the role of `beginSectionDialogDrag`. `beginSectionDialogDrag` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function beginSectionDialogDrag(event) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `event` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for beginSectionDialogDrag

```
function beginSectionDialogDrag(event) {  
  if (event.button !== 0) return;  
  const dialog = sectionDialogFromDragEvent(event);  
  if (!dialog) return;  
  anchorSectionDialog(dialog);  
  const rect = dialog.getBoundingClientRect();  
  activeSectionDialogDrag = {  
    dialog,  
    offsetX: event.clientX - rect.left,  
    offsetY: event.clientY - rect.top,  
    pointerId: event.pointerId,  
    pointerTarget: event.target  
  };  
  event.target.setPointerCapture?.(event.pointerId);  
  dialog.classList.add("is-dragging-dialog");  
  event.preventDefault();  
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `sectionDialogFromDragEvent`, `anchorSectionDialog`. Those calls show that `beginSectionDialogDrag` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `beginSectionDialogDrag`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local state inside `beginSectionDialogDrag` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

dialog: dialog stores a computed value for DOM reference or UI state. It is initialized from `sectionDialogFromDragEvent(event)`. It anchors UI behavior to a specific browser element or window state. Within the function it accesses properties/methods `classList`, `getBoundingClientRect`; passed into `add`, `anchorSectionDialog`; assigned or updated later in the function.

rect: rect stores a computed value for temporary calculation. It is initialized from `dialog.getBoundingClientRect()`. Within the function it is accesses properties/methods `left`, `top`; assigned or updated later in the function.

function moveSectionDialogDrag(event)

Role: Read `moveSectionDialogDrag` first as a named idea, not as syntax. `moveSectionDialogDrag` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function moveSectionDialogDrag(event) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `event` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for moveSectionDialogDrag

```
function moveSectionDialogDrag(event) {  
  if (!activeSectionDialogDrag) return;  
  const next = clampSectionDialogPosition(  
    event.clientX - activeSectionDialogDrag.offsetX,  
    event.clientY - activeSectionDialogDrag.offsetY,  
    activeSectionDialogDrag.dialog  
  );  
  activeSectionDialogDrag.dialog.style.left = `${next.left}px`;  
  activeSectionDialogDrag.dialog.style.top = `${next.top}px`;  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The function also has to be read through the helpers it calls. It works with `clampSectionDialogPosition`. Those calls show that `moveSectionDialogDrag` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `moveSectionDialogDrag`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local objects and variables inside `moveSectionDialogDrag` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

next: next stores a computed value for temporary calculation. It is initialized from `clampSectionDialogPosition()`. Within the function it is accesses properties/methods `left`, `top`; assigned or updated later in the function.

function endSectionDialogDrag()

Role: Read `endSectionDialogDrag` first as a named idea, not as syntax. `endSectionDialogDrag` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function endSectionDialogDrag() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for endSectionDialogDrag

```
function endSectionDialogDrag() {  
  if (!activeSectionDialogDrag) return;  
  activeSectionDialogDrag.pointerTarget?.releasePointerCapture?.(activeSectionDialogDrag.pointerId);  
  activeSectionDialogDrag.dialog.classList.remove("is-dragging-dialog");  
  activeSectionDialogDrag = null;  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means endSectionDialogDrag performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without endSectionDialogDrag, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside endSectionDialogDrag, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function beginSectionDialogResize(event)

Role: Begin with the role of beginSectionDialogResize. beginSectionDialogResize belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function beginSectionDialogResize(event) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. event is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for beginSectionDialogResize

```
function beginSectionDialogResize(event) {  
  if (event.button !== 0) return;  
  const handle = event.target.closest("[data-section-dialog-resize-handle]");  
  const dialog = handle?.closest("dialog");  
  if (!handle || !dialog?.open || dialog.classList.contains("is-minimized-dialog")) return;  
  anchorSectionDialogForResize(dialog);  
  activeSectionDialogResize = {  
    dialog,  
    direction: handle.dataset.sectionDialogResizeHandle,  
    pointerId: event.pointerId,  
    pointerTarget: event.target,  
    rect: dialog.getBoundingClientRect(),  
  };  
}
```

```

    startX: event.clientX,
    startY: event.clientY
  };
  event.target.setPointerCapture?.(event.pointerId);
  dialog.classList.add("is-resizing-dialog");
  event.preventDefault();
}

```

Implementation: The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `anchorSectionDialogForResize`. Those calls show that `beginSectionDialogResize` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `beginSectionDialogResize`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local state inside `beginSectionDialogResize` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

handle: `handle` stores a computed value for temporary calculation. It is initialized from `event.target.closest("[data-section-dialog-resize-handle]")`. Within the function it is accesses properties/methods `dataset`; passed into `closest`; assigned or updated later in the function.

dialog: `dialog` stores a computed value for DOM reference or UI state. It is initialized from `handle?.closest("dialog")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `classList`, `getBoundingClientRect`; passed into `add`, `anchorSectionDialogForResize`, `closest`; assigned or updated later in the function.

function moveSectionDialogResize(event)

Role: Read `moveSectionDialogResize` first as a named idea, not as syntax. `moveSectionDialogResize` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function moveSectionDialogResize(event) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `event` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for moveSectionDialogResize

```

function moveSectionDialogResize(event) {
  if (!activeSectionDialogResize) return;
  const next = sectionDialogResizeBounds(activeSectionDialogResize, event);
  activeSectionDialogResize.dialog.style.left = `${next.left}px`;
  activeSectionDialogResize.dialog.style.top = `${next.top}px`;
  activeSectionDialogResize.dialog.style.width = `${next.width}px`;
  activeSectionDialogResize.dialog.style.height = `${next.height}px`;
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The function also has to be read through the helpers it calls. It works with `sectionDialogResizeBounds`. Those calls show that `moveSectionDialogResize` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `moveSectionDialogResize`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local objects and variables inside `moveSectionDialogResize` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

next: `next` stores a computed value for temporary calculation. It is initialized from `sectionDialogResizeBounds(activeSectionDialogResize, event)`. Within the function it is accesses properties/methods `height`, `left`, `top`, `width`; assigned or updated later in the function.

function endSectionDialogResize()

Role: Read `endSectionDialogResize` first as a named idea, not as syntax. `endSectionDialogResize` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function endSectionDialogResize() { ... }
```

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for endSectionDialogResize

```
function endSectionDialogResize() {  
  if (!activeSectionDialogResize) return;  
  activeSectionDialogResize.pointerTarget?.releasePointerCapture?.(activeSectionDialogResize.pointerId);  
  activeSectionDialogResize.dialog.classList.remove("is-resizing-dialog");  
  activeSectionDialogResize = null;  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `endSectionDialogResize` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `endSectionDialogResize`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `endSectionDialogResize`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function updateSectionDialogMinimize(dialog)

Role: Read updateSectionDialogMinimize first as a named idea, not as syntax. updateSectionDialogMinimize belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function updateSectionDialogMinimize(dialog) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. dialog is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for updateSectionDialogMinimize

```
function updateSectionDialogMinimize(dialog) {  
  const minimized = dialog.classList.contains("is-minimized-dialog");  
  const button = dialog.querySelector(".section-view-minimize");  
  if (!button) return;  
  button.textContent = minimized ? "+" : "-";  
  button.title = minimized ? "Restore window" : "Minimize window";  
  button.setAttribute("aria-label", minimized ? "Restore window" : "Minimize window");  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means updateSectionDialogMinimize performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without updateSectionDialogMinimize, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local objects and variables inside updateSectionDialogMinimize show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

minimized: minimized stores a computed value for temporary calculation. It is initialized from dialog.classList.contains("is-minimized-dialog"). Within the function it is passed into contains, setAttribute; assigned or updated later in the function.

button: button stores a DOM element reference. It is initialized from dialog.querySelector(".section-view-minimize"). It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods setAttribute, textContent, title; assigned or updated later in the function.

function toggleSectionDialogMinimized(dialog)

Role: To understand toggleSectionDialogMinimized, start with the problem it owns. toggleSectionDialogMinimized belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function toggleSectionDialogMinimized(dialog) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `toggleSectionDialogMinimized`

```
function toggleSectionDialogMinimized(dialog) {  
  anchorSectionDialog(dialog);  
  dialog.classList.toggle("is-minimized-dialog");  
  updateSectionDialogMinimize(dialog);  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

Collaborators: Next, trace the helper calls that surround the function. It works with `anchorSectionDialog`, `updateSectionDialogMinimize`. Those calls show that `toggleSectionDialogMinimized` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `toggleSectionDialogMinimized`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `toggleSectionDialogMinimized`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `enableSectionDialogDrag()`

Role: Read `enableSectionDialogDrag` first as a named idea, not as syntax. `enableSectionDialogDrag` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function enableSectionDialogDrag() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `enableSectionDialogDrag`

```
function enableSectionDialogDrag() {  
  if (sectionDialogDragEnabled) return;  
  sectionDialogDragEnabled = true;  
  document.addEventListener("pointerdown", beginSectionDialogResize);  
  document.addEventListener("pointerdown", beginSectionDialogDrag);  
  document.addEventListener("pointermove", moveSectionDialogResize);  
  document.addEventListener("pointermove", moveSectionDialogDrag);  
  document.addEventListener("pointerup", endSectionDialogResize);  
  document.addEventListener("pointerup", endSectionDialogDrag);  
  document.addEventListener("pointercancel", endSectionDialogResize);  
  document.addEventListener("pointercancel", endSectionDialogDrag);  
  window.addEventListener("resize", () => {  
    document.querySelectorAll(".section-view-dialog.is-draggable-dialog[open]").forEach((dialog) => {
```

```

const rect = dialog.getBoundingClientRect();
const next = clampSectionDialogPosition(rect.left, rect.top, dialog);
dialog.style.left = `${next.left}px`;
dialog.style.top = `${next.top}px`;
});
});
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: attaches event handlers that turn user actions into behavior.

Collaborators: The function also has to be read through the helpers it calls. It works with `clampSectionDialogPosition`. Those calls show that `enableSectionDialogDrag` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `enableSectionDialogDrag`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local objects and variables inside `enableSectionDialogDrag` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

rect: `rect` stores a computed value for temporary calculation. It is initialized from `dialog.getBoundingClientRect()`. Within the function it is accesses properties/methods `left`, `top`; passed into `clampSectionDialogPosition`; assigned or updated later in the function.

next: `next` stores a computed value for temporary calculation. It is initialized from `clampSectionDialogPosition(rect.left, rect.top, dialog)`. Within the function it is accesses properties/methods `left`, `top`; assigned or updated later in the function.

function linkifyRichTextNodes(root)

Role: Read `linkifyRichTextNodes` first as a named idea, not as syntax. `linkifyRichTextNodes` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function linkifyRichTextNodes(root) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `root` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for linkifyRichTextNodes

```

function linkifyRichTextNodes(root) {
  const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {
    acceptNode(node) {
      if (!node.textContent || !/(https?:\/\/|www\.|(?:(?:github|linkedin|gitlab|bitbucket|drive|docs)
\.com\/)/i.test(node.textContent)) return NodeFilter.FILTER_REJECT;
      return node.parentElement?.closest("a") ? NodeFilter.FILTER_REJECT : NodeFilter.FILTER_ACCEPT;
    }
  });
}

```

```

const nodes = [];
let node = walker.nextNode();
while (node) {
  nodes.push(node);
  node = walker.nextNode();
}

nodes.forEach((textNode) => {
  const text = textNode.textContent || "";
  const fragment = document.createDocumentFragment();
  let cursor = 0;
  text.replace(/\b(https?:\/\/[^\s<]+|www\.[^\s<]+|(?:(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com\/[^\s<
+\/gi, (match, _url, offset) => {
    if (offset > cursor) fragment.append(document.createTextNode(text.slice(cursor, offset)));
    const trailing = match.match(/[(),.;:!?]+$/)?.[0] || "";
    const clean = trailing ? match.slice(0, -trailing.length) : match;
    const link = document.createElement("a");
    link.href = normalizeLinkTarget(clean, { assumeWeb: true });
    link.target = "_blank";
    link.rel = "noreferrer";
    link.textContent = clean;
    fragment.append(link);
    if (trailing) fragment.append(document.createTextNode(trailing));
    cursor = offset + match.length;
    return match;
  });
  if (cursor < text.length) fragment.append(document.createTextNode(text.slice(cursor)));
  textNode.replaceWith(fragment);
});
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: creates DOM elements instead of relying only on static HTML.

Collaborators: The function also has to be read through the helpers it calls. It works with `normalizeLinkTarget`. Those calls show that `linkifyRichTextNodes` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `linkifyRichTextNodes`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local objects and variables inside `linkifyRichTextNodes` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

walker: `walker` stores a DOM element reference. It is initialized from `document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {`. Within the function it is accesses properties/methods `nextNode`; assigned or updated later in the function.

nodes: `nodes` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods `forEach`, `push`; assigned or updated later in the function.

node: `node` stores a computed value for temporary calculation. It is initialized from `walker.nextNode()`. Within the function it is returned to the caller; accesses properties/methods `parentElement`, `textContent`; passed into `createTreeWalker`, `push`, `test`; assigned or updated later in the function.

text: `text` stores a computed value for temporary calculation. It is initialized from `textNode.textContent || ""`. Within the function it is accesses properties/methods `length`, `replace`, `slice`; passed into `append`; assigned or updated later in the function.

fragment: fragment stores a DOM element reference. It is initialized from `document.createDocumentFragment()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `append`; passed into `replaceWith`; assigned or updated later in the function.

cursor: cursor stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is passed into `append`; assigned or updated later in the function.

trailing: trailing stores a computed value for temporary calculation. It is initialized from `match.match(/[,],... Within the function it is accesses properties/methods length; passed into append, slice; assigned or updated later in the function.`

clean: clean stores a computed value for temporary calculation. It is initialized from `trailing ? match.slice(0, -trailing.length) : match`. Within the function it is passed into `normalizeLinkTarget`; assigned or updated later in the function.

link: link stores a DOM element reference. It is initialized from `document.createElement("a")`. Within the function it is accesses properties/methods `href`, `rel`, `target`, `textContent`; passed into `append`; assigned or updated later in the function.

function `siteSectionHasContent(section)`

Role: To understand `siteSectionHasContent`, start with the problem it owns. `siteSectionHasContent` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function siteSectionHasContent(section) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `siteSectionHasContent`

```
function siteSectionHasContent(section) {  
  return Boolean(  
    section?.description ||  
    section?.richDescription?.blocks?.length ||  
    section?.backgroundImage ||  
    (section?.links || []).some((link) => link.label || link.url) ||  
    (section?.assets || []).some((asset) => asset.title || asset.url) ||  
    (section?.subsections || []).some((item) => item.title || item.description ||  
    item.richDescription?.blocks?.length || (item.links || []).length)  
  );  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `siteSectionHasContent` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `siteSectionHasContent`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `siteSectionHasContent`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `siteSectionRenderable(section)`

Role: Read `siteSectionRenderable` first as a named idea, not as syntax. `siteSectionRenderable` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function siteSectionRenderable(section) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `siteSectionRenderable`

```
function siteSectionRenderable(section) {  
  return section?.visible !== false && siteSectionHasContent(section);  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. It works with `siteSectionHasContent`. Those calls show that `siteSectionRenderable` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `siteSectionRenderable`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `siteSectionRenderable`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `renderSiteSections()`

Role: To understand `renderSiteSections`, start with the problem it owns. `renderSiteSections` turns saved data for `SiteSections` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Function entry point:

```
function renderSiteSections() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `renderSiteSections`

```
function renderSiteSections() {
```

```

const mount = document.querySelector("#dynamic-sections");
if (!mount) return;
const visibleSections = (siteSections || []).filter(siteSectionRenderable);
mount.innerHTML = visibleSections.map((section) => {
  const style = section.backgroundImage ? ` style="--section-bg: url('${escapeHtml(section.backgroundImage)}')` :
  "";
  const links = [...(section.links || []), ...(section.assets || [])];
  const subsections = (section.subsections || []).filter((item) => item.title || item.description ||
  item.richDescription?.blocks?.length || (item.links || []).length);
  return `
    <section class="section dynamic-section" id="${escapeHtml(section.id || "")}"${style}>
      <div class="dynamic-section-surface">
        <div class="section-heading">
          <div>
            <h2>${escapeHtml(section.title || "Untitled section")}</h2>
          </div>
        </div>
        ${section.description || section.richDescription?.blocks?.length
        ? `<div class="dynamic-section-copy">${renderRichContent(section.richDescription, section.description ||
        "")}</div>`
        : ""}
        ${subsections.length ? `
          <div class="dynamic-section-grid">
            ${subsections.map((item) => `
              <article class="dynamic-section-card">
                <h3>${escapeHtml(item.title || "Untitled")}</h3>
                ${item.description || item.richDescription?.blocks?.length
                ? renderRichContent(item.richDescription, item.description || "")
                : ""}
                ${renderDynamicLinks(item.links || [])}
              </article>
            `).join("")}
          </div>
          : ""
        `}
        ${renderDynamicLinks(links)}
      </div>
    </section>
  `;
}).join("");

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Collaborators: Next, trace the helper calls that surround the function. It works with `escapeHtml`, `renderRichContent`, `renderDynamicLinks`. Those calls show that `renderSiteSections` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `renderSiteSections`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: After the main flow, the working values inside `renderSiteSections` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

mount: `mount` stores a DOM element reference. It is initialized from `document.querySelector("#dynamic-sections")`. Within the function it is accesses properties/methods `innerHTML`; assigned or updated later in the function.

visibleSections: `visibleSections` stores a function or callback value for portfolio data state. It is initialized from `(siteSections || []).filter(siteSectionRenderable)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; assigned or updated later in the function.

style: style stores a computed value for appearance configuration. It is initialized from `section.backgroundImage ? ` style="--section-bg: url('${escapeHtml(section.backgroundImage)}')'` : ""`. Within the function it is assigned or updated later in the function.

links: links stores a list for temporary calculation. It is initialized from `[...(section.links || []), ...(section.assets || [])]`. Within the function it is passed into `renderDynamicLinks`; assigned or updated later in the function.

subsections: subsections stores a function or callback value for portfolio data state. It is initialized from `(section.subsections || []).filter((item) => item.title || item.description || item.richDescription?.blocks?.length || (item.links || []).length)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; assigned or updated later in the function.

function customSectionBlocks(project)

Role: Read `customSectionBlocks` first as a named idea, not as syntax. `customSectionBlocks` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function customSectionBlocks(project) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for customSectionBlocks

```
function customSectionBlocks(project) {  
  return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-wide", `  
    ${section.description ? `<p class="evidence-empty">${section.description}</p>` : ""}  
    ${evidenceList(section.items || [], (item) => `  
      <li>  
        ${item.url ? resourceLink(item, item.title) : `<strong>${item.title}</strong>`}  
        <span>${item.type || "Section item"} &middot; ${item.status || "tracked"}</span>  
        ${item.description ? `<p>${item.description}</p>` : ""}  
      </li>  
    `), "No content has been added yet.")}`  
    `)).join("");  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. It works with `detailBlock`, `evidenceList`, `resourceLink`. Those calls show that `customSectionBlocks` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `customSectionBlocks`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `customSectionBlocks`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderCollectedDownloadSections(project)

Role: The best entry point for renderCollectedDownloadSections is its responsibility in the surrounding workflow. renderCollectedDownloadSections turns saved data for CollectedDownloadSections into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Function entry point:

```
function renderCollectedDownloadSections(project) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for renderCollectedDownloadSections

```
function renderCollectedDownloadSections(project) {  
  const blocks = [];  
  
  const addBlock = (title, source, fallbackType = "File") => {  
    const downloads = uniqueDownloads(collectDownloadsFromValue(source, fallbackType, []));  
    if (downloads.length) blocks.push(renderDownloadBlock(title, downloads));  
  };  
  
  addBlock("Overview downloads", [  
    { summaryRich: project.summaryRich },  
    project.design?.brief?.files,  
    project.design?.brief?.summaryRich  
  ], "Overview file");  
  
  addBlock("Design downloads", [  
    project.design?.documentation?.files,  
    project.design?.documentation?.references,  
    project.design?.documentation?.mathAnalysis  
  ], "Design file");  
  
  addBlock("Simulation downloads", [  
    project.design?.simulation?.files,  
    project.design?.simulation?.results  
  ], "Simulation file");  
  
  addBlock("Project documents", project.documents, "Document");  
  addBlock("Test artifacts", project.tests, "Test artifact");  
  addBlock("PCB artifacts", project.pcb, "PCB artifact");  
  addBlock("Images and media", project.media, "Image");  
  
  (project.sections || []).forEach((section) => {  
    addBlock(`${displayTitle(section.title, "Custom section")} downloads`, section.items || section.files || [],  
    section.title || "Section file");  
  });  
  
  (project.portfolioView?.sections || []).  
    .filter((section) => section.id !== "brief")  
    .forEach((section) => {  
      addBlock(`${displayTitle(section.title, "Section")} downloads`, section.items || [], section.title ||  
      "Section file");  
    });  
  
  return blocks.join("");  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `uniqueDownloads`, `collectDownloadsFromValue`, `renderDownloadBlock`, `displayTitle`. Those calls show that `renderCollectedDownloadSections` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `renderCollectedDownloadSections`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The easiest way to follow the body of `renderCollectedDownloadSections` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

blocks: `blocks` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

addBlock: `addBlock` stores a function or callback value for temporary calculation. It is initialized from `(title, source, fallbackType = "File") => {}`. Within the function it is assigned or updated later in the function.

downloads: `downloads` stores a computed value for temporary calculation. It is initialized from `uniqueDownloads(collectDownloadsFromValue(source, fallbackType, []))`. Within the function it is accesses properties/methods `length`; passed into `addBlock`, `push`; assigned or updated later in the function.

function pathToString(path = [])

Role: The best entry point for `pathToString` is its responsibility in the surrounding workflow. `pathToString` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function pathToString(path = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `path` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for pathToString

```
function pathToString(path = []) {  
  return path.join(".");  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: uses Node path utilities so Windows path separators and relative paths are handled consistently.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `pathToString` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `pathToString`, nested project navigation would break down. A visitor might click a

section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `pathToString`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `pathFromString(value = "")`

Role: Begin with the role of `pathFromString`. `pathFromString` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function pathFromString(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `pathFromString`

```
function pathFromString(value = "") {  
  if (!value) return [];  
  return value.split(".").map((item) => Number(item)).filter((item) => Number.isInteger(item));  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `pathFromString` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `pathFromString`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `pathFromString`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `sectionRouteState(projectId, sectionIndex, resourcePath = "")`

Role: To understand `sectionRouteState`, start with the problem it owns. `sectionRouteState` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sectionRouteState(projectId, sectionIndex, resourcePath = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `resourcePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does

not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `sectionRouteState`

```
function sectionRouteState(projectId, sectionIndex, resourcePath = "") {  
  return {  
    appWindow: sectionRouteKey,  
    projectId: String(projectId || ""),  
    resourcePath: String(resourcePath || ""),  
    sectionIndex: String(sectionIndex ?? "")  
  };  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `sectionRouteState` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `sectionRouteState`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `sectionRouteState`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `canUseSectionHistory()`

Role: Read `canUseSectionHistory` first as a named idea, not as syntax. `canUseSectionHistory` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

Function entry point:

```
function canUseSectionHistory() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `canUseSectionHistory`

```
function canUseSectionHistory() {  
  return Boolean(window.history && typeof window.history.pushState === "function" && typeof  
    window.history.replaceState === "function");  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser route/history. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: updates browser history so Back, Forward, and refresh can follow the same window state.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `canUseSectionHistory` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `canUseSectionHistory`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `canUseSectionHistory`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function sectionBaseUrl()

Role: The best entry point for `sectionBaseUrl` is its responsibility in the surrounding workflow. `sectionBaseUrl` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sectionBaseUrl() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for sectionBaseUrl

```
function sectionBaseUrl() {  
  const url = new URL(window.location.href);  
  Object.values(sectionRouteParams).forEach((param) => url.searchParams.delete(param));  
  return url;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `sectionBaseUrl` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `sectionBaseUrl`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The easiest way to follow the body of `sectionBaseUrl` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

url: `url` stores a constructed object for network or routing value. It is initialized from `new URL(window.location.href)`. Within the function it is returned to the caller; accesses properties/methods `searchParams`; assigned or updated later in the function.

function sectionRouteUrl(projectId, sectionIndex, resourcePath = "")

Role: The best entry point for `sectionRouteUrl` is its responsibility in the surrounding workflow. `sectionRouteUrl` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sectionRouteUrl(projectId, sectionIndex, resourcePath = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `resourcePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `sectionRouteUrl`

```
function sectionRouteUrl(projectId, sectionIndex, resourcePath = "") {
  const url = sectionBaseUrl();
  url.searchParams.set(sectionRouteParams.project, String(projectId || ""));
  url.searchParams.set(sectionRouteParams.section, String(sectionIndex ?? ""));
  if (resourcePath) url.searchParams.set(sectionRouteParams.path, String(resourcePath));
  else url.searchParams.delete(sectionRouteParams.path);
  url.hash = "projects";
  return url;
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `sectionBaseUrl`. Those calls show that `sectionRouteUrl` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `sectionRouteUrl`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The easiest way to follow the body of `sectionRouteUrl` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

url: url stores a computed value for network or routing value. It is initialized from `sectionBaseUrl()`. Within the function it is returned to the caller; accesses properties/methods `hash`, `searchParams`; assigned or updated later in the function.

function `sectionRouteFromState(state)`

Role: To understand `sectionRouteFromState`, start with the problem it owns. `sectionRouteFromState` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sectionRouteFromState(state) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `state` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string

data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `sectionRouteFromState`

```
function sectionRouteFromState(state) {  
  if (!state || state.appWindow !== sectionRouteKey) return null;  
  if (!state.projectId || state.sectionIndex === "") return null;  
  return {  
    projectId: String(state.projectId),  
    resourcePath: String(state.resourcePath || ""),  
    sectionIndex: String(state.sectionIndex)  
  };  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `sectionRouteFromState` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `sectionRouteFromState`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `sectionRouteFromState`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function sectionRouteFromLocation()`

Role: To understand `sectionRouteFromLocation`, start with the problem it owns. `sectionRouteFromLocation` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sectionRouteFromLocation() { ... }
```

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `sectionRouteFromLocation`

```
function sectionRouteFromLocation() {  
  const params = new URLSearchParams(window.location.search);  
  const projectId = params.get(sectionRouteParams.project);  
  const sectionIndex = params.get(sectionRouteParams.section);  
  if (!projectId || sectionIndex === null) return null;  
  return {  
    projectId,  
    resourcePath: params.get(sectionRouteParams.path) || "",  
    sectionIndex  
  };  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser route/history. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects. A close reading of the body shows these implementation details: reads or writes URL query parameters so browser history can represent the current view.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `sectionRouteFromLocation` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `sectionRouteFromLocation`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: After the main flow, the working values inside `sectionRouteFromLocation` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

params: `params` stores a constructed object for temporary calculation. It is initialized from `new URLSearchParams(window.location.search)`. Within the function it is accessed properties/methods get; assigned or updated later in the function.

projectId: `projectId` stores a computed value for portfolio data state. It is initialized from `params.get(sectionRouteParams.project)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

sectionIndex: `sectionIndex` stores a computed value for portfolio data state. It is initialized from `params.get(sectionRouteParams.section)`. Within the function it is assigned or updated later in the function.

function sameSectionRoute(left, right)

Role: Read `sameSectionRoute` first as a named idea, not as syntax. `sameSectionRoute` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sameSectionRoute(left, right) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `left` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `right` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for sameSectionRoute

```
function sameSectionRoute(left, right) {  
  return Boolean(left && right &&  
    left.projectId === right.projectId &&  
    String(left.sectionIndex) === String(right.sectionIndex) &&  
    String(left.resourcePath || "") === String(right.resourcePath || ""));  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `sameSectionRoute` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `sameSectionRoute`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `sameSectionRoute`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function syncSectionRouteToHistory(projectId, sectionIndex, resourcePath = "", mode = "push")

Role: Read `syncSectionRouteToHistory` first as a named idea, not as syntax. `syncSectionRouteToHistory` reconciles two states that can drift apart. In this project, sync functions connect local drafts, route state, Git branches, publish mirrors, or frontend state to the current truth.

Function entry point:

```
function syncSectionRouteToHistory(projectId, sectionIndex, resourcePath = "", mode = "push")
{ ... }
```

Inputs: The function-specific inputs are worth reading before the body. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `resourcePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `mode` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for syncSectionRouteToHistory

```
function syncSectionRouteToHistory(projectId, sectionIndex, resourcePath = "", mode = "push") {
  if (isApplyingSectionRoute || mode === "none" || !canUseSectionHistory()) return;
  const route = sectionRouteState(projectId, sectionIndex, resourcePath);
  const url = sectionRouteUrl(projectId, sectionIndex, resourcePath);
  const currentRoute = sectionRouteFromState(history.state) || sectionRouteFromLocation();
  const method = mode === "replace" || sameSectionRoute(currentRoute, route) ? "replaceState" : "pushState";
  try {
    history[method](route, "", url);
  } catch {
    // Preview iframes and restricted browser contexts may reject history writes.
  }
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser route/history. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The function also has to be read through the helpers it calls. It works with `canUseSectionHistory`, `sectionRouteState`, `sectionRouteUrl`, `sectionRouteFromState`, `sectionRouteFromLocation`, `sameSectionRoute`. Those calls show that `syncSectionRouteToHistory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `syncSectionRouteToHistory`, nested project navigation would break down. A visitor might

click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local objects and variables inside `syncSectionRouteToHistory` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

route: `route` stores a resolved filesystem or route value. It is initialized from `sectionRouteState(projectId, sectionIndex, resourcePath)`. Within the function it is passed into `sameSectionRoute`; assigned or updated later in the function.

url: `url` stores a resolved filesystem or route value. It is initialized from `sectionRouteUrl(projectId, sectionIndex, resourcePath)`. Within the function it is assigned or updated later in the function.

currentRoute: `currentRoute` stores a computed value for temporary calculation. It is initialized from `sectionRouteFromState(history.state) || sectionRouteFromLocation()`. Within the function it is passed into `sameSectionRoute`; assigned or updated later in the function.

method: `method` stores a computed value for temporary calculation. It is initialized from `mode === "replace" || sameSectionRoute(currentRoute, route) ? "replaceState" : "pushState"`. Within the function it is assigned or updated later in the function.

function clearSectionRouteInHistory()

Role: Begin with the role of `clearSectionRouteInHistory`. `clearSectionRouteInHistory` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function clearSectionRouteInHistory() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for clearSectionRouteInHistory

```
function clearSectionRouteInHistory() {  
  if (isApplyingSectionRoute || !canUseSectionHistory()) return;  
  try {  
    history.replaceState({ appWindow: "portfolio-base" }, "", sectionBaseUrl());  
  } catch {  
    // The visible dialog is already closed; URL cleanup is best effort.  
  }  
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it updates browser route/history. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: updates browser history so Back, Forward, and refresh can follow the same window state.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `canUseSectionHistory`, `sectionBaseUrl`. Those calls show that `clearSectionRouteInHistory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `clearSectionRouteInHistory`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `clearSectionRouteInHistory`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function initializeSectionHistoryState()

Role: Read `initializeSectionHistoryState` first as a named idea, not as syntax. `initializeSectionHistoryState` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function initializeSectionHistoryState() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for initializeSectionHistoryState

```
function initializeSectionHistoryState() {  
  if (!canUseSectionHistory()) return;  
  const route = sectionRouteFromLocation() || sectionRouteFromState(history.state);  
  try {  
    if (route) history.replaceState(sectionRouteState(route.projectId, route.sectionIndex, route.resourcePath), "",  
sectionRouteUrl(route.projectId, route.sectionIndex, route.resourcePath));  
    else if (!history.state?.appWindow) history.replaceState({ appWindow: "portfolio-base" }, "",  
window.location.href);  
  } catch {  
    // Browser history state is progressive enhancement for the portfolio windows.  
  }  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser route/history. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: updates browser history so Back, Forward, and refresh can follow the same window state.

Collaborators: The function also has to be read through the helpers it calls. It works with `canUseSectionHistory`, `sectionRouteFromLocation`, `sectionRouteFromState`, `sectionRouteState`, `sectionRouteUrl`. Those calls show that `initializeSectionHistoryState` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `initializeSectionHistoryState`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local objects and variables inside `initializeSectionHistoryState` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

route: `route` stores a computed value for temporary calculation. It is initialized from `sectionRouteFromLocation() || sectionRouteFromState(history.state)`. Within the function it is accesses properties/methods `projectId`, `resourcePath`, `sectionIndex`; passed into `replaceState`, `sectionRouteUrl`; assigned or updated later in the function.

function nodeAtPath(section, path = [])

Role: Begin with the role of `nodeAtPath`. `nodeAtPath` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function nodeAtPath(section, path = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `path` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `nodeAtPath`

```
function nodeAtPath(section, path = []) {  
  let node = section;  
  let children = section?.items || [];  
  for (const index of path) {  
    node = children[index];  
    if (!node) return null;  
    children = node.children || [];  
  }  
  return node;  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `nodeAtPath` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `nodeAtPath`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local state inside `nodeAtPath` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

node: `node` stores a computed value for temporary calculation. It is initialized from `section`. Within the function it is returned to the caller; accesses properties/methods `children`; assigned or updated later in the function.

children: `children` stores a computed value for temporary calculation. It is initialized from `section?.items || []`. Within the function it is assigned or updated later in the function.

index: `index` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function.

function `nodeChildren(node)`

Role: The best entry point for `nodeChildren` is its responsibility in the surrounding workflow. `nodeChildren` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function nodeChildren(node) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `nodeChildren`

```
function nodeChildren(node) {  
  return node?.items || node?.children || [];  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `nodeChildren` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `nodeChildren`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `nodeChildren`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function nodeHasRenderableContent(node)`

Role: To understand `nodeHasRenderableContent`, start with the problem it owns. `nodeHasRenderableContent` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function nodeHasRenderableContent(node) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `nodeHasRenderableContent`

```
function nodeHasRenderableContent(node) {  
  if (!node) return false;  
  if (node.kind === "summary") return Boolean(node.description || richHasRenderableContent(node.rich));  
  const children = nodeChildren(node);  
  if (node.kind === "subsection") {  
    return Boolean(node.description || node.url || richHasRenderableContent(node.rich) ||  
      children.some(nodeHasRenderableContent));  
  }  
  if (["tool", "language"].includes(node.kind)) {  
    return Boolean(node.title || node.description || node.url || richHasRenderableContent(node.rich) ||  
      children.some(nodeHasRenderableContent));  
  }  
  return Boolean(node.description || node.url || richHasRenderableContent(node.rich) ||  
    children.some(nodeHasRenderableContent));  
}
```

```
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `richHasRenderableContent`, `nodeChildren`. Those calls show that `nodeHasRenderableContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `nodeHasRenderableContent`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: After the main flow, the working values inside `nodeHasRenderableContent` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

children: `children` stores a computed value for temporary calculation. It is initialized from `nodeChildren(node)`. Within the function it is accesses properties/methods some; assigned or updated later in the function.

function `sectionHasRenderableContent(section)`

Role: Begin with the role of `sectionHasRenderableContent`. `sectionHasRenderableContent` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sectionHasRenderableContent(section) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `sectionHasRenderableContent`

```
function sectionHasRenderableContent(section) {  
  return Boolean(section?.description || richHasRenderableContent(section?.rich) || (section?.items ||  
  []).some(nodeHasRenderableContent));  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `richHasRenderableContent`. Those calls show that `sectionHasRenderableContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `sectionHasRenderableContent`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside sectionHasRenderableContent, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function responsiveNodeHasMedia(node)

Role: Begin with the role of responsiveNodeHasMedia. responsiveNodeHasMedia belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function responsiveNodeHasMedia(node) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. node is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for responsiveNodeHasMedia

```
function responsiveNodeHasMedia(node) {  
  if (!node) return false;  
  if (node.kind === "image" || richContainsMedia(node.rich)) return true;  
  const url = String(node.url || "");  
  if (/\. (apng|avif|gif|jpe?g|png|svg|webp)(\?|#|$)/i.test(url)) return true;  
  return responsiveChildren(node).some(responsiveNodeHasMedia);  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with richContainsMedia, responsiveChildren. Those calls show that responsiveNodeHasMedia is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without responsiveNodeHasMedia, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local state inside responsiveNodeHasMedia explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

url: url stores a computed value for network or routing value. It is initialized from String(node.url || ""). Within the function it is passed into String, test; assigned or updated later in the function.

function responsiveNodeCount(node)

Role: To understand responsiveNodeCount, start with the problem it owns. responsiveNodeCount belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function responsiveNodeCount(node) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `responsiveNodeCount`

```
function responsiveNodeCount(node) {  
  if (!node || !nodeHasRenderableContent(node)) return 0;  
  return 1 + responsiveChildren(node).reduce((count, child) => count + responsiveNodeCount(child), 0);  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `nodeHasRenderableContent`, `responsiveChildren`. Those calls show that `responsiveNodeCount` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `responsiveNodeCount`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `responsiveNodeCount`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function responsiveNodeDepth(node)`

Role: To understand `responsiveNodeDepth`, start with the problem it owns. `responsiveNodeDepth` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function responsiveNodeDepth(node) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `responsiveNodeDepth`

```
function responsiveNodeDepth(node) {  
  if (!node || !nodeHasRenderableContent(node)) return 0;  
  const children = responsiveChildren(node).filter(nodeHasRenderableContent);  
  if (!children.length) return 1;  
  return 1 + Math.max(...children.map(responsiveNodeDepth));  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `nodeHasRenderableContent`, `responsiveChildren`. Those calls show that `responsiveNodeDepth` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `responsiveNodeDepth`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: After the main flow, the working values inside `responsiveNodeDepth` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

children: `children` stores a computed value for temporary calculation. It is initialized from `responsiveChildren(node).filter(nodeHasRenderableContent)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; passed into `max`; assigned or updated later in the function.

function nodeSummary(title, rich, text, emptyMessage = "No summary has been added yet.")

Role: Begin with the role of `nodeSummary`. `nodeSummary` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function nodeSummary(title, rich, text, emptyMessage = "No summary has been added yet.") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `title` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `rich` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `emptyMessage` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for nodeSummary

```
function nodeSummary(title, rich, text, emptyMessage = "No summary has been added yet.") {  
  if (!rich?.blocks?.length && !text) return "";  
  return `  
    <details class="parsed-summary" open>  
      <summary>${escapeHtml(title)}</summary>  
      ${renderRichContent(rich, text || "")}  
    </details>  
  `;  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `escapeHtml`, `renderRichContent`. Those calls show that `nodeSummary` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without nodeSummary, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside nodeSummary, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function nodeIsOverview(item = {})

Role: To understand nodeIsOverview, start with the problem it owns. nodeIsOverview belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function nodeIsOverview(item = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. item is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for nodeIsOverview

```
function nodeIsOverview(item = {}) {  
  const title = normalize(displayTitle(item.title || item.label || ""));  
  return item.kind === "summary" || title === "overview" || title.endsWith(" overview");  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with normalize, displayTitle. Those calls show that nodeIsOverview is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without nodeIsOverview, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: After the main flow, the working values inside nodeIsOverview reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

title: title stores a computed value for temporary calculation. It is initialized from normalize(displayTitle(item.title || item.label || "")). Within the function it is accesses properties/methods endsWith; passed into normalize; assigned or updated later in the function.

function nodeOverviewDetails(node, children = [])

Role: The best entry point for nodeOverviewDetails is its responsibility in the surrounding workflow. nodeOverviewDetails belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function nodeOverviewDetails(node, children = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. `children` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `nodeOverviewDetails`

```
function nodeOverviewDetails(node, children = []) {  
  const overviewItems = children.filter(nodeIsOverview);  
  const blocks = [];  
  if (node?.rich?.blocks?.length || node?.description) {  
    blocks.push(renderRichContent(node.rich, node.description || ""));  
  }  
  overviewItems.forEach((item) => {  
    if (item.rich?.blocks?.length || item.description) blocks.push(renderRichContent(item.rich, item.description || ""));  
  });  
  const itemFiles = renderInlineSectionFiles(nodeChildren(item));  
  if (itemFiles) blocks.push(itemFiles);  
  if (!blocks.length) return "";  
  return `  
    <details class="parsed-summary section-overview-default" open>  
      <summary>Overview</summary>  
      ${blocks.join("")}  
    </details>  
  `;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `renderRichContent`, `renderInlineSectionFiles`, `nodeChildren`. Those calls show that `nodeOverviewDetails` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `nodeOverviewDetails`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The easiest way to follow the body of `nodeOverviewDetails` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

overviewItems: `overviewItems` stores a computed value for temporary calculation. It is initialized from `children.filter(nodeIsOverview)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

blocks: `blocks` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `length`, `push`; assigned or updated later in the function.

itemFiles: `itemFiles` stores a computed value for temporary calculation. It is initialized from `renderInlineSectionFiles(nodeChildren(item))`. Within the function it is passed into `push`; assigned or updated later in the function.

function parsedNodeCard(node, projectId, sectionIndex, path)

Role: The best entry point for parsedNodeCard is its responsibility in the surrounding workflow.

parsedNodeCard interprets dNodeCard text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Function entry point:

```
function parsedNodeCard(node, projectId, sectionIndex, path) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. node is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. projectId identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. sectionIndex is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. path points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for parsedNodeCard

```
function parsedNodeCard(node, projectId, sectionIndex, path) {  
  const title = displayTitle(node.title);  
  return `  
    <button  
      class="section-open-card subsection-open-card"  
      type="button"  
      data-section-project="${projectId}"  
      data-section-index="${sectionIndex}"  
      data-resource-path="${pathToString(path)}"  
    >  
      <span>${escapeHtml(title)}</span>  
    </button>  
  `;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with displayTitle, pathToString, escapeHtml. Those calls show that parsedNodeCard is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without parsedNodeCard, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The easiest way to follow the body of parsedNodeCard is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

title: title stores a computed value for temporary calculation. It is initialized from displayTitle(node.title). Within the function it is passed into displayTitle, escapeHtml; assigned or updated later in the function.

function parsedNodeContent(node, projectId, sectionIndex, path = [])

Role: Begin with the role of parsedNodeContent. parsedNodeContent interprets dNodeContent text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Function entry point:

```
function parsedNodeContent(node, projectId, sectionIndex, path = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. node is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. projectId identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. sectionIndex is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. path points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for parsedNodeContent

```
function parsedNodeContent(node, projectId, sectionIndex, path = []) {  
  const isContainer = !node.kind || node.kind === "subsection";  
  if (!isContainer && node.url) return renderInlineSectionFiles([node]);  
  
  const children = nodeChildren(node);  
  const contentChildren = children.filter((child) => !nodeIsOverview(child));  
  
  return `  
  <article class="parsed-window-panel section-directory">  
    ${nodeOverviewDetails(node, children)}  
    ${parsedChildCards(children, projectId, sectionIndex, path)}  
    ${renderInlineSectionFiles(contentChildren)}  
  </article>  
  `;  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with renderInlineSectionFiles, nodeChildren, nodeIsOverview, nodeOverviewDetails, parsedChildCards. Those calls show that parsedNodeContent is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without parsedNodeContent, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local state inside parsedNodeContent explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

isContainer: isContainer stores a computed value for temporary calculation. It is initialized from !node.kind || node.kind === "subsection". Within the function it is assigned or updated later in the function.

children: children stores a computed value for temporary calculation. It is initialized from `nodeChildren(node)`. Within the function it is iterated or transformed as a collection; accesses properties/methods filter; passed into `nodeOverviewDetails`, `parsedChildCards`; assigned or updated later in the function.

contentChildren: `contentChildren` stores a function or callback value for temporary calculation. It is initialized from `children.filter((child) => !nodeOverview(child))`. Within the function it is passed into `renderInlineSectionFiles`; assigned or updated later in the function.

function `parsedSectionContent(section, projectId, sectionIndex, path = [])`

Role: The best entry point for `parsedSectionContent` is its responsibility in the surrounding workflow. `parsedSectionContent` interprets `dSectionContent` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Function entry point:

```
function parsedSectionContent(section, projectId, sectionIndex, path = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `path` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `parsedSectionContent`

```
function parsedSectionContent(section, projectId, sectionIndex, path = []) {  
  const node = path.length ? nodeAtPath(section, path) : section;  
  if (!node) return `

This section could not be found.

`;  
  return parsedNodeContent(node, projectId, sectionIndex, path);  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `nodeAtPath`, `parsedNodeContent`. Those calls show that `parsedSectionContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `parsedSectionContent`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The easiest way to follow the body of `parsedSectionContent` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

node: node stores a resolved filesystem or route value. It is initialized from `path.length ? nodeAtPath(section, path) : section`. Within the function it is passed into `parsedNodeContent`; assigned or updated later in the function.

function parsedSection(section, index, project)

Role: Begin with the role of `parsedSection`. `parsedSection` interprets `dSection` text and returns structured meaning. Parsing functions are needed because the rest of the app should not repeatedly scan raw strings when it can work with a stable object, array, or normalized value.

Function entry point:

```
function parsedSection(section, index, project) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `section` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `index` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `parsedSection`

```
function parsedSection(section, index, project) {  
  const visibleItems = (section.items || []).filter(nodeHasRenderableContent);  
  const hasImages = visibleItems.some((item) => item.kind === "image");  
  
  return `  
    <button  
      class="section-open-card evidence-block ${hasImages ? "evidence-wide" : ""}"  
      type="button"  
      data-section-project="${project.id}"  
      data-section-index="${index}"  
    >  
      <span>${escapeHtml(displayTitle(section.title, "Section"))}</span>  
    </button>  
  `;  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `escapeHtml`, `displayTitle`. Those calls show that `parsedSection` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `parsedSection`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local state inside `parsedSection` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

visibleItems: `visibleItems` stores a function or callback value for temporary calculation. It is initialized from `(section.items || []).filter(nodeHasRenderableContent)`. Within the function it is accesses properties/methods some; assigned or updated later in the function.

hasImages: hasImages stores a function or callback value for temporary calculation. It is initialized from `visibleItems.some((item) => item.kind === "image")`. Within the function it is assigned or updated later in the function.

function categorySection(category, visibleProjects)

Role: The best entry point for categorySection is its responsibility in the surrounding workflow. categorySection handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Function entry point:

```
function categorySection(category, visibleProjects) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. category is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. visibleProjects identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for categorySection

```
function categorySection(category, visibleProjects) {
  return `
    <section class="category-section ${visibleProjects.length ? "" : "empty-category-section"}" aria-labelledby="${
      {escapeHtml(category.id)}-title">
      <div class="category-heading">
        <div>
          <h3 id="${{escapeHtml(category.id)}-title">${escapeHtml(category.label)}</h3>
        </div>
        ${visibleProjects.length ? `<span>${visibleProjects.length} project${visibleProjects.length === 1 ? "" :
"s"}</span>` : ""}
      </div>
      ${visibleProjects.length && category.description ? `<p class="category-description">${
        {escapeHtml(category.description)}</p>` : ""}
      <div class="category-projects">
        ${visibleProjects.map(projectCard).join("")}
      </div>
    </section>
  `;
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with escapeHtml. Those calls show that categorySection is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without categorySection, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside categorySection, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function ensureSectionDialog()

Role: The best entry point for ensureSectionDialog is its responsibility in the surrounding workflow. ensureSectionDialog creates and wires the reusable full-window dialog used for project sections. It builds the top controls, back/forward behavior, close behavior, content container, and event delegation for child cards. It creates the window once and then later calls reuse it. The function exists so every project section window behaves consistently. If each click created its own markup by hand, close buttons, back buttons, keyboard behavior, and nested navigation would drift apart.

Function entry point:

```
function ensureSectionDialog() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for ensureSectionDialog

```
function ensureSectionDialog() {
  let dialog = document.querySelector("#section-view-dialog");
  if (dialog) return dialog;

  dialog = document.createElement("dialog");
  dialog.id = "section-view-dialog";
  dialog.className = "section-view-dialog";
  dialog.innerHTML = `
    <div class="section-view-shell">
      <div class="section-view-heading">
        <div class="section-view-titlebar">
          <button class="section-view-back" type="button" title="Previous view" aria-label="Previous view"
hidden>&larr;</button>
          <button class="section-view-forward" type="button" title="Next view" aria-label="Next view"
hidden>&rarr;</button>
          <h2 id="section-view-title">Section</h2>
        </div>
        <div class="section-view-actions">
          <button class="section-view-close" type="button" aria-label="Close section">&times;</button>
        </div>
      </div>
      <div class="section-view-content" id="section-view-content"></div>
    </div>
  `;
  document.body.append(dialog);
  dialog.querySelector(".section-view-close").addEventListener("click", () => closeSectionDialog(dialog));
  dialog.addEventListener("cancel", () => clearSectionRouteInHistory());
  dialog.addEventListener("close", () => {
    document.body.classList.remove("full-window-open");
    dialog.classList.remove("is-minimized-dialog");
    updateSectionDialogMinimize(dialog);
    const returnTarget = document.querySelector(
      `[data-section-project="${dialog.dataset.projectId}"][data-section-index="${dialog.dataset.sectionIndex}"]`
    );
    returnTarget?.focus({ preventScroll: true });
  });
  dialog.querySelector(".section-view-back").addEventListener("click", () => {
    const path = pathFromString(dialog.dataset.resourcePath || "");
    if (!path.length) {
      closeSectionDialog(dialog);
      return;
    }
    const stack = sectionForwardStack(dialog);
    stack.push(pathToString(path));
    dialog.dataset.forwardStack = JSON.stringify(stack);
    path.pop();
    openParsedSection(dialog.dataset.projectId, dialog.dataset.sectionIndex, pathToString(path), { historyMode:
"replace", preserveForward: true });
  });
  dialog.querySelector(".section-view-forward").addEventListener("click", () => {
    const stack = sectionForwardStack(dialog);
    const nextPath = stack.pop();
    if (!nextPath) return;
    dialog.dataset.forwardStack = JSON.stringify(stack);
  });
}
```

```

    openParsedSection(dialog.dataset.projectId, dialog.dataset.sectionIndex, nextPath, { historyMode: "replace",
preserveForward: true });
  });
  dialog.querySelector("#section-view-content").addEventListener("click", (event) => {
    const nodeButton = event.target.closest("[data-resource-path]");
    if (!nodeButton) return;
    openParsedSection(nodeButton.dataset.sectionProject, nodeButton.dataset.sectionIndex,
nodeButton.dataset.resourcePath);
  });
  return dialog;
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; creates DOM elements instead of relying only on static HTML; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter; toggles CSS classes to express UI state; attaches event handlers that turn user actions into behavior.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `closeSectionDialog`, `clearSectionRouteInHistory`, `updateSectionDialogMinimize`, `pathFromString`, `sectionForwardStack`, `pathToString`, `openParsedSection`. Those calls show that `ensureSectionDialog` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `ensureSectionDialog`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The easiest way to follow the body of `ensureSectionDialog` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

dialog: `dialog` stores a DOM element reference. It is initialized from `document.querySelector("#section-view-dialog")`. It anchors UI behavior to a specific browser element or window state. Within the function it is returned to the caller; accesses properties/methods `addEventListener`, `classList`, `className`, `dataset`, `id`, `innerHTML`, `querySelector`; passed into `append`, `closeSectionDialog`, `createElement`, `openParsedSection`, `pathFromString`, `querySelector`, `remove`; assigned or updated later in the function.

returnTarget: `returnTarget` stores a DOM element reference. It is initialized from `document.querySelector()`. Within the function it is assigned or updated later in the function.

path: `path` stores a resolved filesystem or route value. It is initialized from `pathFromString(dialog.dataset.resourcePath || "")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accessed properties/methods `length`, `pop`; passed into `closest`, `openParsedSection`, `push`; assigned or updated later in the function.

stack: `stack` stores a computed value for runtime state or cache. It is initialized from `sectionForwardStack(dialog)`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `pop`, `push`; passed into `stringify`; assigned or updated later in the function.

stack: `stack` stores a computed value for runtime state or cache. It is initialized from `sectionForwardStack(dialog)`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `pop`, `push`; passed into `stringify`; assigned or updated later in the function.

nextPath: `nextPath` stores a computed value for filesystem location. It is initialized from `stack.pop()`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `openParsedSection`; assigned or updated later in the function.

nodeButton: nodeButton stores a computed value for DOM reference or UI state. It is initialized from event.target.closest("[data-resource-path]"). It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods dataset; passed into openParsedSection; assigned or updated later in the function.

function sectionForwardStack(dialog)

Role: Begin with the role of sectionForwardStack. sectionForwardStack belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function sectionForwardStack(dialog) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. dialog is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for sectionForwardStack

```
function sectionForwardStack(dialog) {  
  try {  
    return JSON.parse(dialog.dataset.forwardStack || "[]");  
  } catch {  
    return [];  
  }  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates. A close reading of the body shows these implementation details: parses JSON rather than treating incoming text as already trusted data.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means sectionForwardStack performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without sectionForwardStack, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside sectionForwardStack, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function updateSectionNavigation(dialog, path)

Role: Begin with the role of updateSectionNavigation. updateSectionNavigation belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function updateSectionNavigation(dialog, path) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. `path` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `updateSectionNavigation`

```
function updateSectionNavigation(dialog, path) {  
  const back = dialog.querySelector(".section-view-back");  
  back.hidden = false;  
  back.title = path.length ? "Previous view" : "Back to projects";  
  back.setAttribute("aria-label", path.length ? "Previous view" : "Back to projects");  
  dialog.querySelector(".section-view-forward").hidden = !sectionForwardStack(dialog).length;  
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `sectionForwardStack`. Those calls show that `updateSectionNavigation` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `updateSectionNavigation`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local state inside `updateSectionNavigation` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

back: `back` stores a DOM element reference. It is initialized from `dialog.querySelector(".section-view-back")`. Within the function it is accesses properties/methods `hidden`, `setAttribute`, `title`; passed into `querySelector`; assigned or updated later in the function.

function `closeSectionDialog(dialog)`

Role: The best entry point for `closeSectionDialog` is its responsibility in the surrounding workflow. `closeSectionDialog` reverses an open window or route state. It does more than hide markup: it must also clean navigation state, restore the previous view when appropriate, and avoid trapping the visitor inside a dialog.

Function entry point:

```
function closeSectionDialog(dialog) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `closeSectionDialog`

```
function closeSectionDialog(dialog) {  
  dialog.close();  
  clearSectionRouteInHistory();  
}
```

```
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `clearSectionRouteInHistory`. Those calls show that `closeSectionDialog` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `closeSectionDialog`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: Inside `closeSectionDialog`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function closeOrStepBackSectionDialog(dialog)

Role: To understand `closeOrStepBackSectionDialog`, start with the problem it owns. `closeOrStepBackSectionDialog` reverses an open window or route state. It does more than hide markup: it must also clean navigation state, restore the previous view when appropriate, and avoid trapping the visitor inside a dialog.

Function entry point:

```
function closeOrStepBackSectionDialog(dialog) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `dialog` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for closeOrStepBackSectionDialog

```
function closeOrStepBackSectionDialog(dialog) {  
  const path = pathFromString(dialog.dataset.resourcePath || "");  
  if (path.length) {  
    const stack = sectionForwardStack(dialog);  
    stack.push(pathToString(path));  
    dialog.dataset.forwardStack = JSON.stringify(stack);  
    path.pop();  
    openParsedSection(dialog.dataset.projectId, dialog.dataset.sectionIndex, pathToString(path), { historyMode:  
      "replace", preserveForward: true });  
    return;  
  }  
  closeSectionDialog(dialog);  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: serializes structured data before sending or saving it.

Collaborators: Next, trace the helper calls that surround the function. It works with `pathFromString`, `sectionForwardStack`, `pathToString`, `openParsedSection`, `closeSectionDialog`. Those calls show that `closeOrStepBackSectionDialog` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `closeOrStepBackSectionDialog`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: After the main flow, the working values inside `closeOrStepBackSectionDialog` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

path: `path` stores a resolved filesystem or route value. It is initialized from `pathFromString(dialog.dataset.resourcePath || "")`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it accesses properties/methods `length`, `pop`; passed into `openParsedSection`, `push`; assigned or updated later in the function.

stack: `stack` stores a computed value for runtime state or cache. It is initialized from `sectionForwardStack(dialog)`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `push`; passed into `stringify`; assigned or updated later in the function.

function `openParsedSection(projectId, sectionIndex, resourcePath = "", options = {})`

Role: Read `openParsedSection` first as a named idea, not as syntax. `openParsedSection` opens a project section or nested subsection as a full-window public view. It finds the project, finds the section, walks the requested subsection path, applies the selected appearance template, renders the overview and child sections, updates navigation buttons, and syncs browser history. This function is the heart of the website's project-window behavior. Without it, clicking Design, Simulation, Documentation, or any user-created section would either scroll the page awkwardly or render content in the wrong place.

Function entry point:

```
function openParsedSection(projectId, sectionIndex, resourcePath = "", options = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `projectId` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `sectionIndex` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `resourcePath` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `openParsedSection`

```
function openParsedSection(projectId, sectionIndex, resourcePath = "", options = {}) {  
  const project = projects.find((item) => item.id === projectId);  
  const sections = (project?.portfolioView?.sections || []).filter((section) => section.id !== "brief" &&  
    sectionHasRenderableContent(section));  
  const section = sections[Number(sectionIndex)];  
  if (!section) return false;  
  const path = pathFromString(resourcePath);  
  const node = path.length ? nodeAtPath(section, path) : section;  
  if (!node) return false;  
  if (path.length ? !nodeHasRenderableContent(node) : !sectionHasRenderableContent(section)) return false;  
  
  const dialog = ensureSectionDialog();
```

```

const category = categories.find((item) => item.id === project.category) || {};
applyProjectTemplateToElement(dialog, project, category.accent || "#117c7a");
dialog.dataset.projectId = projectId;
dialog.dataset.sectionIndex = String(sectionIndex);
dialog.dataset.resourcePath = pathToString(path);
if (!options.preserveForward) dialog.dataset.forwardStack = "[]";
syncSectionRouteToHistory(projectId, sectionIndex, pathToString(path), options.historyMode || "push");
dialog.querySelector("#section-view-title").textContent = displayTitle(node.title || section.title, "Section");
updateSectionNavigation(dialog, path);
dialog.querySelector("#section-view-content").innerHTML = parsedSectionContent(section, projectId,
Number(sectionIndex), path);
document.body.classList.add("full-window-open");
if (!dialog.open) dialog.showModal();
dialog.scrollTop = 0;
queueSearchHighlights();
return true;
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter; toggles CSS classes to express UI state.

Collaborators: The function also has to be read through the helpers it calls. It works with `sectionHasRenderableContent`, `pathFromString`, `nodeAtPath`, `nodeHasRenderableContent`, `ensureSectionDialog`, `applyProjectTemplateToElement`, `pathToString`, `syncSectionRouteToHistory`. Those calls show that `openParsedSection` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `openParsedSection`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local objects and variables inside `openParsedSection` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

project: project stores a function or callback value for portfolio data state. It is initialized from `projects.find((item) => item.id === projectId)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `category`; passed into `applyProjectTemplateToElement`; assigned or updated later in the function.

sections: sections stores a function or callback value for portfolio data state. It is initialized from `(project?.portfolioView?.sections || []).filter((section) => section.id !== "brief" && sectionHasRenderableContent(section))`. Within the function it is assigned or updated later in the function.

section: section stores a computed value for portfolio data state. It is initialized from `sections[Number(sectionIndex)]`. Within the function it is accesses properties/methods `id`, `title`; passed into `displayTitle`, `filter`, `nodeAtPath`, `parsedSectionContent`, `querySelector`, `sectionHasRenderableContent`; assigned or updated later in the function.

path: path stores a resolved filesystem or route value. It is initialized from `pathFromString(resourcePath)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is accesses properties/methods `length`; passed into `nodeAtPath`, `pathToString`, `syncSectionRouteToHistory`, `updateSectionNavigation`; assigned or updated later in the function.

node: node stores a resolved filesystem or route value. It is initialized from `path.length ? nodeAtPath(section, path) : section`. Within the function it is accesses properties/methods `title`; passed into `displayTitle`; assigned or updated later in the function.

dialog: dialog stores a computed value for DOM reference or UI state. It is initialized from `ensureSectionDialog()`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `dataset`, `open`, `querySelector`, `scrollTop`, `showModal`; passed into `applyProjectTemplateToElement`, `updateSectionNavigation`; assigned or updated later in the function.

category: category stores a function or callback value for portfolio data state. It is initialized from `categories.find((item) => item.id === project.category) || {}`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `accent`; passed into `applyProjectTemplateToElement`; assigned or updated later in the function.

function closeSectionDialogForRoute()

Role: To understand `closeSectionDialogForRoute`, start with the problem it owns.

`closeSectionDialogForRoute` reverses an open window or route state. It does more than hide markup: it must also clean navigation state, restore the previous view when appropriate, and avoid trapping the visitor inside a dialog.

Function entry point:

```
function closeSectionDialogForRoute() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for closeSectionDialogForRoute

```
function closeSectionDialogForRoute() {  
  const dialog = document.querySelector("#section-view-dialog");  
  if (dialog?.open) dialog.close();  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `closeSectionDialogForRoute` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `closeSectionDialogForRoute`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: After the main flow, the working values inside `closeSectionDialogForRoute` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

dialog: dialog stores a DOM element reference. It is initialized from `document.querySelector("#section-view-dialog")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `close`; passed into `querySelector`; assigned or updated later in the function.

function applySectionRoute(route)

Role: Read `applySectionRoute` first as a named idea, not as syntax. `applySectionRoute` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function applySectionRoute(route) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `route` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `applySectionRoute`

```
function applySectionRoute(route) {
  isApplyingSectionRoute = true;
  try {
    if (!route) {
      closeSectionDialogForRoute();
      return;
    }

    const opened = openParsedSection(route.projectId, route.sectionIndex, route.resourcePath, {
      historyMode: "none",
      preserveForward: true
    });
    if (!opened) closeSectionDialogForRoute();
  } finally {
    isApplyingSectionRoute = false;
  }
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The function also has to be read through the helpers it calls. It works with `closeSectionDialogForRoute`, `openParsedSection`. Those calls show that `applySectionRoute` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `applySectionRoute`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The local objects and variables inside `applySectionRoute` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

opened: `opened` stores a resolved filesystem or route value. It is initialized from `openParsedSection(route.projectId, route.sectionIndex, route.resourcePath, {`. Within the function it is assigned or updated later in the function.

`function restoreSectionRouteAfterCatalogLoad()`

Role: To understand `restoreSectionRouteAfterCatalogLoad`, start with the problem it owns. `restoreSectionRouteAfterCatalogLoad` belongs to project windows and navigation. This function supports full-window project sections, nested subsections, browser history, back/forward behavior, or content tree traversal. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function restoreSectionRouteAfterCatalogLoad() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string

data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `restoreSectionRouteAfterCatalogLoad`

```
function restoreSectionRouteAfterCatalogLoad() {
  initializeSectionHistoryState();
  applySectionRoute(sectionRouteFromLocation() || sectionRouteFromState(history.state));
}

window.addEventListener("popstate", (event) => {
  applySectionRoute(sectionRouteFromState(event.state) || sectionRouteFromLocation());
});

ensureSearchPanel();
searchInput.addEventListener("input", handleSearchInput);
searchInput.addEventListener("focus", showSearchPanelIfNeeded);
searchInput.addEventListener("keydown", (event) => {
  if (event.key === "Enter" && currentSearchResults.length) {
    event.preventDefault();
    goToSearchResult(currentSearchResults[0]);
  }
  if (event.key === "Escape") {
    clearSearchHighlights();
    if (searchPanel) searchPanel.hidden = true;
  }
});

document.addEventListener("click", (event) => {
  if (!searchPanel || searchPanel.hidden) return;
  if (event.target.closest(".search-engine")) return;
  searchPanel.hidden = true;
});

document.addEventListener("click", (event) => {
  const topLink = event.target.closest('a[href="#top"]');
  if (!topLink) return;
  event.preventDefault();
  closeSectionDialogForRoute();
  window.scrollTo({ top: 0, behavior: "smooth" });
  try {
    history.replaceState({ appWindow: "portfolio-base" }, "", "#top");
  } catch {
    window.location.hash = "top";
  }
});

aiAssistantForm?.addEventListener("submit", (event) => {
  event.preventDefault();
  const question = aiAssistantInput?.value || "";
  if (aiAssistantInput) aiAssistantInput.value = "";
  answerAssistantQuestion(question);
});

aiClearChatButton?.addEventListener("click", clearAssistantChat);
window.addEventListener("resize", updateAssistantPanelGrowth);

aiAssistantLog?.addEventListener("click", (event) => {
  const button = event.target.closest("[data-ai-source-id]");
  if (!button) return;
  const source = assistantSourceMap.get(button.dataset.aiSourceId);
  if (source) goToSearchResult(source);
});

grid.addEventListener("click", (event) => {
  const sectionButton = event.target.closest("[data-section-project]");
  if (!sectionButton) return;
  openParsedSection(sectionButton.dataset.sectionProject, sectionButton.dataset.sectionIndex);
})
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser route/history. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: attaches event handlers that turn

user actions into behavior; updates browser history so Back, Forward, and refresh can follow the same window state.

Collaborators: Next, trace the helper calls that surround the function. It works with `initializeSectionHistoryState`, `applySectionRoute`, `sectionRouteFromLocation`, `sectionRouteFromState`, `ensureSearchPanel`, `goToSearchResult`, `clearSearchHighlights`, `closeSectionDialogForRoute`. Those calls show that `restoreSectionRouteAfterCatalogLoad` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `restoreSectionRouteAfterCatalogLoad`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: After the main flow, the working values inside `restoreSectionRouteAfterCatalogLoad` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

topLink: `topLink` stores a computed value for temporary calculation. It is initialized from `event.target.closest('a[href="#top"]')`. Within the function it is assigned or updated later in the function.

question: `question` stores a computed value for temporary calculation. It is initialized from `aiAssistantInput?.value || ""`. Within the function it is passed into `answerAssistantQuestion`; assigned or updated later in the function.

button: `button` stores a computed value for DOM reference or UI state. It is initialized from `event.target.closest("[data-ai-source-id]")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `dataset`; passed into `get`; assigned or updated later in the function.

source: `source` stores a computed value for lookup collection. It is initialized from `assistantSourceMap.get(button.dataset.aiSourceId)`. Within the function it is passed into `closest`, `goToSearchResult`; assigned or updated later in the function.

sectionButton: `sectionButton` stores a computed value for portfolio data state. It is initialized from `event.target.closest("[data-section-project]")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `dataset`; passed into `openParsedSection`; assigned or updated later in the function.

function renderInlineSectionFiles(items = [])

Role: The best entry point for `renderInlineSectionFiles` is its responsibility in the surrounding workflow. `renderInlineSectionFiles` turns saved data for `InlineSectionFiles` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Function entry point:

```
function renderInlineSectionFiles(items = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `items` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for renderInlineSectionFiles

```
function renderInlineSectionFiles(items = []) {
  const downloads = uniqueDownloads(items.map((item) => normalizeDownloadAsset(item, "File")).filter(Boolean));
  if (!downloads.length) return "";

  return `
<div class="section-file-list">
  ${downloads.map((item) => `
    <a
      class="section-file-link"
      href="${escapeHtml(normalizeLinkTarget(item.url, { assumeWeb: isWebsiteLinkItem(item, item.url) })))}"
      ${linkAttributes(item.url, item)}
      ${downloadAttribute(item.url, item)}
    >
      ${escapeHtml(item.title || fileNameFromUrl(item.url))}
    </a>
  `).join("")}
</div>
`;
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `uniqueDownloads`, `normalizeDownloadAsset`, `escapeHtml`, `normalizeLinkTarget`, `isWebsiteLinkItem`, `linkAttributes`, `downloadAttribute`, `fileNameFromUrl`. Those calls show that `renderInlineSectionFiles` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `renderInlineSectionFiles`, nested project navigation would break down. A visitor might click a section and see nothing, lose Back/Forward behavior, remain trapped in a dialog, or see content rendered in the wrong hierarchy.

Local values: The easiest way to follow the body of `renderInlineSectionFiles` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

downloads: `downloads` stores a function or callback value for lookup collection. It is initialized from `uniqueDownloads(items.map((item) => normalizeDownloadAsset(item, "File")).filter(Boolean))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; assigned or updated later in the function.

Resources and links

The functions in this group all support resources and links. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function normalizeLinkTarget(target = "", options = {})

Role: Read `normalizeLinkTarget` first as a named idea, not as syntax. `normalizeLinkTarget` standardizes `LinkTarget` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeLinkTarget(target = "", options = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `target` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can

make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `normalizeLinkTarget`

```
function normalizeLinkTarget(target = "", options = {}) {  
  const clean = String(target || "").trim();  
  if (!clean) return "";  
  if (/^\//.test(clean)) return `https:${clean}`;  
  if (/^www\./i.test(clean)) return `https://${clean}`;  
  if (options.assumeWeb && looksLikeBareWebAddress(clean)) return `https://${clean}`;  
  return clean;  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The function also has to be read through the helpers it calls. It works with `looksLikeBareWebAddress`. Those calls show that `normalizeLinkTarget` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `normalizeLinkTarget`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside `normalizeLinkTarget` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(target || "").trim()`. Within the function it is returned to the caller; assigned or updated later in the function.

`function linkAttributes(url, item = {})`

Role: To understand `linkAttributes`, start with the problem it owns. `linkAttributes` belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function linkAttributes(url, item = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `linkAttributes`

```
function linkAttributes(url, item = {}) {
```

```

const target = normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(item, url) });
return /^https?:\/\/\./i.test(target) ? ' target="_blank" rel="noreferrer"' : '';
}

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalizeLinkTarget`, `isWebsiteLinkItem`. Those calls show that `linkAttributes` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `linkAttributes`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: After the main flow, the working values inside `linkAttributes` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

target: `target` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(item, url) })`. Within the function it is passed into `test`; assigned or updated later in the function.

function isLocalDownloadTarget(target = "")

Role: Read `isLocalDownloadTarget` first as a named idea, not as syntax. `isLocalDownloadTarget` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

Function entry point:

```
function isLocalDownloadTarget(target = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `target` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for isLocalDownloadTarget

```

function isLocalDownloadTarget(target = "") {
  const value = normalizeLinkTarget(target, { assumeWeb: true });
  return Boolean(value) && !/^(https?:)?\/\./i.test(value) && !/^(mailto:|tel:|#)/i.test(value);
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. It works with `normalizeLinkTarget`. Those calls show that `isLocalDownloadTarget` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `isLocalDownloadTarget`, the specific rule it owns would disappear from the named workflow.

Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside `isLocalDownloadTarget` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

value: value stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(target, { assumeWeb: true })`. Within the function it is passed into `Boolean`, `test`; assigned or updated later in the function.

function downloadAttribute(target = "", item = {})

Role: Begin with the role of `downloadAttribute`. `downloadAttribute` belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function downloadAttribute(target = "", item = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `target` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for downloadAttribute

```
function downloadAttribute(target = "", item = {}) {  
  const value = normalizeLinkTarget(target, { assumeWeb: isWebsiteLinkItem(item, target) });  
  return isLocalDownloadTarget(value) ? "download" : "";  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalizeLinkTarget`, `isWebsiteLinkItem`, `isLocalDownloadTarget`. Those calls show that `downloadAttribute` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `downloadAttribute`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside `downloadAttribute` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

value: value stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(target, { assumeWeb: isWebsiteLinkItem(item, target) })`. Within the function it is passed into `isLocalDownloadTarget`; assigned or updated later in the function.

function fileNameFromUrl(url = "")

Role: Begin with the role of fileNameFromUrl. fileNameFromUrl belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function fileNameFromUrl(url = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. url points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for fileNameFromUrl

```
function fileNameFromUrl(url = "") {  
  const clean = String(url || "").split(/[?#]/)[0].split("/").pop() || "Download file";  
  try {  
    return decodeURIComponent(clean);  
  } catch {  
    return clean;  
  }  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means fileNameFromUrl performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without fileNameFromUrl, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside fileNameFromUrl explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from String(url || "").split(/[?#]/)[0].split("/").pop() || "Download file". Within the function it is returned to the caller; passed into decodeURIComponent; assigned or updated later in the function.

function resourceLink(item = {}, label = item.label || item.title || item.name || fileNameFromUrl(item.url || item.artifact))

Role: Read resourceLink first as a named idea, not as syntax. resourceLink belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function resourceLink(item = {}, label = item.label || item.title || item.name ||  
fileNameFromUrl(item.url || item.artifact)) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. item is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more

predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. label is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for resourceLink

```
function resourceLink(item = {}, label = item.label || item.title || item.name || fileNameFromUrl(item.url ||
item.artifact)) {
  const rawTarget = item.url || item.artifact || item.href || item.file || item.path || item.src || "";
  const target = normalizeLinkTarget(rawTarget, { assumeWeb: isWebsiteLinkItem(item, rawTarget) });

  if (!target || item.status === "planned") {
    return `<span class="resource-link muted">${escapeHtml(label || "Planned file")}</span>`;
  }

  return `
<a
  class="resource-link"
  href="${escapeHtml(target)}"
  ${linkAttributes(target, item)}
  ${downloadAttribute(target, item)}
>
  ${escapeHtml(label || fileNameFromUrl(target))}
</a>
`;
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The function also has to be read through the helpers it calls. It works with fileNameFromUrl, normalizeLinkTarget, isWebsiteLinkItem, escapeHtml, linkAttributes, downloadAttribute. Those calls show that resourceLink is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without resourceLink, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside resourceLink show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

rawTarget: rawTarget stores a computed value for temporary calculation. It is initialized from item.url || item.artifact || item.href || item.file || item.path || item.src || "". Within the function it is passed into normalizeLinkTarget; assigned or updated later in the function.

target: target stores a computed value for temporary calculation. It is initialized from normalizeLinkTarget(rawTarget, { assumeWeb: isWebsiteLinkItem(item, rawTarget) }). Within the function it is passed into downloadAttribute, escapeHtml, linkAttributes; assigned or updated later in the function.

function itemUrl(item = {})

Role: Read itemUrl first as a named idea, not as syntax. itemUrl belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function itemUrl(item = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `itemUrl`

```
function itemUrl(item = {}) {  
  return item.url || item.artifact || item.href || item.file || item.path || item.src || "";  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `itemUrl` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `itemUrl`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `itemUrl`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function normalizeDownloadAsset(item = {}, fallbackType = "File")`

Role: To understand `normalizeDownloadAsset`, start with the problem it owns. `normalizeDownloadAsset` standardizes `DownloadAsset` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeDownloadAsset(item = {}, fallbackType = "File") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `item` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `fallbackType` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `normalizeDownloadAsset`

```
function normalizeDownloadAsset(item = {}, fallbackType = "File") {  
  const url = itemUrl(item);
```

```

    if (!url || item.status === "planned") return null;

    return {
      title: itemLabel(item),
      url,
      status: item.status || "uploaded",
      type: item.type || item.kind || item.meta || fallbackType,
      description: item.description || item.caption || item.summary || ""
    };
  }
}

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Next, trace the helper calls that surround the function. It works with `itemUrl`, `itemLabel`. Those calls show that `normalizeDownloadAsset` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `normalizeDownloadAsset`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: After the main flow, the working values inside `normalizeDownloadAsset` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

url: url stores a computed value for network or routing value. It is initialized from `itemUrl(item)`. Within the function it is assigned or updated later in the function.

function collectDownloadsFromValue(value, fallbackType = "File", output = [])

Role: Read `collectDownloadsFromValue` first as a named idea, not as syntax. `collectDownloadsFromValue` gathers scattered nested data into a shape that another function can loop over. This is important because portfolio data is hierarchical while rendering, search, and AI context often need a flat list.

Function entry point:

```
function collectDownloadsFromValue(value, fallbackType = "File", output = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `fallbackType` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `output` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for collectDownloadsFromValue

```

function collectDownloadsFromValue(value, fallbackType = "File", output = []) {
  if (!value) return output;

  if (Array.isArray(value)) {
    value.forEach((item) => collectDownloadsFromValue(item, fallbackType, output));
    return output;
  }
}

```

```

if (typeof value !== "object") return output;

const asset = normalizeDownloadAsset(value, fallbackType);
if (asset) output.push(asset);

collectRichDownloads(value.rich, fallbackType, output);
collectRichDownloads(value.summaryRich, fallbackType, output);

[
  "files",
  "assets",
  "items",
  "children",
  "links",
  "results",
  "references",
  "mathAnalysis"
].forEach((key) => collectDownloadsFromValue(value[key], fallbackType, output));

return output;
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. It works with `normalizeDownloadAsset`, `collectRichDownloads`. Those calls show that `collectDownloadsFromValue` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `collectDownloadsFromValue`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside `collectDownloadsFromValue` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

asset: `asset` stores a computed value for lookup collection. It is initialized from `normalizeDownloadAsset(value, fallbackType)`. Within the function it is passed into `push`; assigned or updated later in the function.

function uniqueDownloads(items = [])

Role: Read `uniqueDownloads` first as a named idea, not as syntax. `uniqueDownloads` belongs to resources and links. This function decides whether content is a website link, local download, uploaded file, or resource card. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function uniqueDownloads(items = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `items` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for uniqueDownloads

```
function uniqueDownloads(items = []) {
  const seen = new Set();

  return items.filter((item) => {
    const key = `${item.url}:${item.title}`;
    if (!item.url || seen.has(key)) return false;
    seen.add(key);
    return true;
  });
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means uniqueDownloads performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without uniqueDownloads, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside uniqueDownloads show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

seen: seen stores a constructed object for lookup collection. It is initialized from new Set(). Within the function it is mutated as a collection or DOM container; accesses properties/methods add, has; assigned or updated later in the function.

key: key stores a string value for temporary calculation. It is initialized from `\${item.url}:\${item.title}`. Within the function it is passed into add; assigned or updated later in the function.

Appearance and responsive presentation

The functions in this group all support appearance and responsive presentation. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function cleanFontFamily(value = "")

Role: Begin with the role of cleanFontFamily. cleanFontFamily belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function cleanFontFamily(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for cleanFontFamily

```
function cleanFontFamily(value = "") {
  return String(value)
    .split(",")
```

```

    .map((item) => item.replace(/^[^w\s-]/g, "").trim())
    .filter(Boolean)
    .slice(0, 3)
    .join(", ");
}

```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means cleanFontFamily performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without cleanFontFamily, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside cleanFontFamily, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function normalizeFontPx(value)

Role: Begin with the role of normalizeFontPx. normalizeFontPx standardizes FontPx data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeFontPx(value) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for normalizeFontPx

```

function normalizeFontPx(value) {
  const size = Number(value);
  return Number.isFinite(size) && size >= 8 && size <= 24 ? String(size) : "";
}

```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means normalizeFontPx performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without normalizeFontPx, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside normalizeFontPx explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

size: size stores a computed value for temporary calculation. It is initialized from Number(value). Within the function it is passed into String, isFinite; assigned or updated later in the function.

function normalizeTextColor(value = "")

Role: To understand normalizeTextColor, start with the problem it owns. normalizeTextColor standardizes TextColor data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeTextColor(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for normalizeTextColor

```
function normalizeTextColor(value = "") {  
  const color = String(value || "").trim();  
  return /^#([0-9a-f]{3}|[0-9a-f]{6})$/i.test(color) ||  
    /^rgba?\(/i.test(color) ||  
    /^hsla?\(/i.test(color)  
    ? color  
    : "";  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means normalizeTextColor performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without normalizeTextColor, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: After the main flow, the working values inside normalizeTextColor reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

color: color stores a computed value for appearance configuration. It is initialized from String(value || "").trim(). Within the function it is passed into test; assigned or updated later in the function.

function rgbColorToHex(value = "")

Role: Read rgbColorToHex first as a named idea, not as syntax. rgbColorToHex belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function rgbColorToHex(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `rgbColorToHex`

```
function rgbColorToHex(value = "") {  
  const color = String(value || "").trim();  
  if (/^#[0-9a-f]{6}$/i.test(color)) return color.toLowerCase();  
  const shortHex = color.match(/^#([0-9a-f])([0-9a-f])([0-9a-f])$/i);  
  if (shortHex) return `#${shortHex[1]}${shortHex[1]}${shortHex[2]}${shortHex[2]}${shortHex[3]}${shortHex[3]}`  
    .toLowerCase();  
  const rgb = color.match(/^rgba?\s*((\d+),\s*(\d+),\s*(\d+))/i);  
  if (!rgb) return color;  
  return `#${[rgb[1], rgb[2], rgb[3]].map((part) => Math.max(0, Math.min(255, Number(part))).toString(16).padStart(2, "0")).join("")}`;  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `rgbColorToHex` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `rgbColorToHex`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside `rgbColorToHex` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

color: `color` stores a computed value for appearance configuration. It is initialized from `String(value || "").trim()`. Within the function it is returned to the caller; accesses properties/methods `match`, `toLowerCase`; assigned or updated later in the function.

shortHex: `shortHex` stores a computed value for temporary calculation. It is initialized from `color.match(/^#([0-9a-f])([0-9a-f])([0-9a-f])$/i)`. Within the function it is assigned or updated later in the function.

rgb: `rgb` stores a computed value for temporary calculation. It is initialized from `color.match(/^rgba?\s*((\d+),\s*(\d+),\s*(\d+))/i)`. Within the function it is assigned or updated later in the function.

`function normalizePlainFieldStyle(style = {})`

Role: Read `normalizePlainFieldStyle` first as a named idea, not as syntax. `normalizePlainFieldStyle` standardizes `PlainFieldStyle` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizePlainFieldStyle(style = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `style` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more

predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `normalizePlainFieldStyle`

```
function normalizePlainFieldStyle(style = {}) {
  const normalized = {};
  const fontFamily = cleanFontFamily(style.fontFamily || "");
  const fontPx = normalizeFontPx(style.fontPx || style.fontSize || "");
  const color = normalizeTextColor(style.color || "");
  if (fontFamily) normalized.fontFamily = fontFamily;
  if (fontPx) normalized.fontPx = fontPx;
  if (color) normalized.color = rgbColorToHex(color);
  if (style.bold === true || style.bold === "true") normalized.bold = true;
  if (style.italic === true || style.italic === "true") normalized.italic = true;
  if (style.underline === true || style.underline === "true") normalized.underline = true;
  return normalized;
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. It works with `cleanFontFamily`, `normalizeFontPx`, `normalizeTextColor`, `rgbColorToHex`. Those calls show that `normalizePlainFieldStyle` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `normalizePlainFieldStyle`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside `normalizePlainFieldStyle` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

normalized: `normalized` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; accesses properties/methods `bold`, `color`, `fontFamily`, `fontPx`, `italic`, `underline`; assigned or updated later in the function.

fontFamily: `fontFamily` stores a computed value for temporary calculation. It is initialized from `cleanFontFamily(style.fontFamily || "")`. Within the function it is passed into `cleanFontFamily`; assigned or updated later in the function.

fontPx: `fontPx` stores a computed value for temporary calculation. It is initialized from `normalizeFontPx(style.fontPx || style.fontSize || "")`. Within the function it is passed into `normalizeFontPx`; assigned or updated later in the function.

color: `color` stores a computed value for appearance configuration. It is initialized from `normalizeTextColor(style.color || "")`. Within the function it is passed into `normalizeTextColor`, `rgbColorToHex`; assigned or updated later in the function.

`function normalizeFieldStyles(styles = {})`

Role: To understand `normalizeFieldStyles`, start with the problem it owns. `normalizeFieldStyles` standardizes `FieldStyles` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeFieldStyles(styles = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `styles` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `normalizeFieldStyles`

```
function normalizeFieldStyles(styles = {}) {  
  return Object.fromEntries(  
    Object.entries(styles || {}).  
      .map(([fieldId, style]) => [fieldId, normalizePlainFieldStyle(style)])  
      .filter(([, style]) => Object.keys(style).length)  
  );  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalizePlainFieldStyle`. Those calls show that `normalizeFieldStyles` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `normalizeFieldStyles`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `normalizeFieldStyles`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function plainFieldStyleToCss(style = {})`

Role: To understand `plainFieldStyleToCss`, start with the problem it owns. `plainFieldStyleToCss` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function plainFieldStyleToCss(style = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `style` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `plainFieldStyleToCss`

```
function plainFieldStyleToCss(style = {}) {  
  const normalized = normalizePlainFieldStyle(style);  
  const rules = [];  
  if (normalized.fontFamily) rules.push(`font-family: ${normalized.fontFamily}`);  
}
```

```

if (normalized.fontPx) rules.push(`font-size: ${normalized.fontPx}px`);
if (normalized.color) rules.push(`color: ${normalized.color}`);
if (normalized.bold) rules.push("font-weight: 700");
if (normalized.italic) rules.push("font-style: italic");
if (normalized.underline) rules.push("text-decoration: underline");
return rules.join("; ");
}

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalizePlainFieldStyle`. Those calls show that `plainFieldStyleToCss` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `plainFieldStyleToCss`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: After the main flow, the working values inside `plainFieldStyleToCss` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

normalized: `normalized` stores a computed value for temporary calculation. It is initialized from `normalizePlainFieldStyle(style)`. Within the function it accesses properties/methods `bold`, `color`, `fontFamily`, `fontPx`, `italic`, `underline`; passed into `push`; assigned or updated later in the function.

rules: `rules` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

function applyPlainFieldStyle(element, fieldId = "")

Role: The best entry point for `applyPlainFieldStyle` is its responsibility in the surrounding workflow. `applyPlainFieldStyle` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function applyPlainFieldStyle(element, fieldId = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `element` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. `fieldId` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for applyPlainFieldStyle

```

function applyPlainFieldStyle(element, fieldId = "") {
  if (!element) return;
  const style = normalizePlainFieldStyle(fieldStyles[fieldId] || {});
  element.style.fontFamily = style.fontFamily || "";
  element.style.fontSize = style.fontPx ? `${style.fontPx}px` : "";
  element.style.color = style.color || "";
  element.style.fontWeight = style.bold ? "700" : "";
}

```

```

    element.style.fontStyle = style.italic ? "italic" : "";
    element.style.textDecoration = style.underline ? "underline" : "";
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `normalizePlainFieldStyle`. Those calls show that `applyPlainFieldStyle` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `applyPlainFieldStyle`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The easiest way to follow the body of `applyPlainFieldStyle` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

style: `style` stores a computed value for appearance configuration. It is initialized from `normalizePlainFieldStyle(fieldStyles[fieldId] || {})`. Within the function it is accessed properties/methods `bold`, `color`, `fontFamily`, `fontPx`, `fontSize`, `fontStyle`, `fontWeight`, `italic`; assigned or updated later in the function.

function plainFieldStyleAttribute(fieldId = "")

Role: To understand `plainFieldStyleAttribute`, start with the problem it owns. `plainFieldStyleAttribute` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function plainFieldStyleAttribute(fieldId = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `fieldId` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for plainFieldStyleAttribute

```

function plainFieldStyleAttribute(fieldId = "") {
  const css = plainFieldStyleToCss(fieldStyles[fieldId] || {});
  return css ? `style="${escapeHtml(css)}"` : "";
}

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `plainFieldStyleToCss`, `escapeHtml`. Those calls show that `plainFieldStyleAttribute` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `plainFieldStyleAttribute`, the specific rule it owns would disappear from the named workflow. Callers would either

skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: After the main flow, the working values inside `plainFieldStyleAttribute` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

css: `css` stores a computed value for temporary calculation. It is initialized from `plainFieldStyleToCss(fieldStyles[fieldId] || {})`. Within the function it is returned to the caller; passed into `escapeHtml`; assigned or updated later in the function.

function `normalizeCropAspect(value = "")`

Role: To understand `normalizeCropAspect`, start with the problem it owns. `normalizeCropAspect` standardizes `CropAspect` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeCropAspect(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `normalizeCropAspect`

```
function normalizeCropAspect(value = "") {  
  return ["1 / 1", "4 / 3", "16 / 9", "3 / 4"].includes(value) ? value : "original";  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `normalizeCropAspect` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `normalizeCropAspect`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `normalizeCropAspect`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `normalizeCropZoom(value = 1)`

Role: Read `normalizeCropZoom` first as a named idea, not as syntax. `normalizeCropZoom` standardizes `CropZoom` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeCropZoom(value = 1) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for normalizeCropZoom

```
function normalizeCropZoom(value = 1) {  
  const zoom = Number(value);  
  return Number.isFinite(zoom) ? Math.min(3, Math.max(1, zoom)) : 1;  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means normalizeCropZoom performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without normalizeCropZoom, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside normalizeCropZoom show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

zoom: zoom stores a computed value for temporary calculation. It is initialized from Number(value). Within the function it is passed into isFinite, min; assigned or updated later in the function.

function normalizeCropPosition(value = 50)

Role: Read normalizeCropPosition first as a named idea, not as syntax. normalizeCropPosition standardizes CropPosition data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeCropPosition(value = 50) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. value is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for normalizeCropPosition

```
function normalizeCropPosition(value = 50) {  
  const position = Number(value);  
  return Number.isFinite(position) ? Math.min(100, Math.max(0, position)) : 50;  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `normalizeCropPosition` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `normalizeCropPosition`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside `normalizeCropPosition` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

position: `position` stores a computed value for temporary calculation. It is initialized from `Number(value)`. Within the function it is passed into `isFinite`, `min`; assigned or updated later in the function.

function canonicalTemplateId(id)

Role: To understand `canonicalTemplateId`, start with the problem it owns. `canonicalTemplateId` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

Function entry point:

```
function canonicalTemplateId(id) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `id` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for canonicalTemplateId

```
function canonicalTemplateId(id) {  
  return legacyTemplateSkins[id] || id || "";  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `canonicalTemplateId` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `canonicalTemplateId`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `canonicalTemplateId`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function projectTemplateId(project)

Role: Begin with the role of projectTemplateId. projectTemplateId handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Function entry point:

```
function projectTemplateId(project) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for projectTemplateId

```
function projectTemplateId(project) {  
  return canonicalTemplateId(project?.portfolioView?.template?.id || project?.templateId || "");  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with canonicalTemplateId. Those calls show that projectTemplateId is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without projectTemplateId, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside projectTemplateId, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function projectTemplateClass(project)

Role: To understand projectTemplateClass, start with the problem it owns. projectTemplateClass handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Function entry point:

```
function projectTemplateClass(project) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for projectTemplateClass

```
function projectTemplateClass(project) {  
  const templateId = projectTemplateId(project);  
  return templateId ? `project-template project-template-${templateId}` : "project-template-white";  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `projectTemplateId`. Those calls show that `projectTemplateClass` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `projectTemplateClass`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: After the main flow, the working values inside `projectTemplateClass` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

templateId: `templateId` stores a computed value for appearance configuration. It is initialized from `projectTemplateId(project)`. Within the function it is returned to the caller; assigned or updated later in the function.

function responsiveClassName(value, fallback)

Role: To understand `responsiveClassName`, start with the problem it owns. `responsiveClassName` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function responsiveClassName(value, fallback) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `fallback` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for responsiveClassName

```
function responsiveClassName(value, fallback) {  
  return String(value || fallback || "")  
    .toLowerCase()  
    .replace(/^[^a-z0-9]+/g, "-")  
    .replace(/^-|-$/g, "") || fallback;  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `responsiveClassName` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `responsiveClassName`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `responsiveClassName`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function projectResponsiveClass(project)

Role: The best entry point for `projectResponsiveClass` is its responsibility in the surrounding workflow. `projectResponsiveClass` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Function entry point:

```
function projectResponsiveClass(project) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for projectResponsiveClass

```
function projectResponsiveClass(project) {  
  const profile = projectResponsiveProfile(project);  
  return `responsive-project responsive-density-${responsiveClassName(profile.density, "balanced")} responsive-card-  
    ${responsiveClassName(profile.cardLayout, "single")}`;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `projectResponsiveProfile`, `responsiveClassName`. Those calls show that `projectResponsiveClass` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `projectResponsiveClass`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The easiest way to follow the body of `projectResponsiveClass` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

profile: `profile` stores a computed value for portfolio data state. It is initialized from `projectResponsiveProfile(project)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `cardLayout`, `density`; passed into `responsiveClassName`; assigned or updated later in the function.

function responsiveStyleValues(project)

Role: The best entry point for `responsiveStyleValues` is its responsibility in the surrounding workflow. `responsiveStyleValues` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function responsiveStyleValues(project) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `responsiveStyleValues`

```
function responsiveStyleValues(project) {  
  const profile = projectResponsiveProfile(project);  
  return {  
    "--responsive-section-card-min": profile.sectionCardMin,  
    "--responsive-content-max": profile.contentMax,  
    "--responsive-media-max": profile.mediaMax,  
    "--responsive-touch-target": profile.touchTarget  
  };  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `projectResponsiveProfile`. Those calls show that `responsiveStyleValues` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `responsiveStyleValues`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The easiest way to follow the body of `responsiveStyleValues` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

profile: `profile` stores a computed value for portfolio data state. It is initialized from `projectResponsiveProfile(project)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `contentMax`, `mediaMax`, `sectionCardMin`, `touchTarget`; assigned or updated later in the function.

function `hasPublicTemplate(project)`

Role: The best entry point for `hasPublicTemplate` is its responsibility in the surrounding workflow. `hasPublicTemplate` is a decision predicate. It answers one yes/no question so the larger workflow can branch clearly instead of embedding the same condition in several places.

Function entry point:

```
function hasPublicTemplate(project) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `hasPublicTemplate`

```
function hasPublicTemplate(project) {
```

```
    return Boolean(projectTemplateId(project));
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `projectTemplateId`. Those calls show that `hasPublicTemplate` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `hasPublicTemplate`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `hasPublicTemplate`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function projectTemplateVisual(project)

Role: The best entry point for `projectTemplateVisual` is its responsibility in the surrounding workflow. `projectTemplateVisual` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Function entry point:

```
function projectTemplateVisual(project) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for projectTemplateVisual

```
function projectTemplateVisual(project) {
  return project?.portfolioView?.template?.visual || project?.templateVisual || null;
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `projectTemplateVisual` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `projectTemplateVisual`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `projectTemplateVisual`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function projectTemplateStyle(project, accent)

Role: Begin with the role of `projectTemplateStyle`. `projectTemplateStyle` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Function entry point:

```
function projectTemplateStyle(project, accent) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `accent` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `projectTemplateStyle`

```
function projectTemplateStyle(project, accent) {  
  const visual = projectTemplateVisual(project) || {};  
  const values = {  
    "--accent": accent,  
    "--template-bg": visual.background,  
    "--template-panel": visual.panel,  
    "--template-accent": visual.accent,  
    "--template-hover": visual.hover,  
    "--template-click": visual.click,  
    "--template-click-text": visual.clickText,  
    "--template-line": visual.line,  
    "--template-text": visual.text,  
    ...responsiveStyleValues(project)  
  };  
  return Object.entries(values)  
    .filter(([ , value]) => value)  
    .map(([key, value]) => `${key}: ${value}`)  
    .join("; ");  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `projectTemplateVisual`, `responsiveStyleValues`. Those calls show that `projectTemplateStyle` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `projectTemplateStyle`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local state inside `projectTemplateStyle` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

visual: `visual` stores a computed value for temporary calculation. It is initialized from `projectTemplateVisual(project) || {}`. Within the function it accesses properties/methods `accent`, `background`, `click`, `clickText`, `hover`, `line`, `panel`, `text`; assigned or updated later in the function.

values: `values` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is passed into `entries`; assigned or updated later in the function.

function `applyProjectTemplateToElement(element, project, accent)`

Role: Read `applyProjectTemplateToElement` first as a named idea, not as syntax. `applyProjectTemplateToElement` belongs to appearance and responsive presentation. This function applies

templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function applyProjectTemplateToElement(element, project, accent) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `element` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `accent` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `applyProjectTemplateToElement`

```
function applyProjectTemplateToElement(element, project, accent) {
  if (!element) return;
  [...element.classList]
    .filter((className) =>
      className === "project-template" ||
      className === "project-template-white" ||
      className === "responsive-project" ||
      className.startsWith("project-template-") ||
      className.startsWith("responsive-density-") ||
      className.startsWith("responsive-card-")
    )
    .forEach((className) => element.classList.remove(className));
  projectTemplateClass(project).split(" ").forEach((className) => element.classList.add(className));
  projectResponsiveClass(project).split(" ").forEach((className) => element.classList.add(className));
  const visual = projectTemplateVisual(project) || {};
  const values = {
    "--accent": accent,
    "--template-bg": visual.background,
    "--template-panel": visual.panel,
    "--template-accent": visual.accent,
    "--template-hover": visual.hover,
    "--template-click": visual.click,
    "--template-click-text": visual.clickText,
    "--template-line": visual.line,
    "--template-text": visual.text,
    ...responsiveStyleValues(project)
  };
  Object.entries(values).forEach(([key, value]) => {
    if (value) {
      element.style.setProperty(key, value);
    } else {
      element.style.removeProperty(key);
    }
  });
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

Collaborators: The function also has to be read through the helpers it calls. It works with `projectTemplateClass`, `projectResponsiveClass`, `projectTemplateVisual`, `responsiveStyleValues`. Those calls show that `applyProjectTemplateToElement` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `applyProjectTemplateToElement`, the specific rule it owns would disappear from the named

workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside `applyProjectTemplateToElement` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

visual: `visual` stores a computed value for temporary calculation. It is initialized from `projectTemplateVisual(project) || {}`. Within the function it accesses properties/methods `accent`, `background`, `click`, `clickText`, `hover`, `line`, `panel`, `text`; assigned or updated later in the function.

values: `values` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is passed into entries; assigned or updated later in the function.

function responsiveChildren(node)

Role: The best entry point for `responsiveChildren` is its responsibility in the surrounding workflow. `responsiveChildren` belongs to appearance and responsive presentation. This function applies templates, visual style, font/color/crop settings, or responsive layout decisions. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function responsiveChildren(node) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `node` is a DOM object or parsed content node. The function reads from it, writes into it, or uses it as the current place where UI or project content should be rendered.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for responsiveChildren

```
function responsiveChildren(node) {  
  return node?.items || node?.children || [];  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `responsiveChildren` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `responsiveChildren`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: Inside `responsiveChildren`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function normalizeTextColor(value = "")

Role: To understand `normalizeTextColor`, start with the problem it owns. `normalizeTextColor` standardizes `TextColor` data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeTextColor(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `normalizeTextColor`

```
function normalizeTextColor(value = "") {  
  const color = String(value || "").trim();  
  return /^#([0-9a-f]{3}|[0-9a-f]{6})$/i.test(color) ||  
    /^rgba?\(/i.test(color) ||  
    /^hsla?\(/i.test(color)  
    ? color  
    : "";  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `normalizeTextColor` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `normalizeTextColor`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: After the main flow, the working values inside `normalizeTextColor` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

color: `color` stores a computed value for appearance configuration. It is initialized from `String(value || "").trim()`. Within the function it is passed into `test`; assigned or updated later in the function.

Project directory

The functions in this group all support project directory. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function `flattenProject(project)`

Role: To understand `flattenProject`, start with the problem it owns. `flattenProject` gathers scattered nested data into a shape that another function can loop over. This is important because portfolio data is hierarchical while rendering, search, and AI context often need a flat list.

Function entry point:

```
function flattenProject(project) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for flattenProject

```
function flattenProject(project) {
  const categoryLabel = slugLabel(project.category);
  const electronicsKeywords = window.electronicsSearchKeywords
    ? window.electronicsSearchKeywords(project, categoryLabel)
    : [];
  const richSummaryTerms = (project.summaryRich?.blocks || []).flatMap((block) => [
    block.text,
    block.formula,
    block.title,
    block.caption
  ]);
  const parsedChildTerms = (project.portfolioView?.sections || []).flatMap((section) =>
    (section.items || []).flatMap(parsedItemTerms)
  );
  const design = project.design || {};
  const designTerms = [
    design.brief?.summary,
    ...(design.brief?.files || []).flatMap((item) => [item.title, item.description, item.url]),
    design.documentation?.summary,
    ...(design.documentation?.files || []).flatMap((item) => [item.title, item.description, item.url]),
    ...(design.documentation?.references || []).flatMap((item) => [item.title, item.description, item.url]),
    ...(design.documentation?.mathAnalysis || []).flatMap((item) => [item.title, item.description]),
    design.simulation?.summary,
    ...(design.simulation?.files || []).flatMap((item) => [item.title, item.description, item.url]),
    ...(design.simulation?.results || []).flatMap((item) => [item.title, item.description, item.url])
  ];
  return [
    project.id,
    project.title,
    categoryLabel,
    project.status,
    project.summary,
    ...richSummaryTerms,
    ...parsedChildTerms,
    ...designTerms,
    ...(project.focus || []),
    ...(project.highlights || []),
    ...(project.tools || []).map((item) => typeof item === "string" ? item : [item.name, item.title, item.label,
item.description].filter(Boolean).join(" ")),
    ...(project.languages || []),
    ...(project.documents || []).flatMap((item) => [item.title, item.type, item.status, item.url]),
    ...(project.tests || []).flatMap((item) => [item.name, item.method, item.status, item.result, item.artifact]),
    ...(project.pcbS || []).flatMap((item) => [item.name, item.revision, item.status, item.artifact]),
    ...(project.media || []).flatMap((item) => [item.title, item.caption, item.url]),
    ...(project.sections || []).flatMap((section) => [
      section.title,
      section.description,
      ...(section.items || []).flatMap((item) => [item.title, item.description, item.type, item.status, item.url])
    ]),
    ...(project.links || []).flatMap((item) => [item.label, item.url]),
    ...electronicsKeywords
  ]
  .map(normalize)
  .join(" ");
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Next, trace the helper calls that surround the function. It works with slugLabel. Those calls show that flattenProject is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without flattenProject, the specific rule it owns would disappear from the named workflow. Callers would either skip that

behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: After the main flow, the working values inside `flattenProject` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

categoryLabel: `categoryLabel` stores a computed value for portfolio data state. It is initialized from `slugLabel(project.category)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `electronicsSearchKeywords`; assigned or updated later in the function.

electronicsKeywords: `electronicsKeywords` stores a computed value for temporary calculation. It is initialized from `window.electronicsSearchKeywords`. Within the function it is assigned or updated later in the function.

richSummaryTerms: `richSummaryTerms` stores a function or callback value for lookup collection. It is initialized from `(project.summaryRich?.blocks || []).flatMap((block) => [...])`. Within the function it is assigned or updated later in the function.

parsedChildTerms: `parsedChildTerms` stores a function or callback value for lookup collection. It is initialized from `(project.portfolioView?.sections || []).flatMap((section) => [...])`. Within the function it is assigned or updated later in the function.

design: `design` stores a computed value for temporary calculation. It is initialized from `project.design || {}`. Within the function it is accesses properties/methods `brief`, `documentation`, `simulation`; assigned or updated later in the function.

designTerms: `designTerms` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is assigned or updated later in the function.

function projectVisible(project, query)

Role: To understand `projectVisible`, start with the problem it owns. `projectVisible` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Function entry point:

```
function projectVisible(project, query) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for projectVisible

```
function projectVisible(project, query) {  
  const categoryMatch = activeFilter === "all" || project.category === activeFilter;  
  return categoryMatch && projectMatches(project, query);  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `projectMatches`. Those calls show that `projectVisible` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `projectVisible`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: After the main flow, the working values inside `projectVisible` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

categoryMatch: `categoryMatch` stores a computed value for portfolio data state. It is initialized from `activeFilter === "all" || project.category === activeFilter`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is returned to the caller; assigned or updated later in the function.

function normalizeCategory(category = {})

Role: The best entry point for `normalizeCategory` is its responsibility in the surrounding workflow. `normalizeCategory` standardizes Category data before another part of the app uses it. The point is not decoration; it prevents different callers from interpreting the same input differently. A normalizer usually accepts loose user, catalog, or file data and turns it into the one shape the rest of the subsystem expects.

Function entry point:

```
function normalizeCategory(category = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `category` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for normalizeCategory

```
function normalizeCategory(category = {}) {  
  const label = String(category.label || category.title || category.id || "Project category").trim() || "Project  
category";  
  return {  
    accent: normalizeTextColor(category.accent || "") || "#1677a8",  
    description: String(category.description || "").trim(),  
    id: String(category.id || label.toLowerCase().replace(/^[a-z0-9]+/g, "-").replace(/^-|$/g, "")) || "category",  
    label  
  };  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `normalizeTextColor`. Those calls show that `normalizeCategory` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `normalizeCategory`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The easiest way to follow the body of `normalizeCategory` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

label: `label` stores a computed value for temporary calculation. It is initialized from `String(category.label || category.title || category.id || "Project category").trim() || "Project category"`. Within the function it is accessed, properties/methods `toLowerCase`; passed into `String`; assigned or updated later in the function.

function hydrateProjectCategories(rawCategories = [], rawProjects = [])

Role: Read `hydrateProjectCategories` first as a named idea, not as syntax. `hydrateProjectCategories` belongs to project directory. This function normalizes, flattens, filters, or renders projects and categories. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function hydrateProjectCategories(rawCategories = [], rawProjects = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `rawCategories` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `rawProjects` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for hydrateProjectCategories

```
function hydrateProjectCategories(rawCategories = [], rawProjects = []) {  
  const normalized = rawCategories.map(normalizeCategory);  
  const knownIds = new Set(normalized.map((category) => category.id));  
  rawProjects.forEach((project) => {  
    const id = String(project?.category || "").trim();  
    if (!id || knownIds.has(id)) return;  
    const label = id  
      .split(/[-_\s]+/)   
      .filter(Boolean)   
      .map((part) => part.charAt(0).toUpperCase() + part.slice(1))   
      .join(" ") || "Project category";  
    normalized.push(normalizeCategory({ id, label, description: "Project category" }));  
    knownIds.add(id);  
  });  
  return normalized;  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. It works with `normalizeCategory`. Those calls show that `hydrateProjectCategories` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `hydrateProjectCategories`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside `hydrateProjectCategories` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

normalized: normalized stores a computed value for lookup collection. It is initialized from `rawCategories.map(normalizeCategory)`. Within the function it is returned to the caller; iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods `map`, `push`; passed into `Set`.

knownIds: knownIds stores a constructed object for lookup collection. It is initialized from `new Set(normalized.map((category) => category.id))`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `add`, `has`; assigned or updated later in the function.

id: id stores a computed value for temporary calculation. It is initialized from `String(project?.category || "").trim()`. Within the function it is passed into `add`, `push`; assigned or updated later in the function.

label: label stores a computed value for temporary calculation. It is initialized from `id`. Within the function it is passed into `push`; assigned or updated later in the function.

function projectCard(project)

Role: To understand `projectCard`, start with the problem it owns. `projectCard` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Function entry point:

```
function projectCard(project) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for projectCard

```
function projectCard(project) {
  if (project.portfolioView) {
    const category = categories.find((item) => item.id === project.category) || {};
    const accent = category.accent || "#117c7a";
    const sections = project.portfolioView.sections || [];
    const briefSection = sections.find((section) => section.id === "brief");
    const otherSections = sections.filter((section) => section.id !== "brief" &&
sectionHasRenderableContent(section));

    return `
      <article class="project-card catalog-card ${projectTemplateClass(project)} ${projectResponsiveClass(project)}"
id="${project.id}" style="${projectTemplateStyle(project, accent)}">
        <div class="project-body">
          <h3>${project.portfolioView.title || project.title}</h3>
          ${renderParsedBriefBlock(briefSection, project.summary)}
          <div class="evidence-grid" aria-label="${project.title} parsed project content">
            ${otherSections.map((section, index) => parsedSection(section, index, project)).join("")}
          </div>
        </div>
      </article>
    `;
  }

  const category = categories.find((item) => item.id === project.category) || {};
  const accent = category.accent || "#117c7a";

  return `
    <article class="project-card catalog-card ${projectTemplateClass(project)} ${projectResponsiveClass(project)}"
id="${project.id}" style="${projectTemplateStyle(project, accent)}">
      <div class="project-body">
        <h3>${project.title}</h3>
        <p class="rich-paragraph">${renderMultilineInlineText(project.summary)}</p>
      </div>
    </article>
  `;
}
```

```

    ${project.highlights && project.highlights.length ? detailBlock("Project highlights", "project-drawer", `
    <ul class="highlight-list">
      ${project.highlights.map((item) => `<li>${item}</li>`).join("")}
    </ul>
    `) : ""}

    <div class="evidence-grid" aria-label="${project.title} evidence blocks">
      ${detailBlock("Documents", "evidence-block", evidenceList(project.documents, (item) => `
        <li>
          ${resourceLink(item, item.title)}
          <span>${item.type || "Document"} &middot; ${item.status || "tracked"}</span>
        </li>
        `, "No document artifact has been added yet."))}

      ${detailBlock("Tests and results", "evidence-block", evidenceList(project.tests, (item) => `
        <li>
          ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
          <span>${item.method || "Validation"} &middot; ${item.status || "tracked"}</span>
          ${item.result ? `<p>${item.result}</p>` : ""}
        </li>
        `, "No test artifact has been added yet."))}

      ${detailBlock("PCBs built", "evidence-block", evidenceList(project.pcb, (item) => `
        <li>
          ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
          <span>${item.revision || "Revision"} &middot; ${item.status || "tracked"}</span>
        </li>
        `, "No PCB build has been added yet."))}

      ${detailBlock("Images", "evidence-block evidence-wide", mediaGrid(project.media))}
    ${customSectionBlocks(project)}
    </div>

    ${detailBlock("Tools and implementation files", "project-drawer", `
    <div class="project-tooling">
      <div>
        <h4>Tools Used</h4>
        <div class="project-meta">${pillList(project.tools, "tool-tag")}</div>
      </div>
      <div>
        <h4>Languages</h4>
        <div class="project-meta">${pillList(project.languages, "language-tag")}</div>
      </div>
    </div>
    `)}

    <div class="resource-list">
      ${project.links || []}.map((item) => resourceLink(item)).join("")
    </div>
  </div>
</article>
`;
}

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Next, trace the helper calls that surround the function. It works with `sectionHasRenderableContent`, `projectTemplateClass`, `projectResponsiveClass`, `projectTemplateStyle`, `renderParsedBriefBlock`, `parsedSection`, `renderMultilineInlineText`, `detailBlock`. Those calls show that `projectCard` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `projectCard`, the specific rule it owns would disappear from the named workflow. Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: After the main flow, the working values inside projectCard reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

category: category stores a function or callback value for portfolio data state. It is initialized from `categories.find((item) => item.id === project.category) || {}`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `accent`; assigned or updated later in the function.

accent: `accent` stores a computed value for temporary calculation. It is initialized from `category.accent || "#117c7a"`. Within the function it is passed into `projectTemplateStyle`; assigned or updated later in the function.

sections: `sections` stores a computed value for portfolio data state. It is initialized from `project.portfolioView.sections || []`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `find`; assigned or updated later in the function.

briefSection: `briefSection` stores a function or callback value for portfolio data state. It is initialized from `sections.find((section) => section.id === "brief")`. Within the function it is passed into `renderParsedBriefBlock`; assigned or updated later in the function.

otherSections: `otherSections` stores a function or callback value for portfolio data state. It is initialized from `sections.filter((section) => section.id !== "brief" && sectionHasRenderableContent(section))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `map`; assigned or updated later in the function.

category: category stores a function or callback value for portfolio data state. It is initialized from `categories.find((item) => item.id === project.category) || {}`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `accent`; assigned or updated later in the function.

accent: `accent` stores a computed value for temporary calculation. It is initialized from `category.accent || "#117c7a"`. Within the function it is passed into `projectTemplateStyle`; assigned or updated later in the function.

function loadProjectCatalog()

Role: Read `loadProjectCatalog` first as a named idea, not as syntax. `loadProjectCatalog` is the public website startup pipeline. It chooses embedded preview data when the builder is previewing, otherwise it fetches `projects.json` from the published site. After that it hydrates the major runtime state: categories, projects, profile, site content, fun facts, custom sections, text styles, and rich text variants. Then it renders the visible page and restores any deep-linked project window. This function is `async` because fetching `projects.json` can take time and can fail. It is the reason the static HTML shell becomes a real portfolio. Without it, the page would show placeholders but no saved projects, profile details, resume, fun facts, project categories, search index, or assistant context.

Function entry point:

```
function loadProjectCatalog() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for loadProjectCatalog

```
function loadProjectCatalog() {  
  if (window.__PORTFOLIO_CATALOG__) {  
    projects = window.__PORTFOLIO_CATALOG__.projects || [];  
    categories = hydrateProjectCategories(window.__PORTFOLIO_CATALOG__.categories || [], projects);  
    siteSections = window.__PORTFOLIO_CATALOG__.siteSections || [];  
    fieldStyles = normalizeFieldStyles(window.__PORTFOLIO_CATALOG__.fieldStyles || {});  
  }  
}
```

```

    siteContent = normalizeSiteContent(window.__PORTFOLIO_CATALOG__.siteContent || {});
    siteContentRich = window.__PORTFOLIO_CATALOG__.siteContentRich || {};
    profile = normalizeProfile(window.__PORTFOLIO_CATALOG__.profile || {});
    profileRich = window.__PORTFOLIO_CATALOG__.profileRich || {};
    funFacts = normalizeFunFacts(window.__PORTFOLIO_CATALOG__.funFacts || []);
    funFactsRich = window.__PORTFOLIO_CATALOG__.funFactsRich || null;
    renderProfile();
    renderSiteContent();
    renderFunFacts();
    renderSiteSections();
    renderCategoryFilters();
    renderProjects();
    rebuildSearchIndex();
    updateSearchDropdown();
    restoreSectionRouteAfterCatalogLoad();
    return;
}

const catalogUrl = new URL("projects.json", window.location.href);
catalogUrl.searchParams.set("v", String(Date.now()));
fetch(catalogUrl.href, { cache: "no-store" })
    .then((response) => {
        if (!response.ok) throw new Error("Could not load projects.json");
        return response.json();
    })
    .then((data) => {
        projects = data.projects || [];
        categories = hydrateProjectCategories(data.categories || [], projects);
        siteSections = data.siteSections || [];
        fieldStyles = normalizeFieldStyles(data.fieldStyles || {});
        siteContent = normalizeSiteContent(data.siteContent || {});
        siteContentRich = data.siteContentRich || {};
        profile = normalizeProfile(data.profile || {});
        profileRich = data.profileRich || {};
        funFacts = normalizeFunFacts(data.funFacts || []);
        funFactsRich = data.funFactsRich || null;
        renderProfile();
        renderSiteContent();
        renderFunFacts();
        renderSiteSections();
        renderCategoryFilters();
        renderProjects();
        rebuildSearchIndex();
        updateSearchDropdown();
        restoreSectionRouteAfterCatalogLoad();
    })
    .catch(() => {
        grid.innerHTML = `
            <div class="empty-state">
                <h3>Project data did not load.</h3>
                <p>The project catalog is temporarily unavailable.</p>
            </div>
        `;
    });
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it calls a network resource and updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: throws explicit errors when a required safety or compatibility condition fails; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter; performs a fetch and therefore must handle network delay and failure.

Collaborators: The function also has to be read through the helpers it calls. It works with `hydrateProjectCategories`, `normalizeFieldStyles`, `normalizeSiteContent`, `normalizeProfile`, `normalizeFunFacts`, `renderProfile`, `renderSiteContent`, `renderFunFacts`. Those calls show that `loadProjectCatalog` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `loadProjectCatalog`, the specific rule it owns would disappear from the named workflow.

Callers would either skip that behavior or reimplement it inconsistently, which is exactly how rendering, publishing, compiling, searching, or AI context drifts over time.

Local values: The local objects and variables inside `loadProjectCatalog` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

catalogUrl: `catalogUrl` stores a constructed object for network or routing value. It is initialized from `new URL("projects.json", window.location.href)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accessed properties/methods `href`, `searchParams`; passed into `fetch`; assigned or updated later in the function.

Search and filtering

The functions in this group all support search and filtering. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function projectMatches(project, query)

Role: Begin with the role of `projectMatches`. `projectMatches` handles project-directory structure. It helps turn saved catalog data into visible categories, cards, filters, responsive layout, or searchable project text.

Function entry point:

```
function projectMatches(project, query) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for projectMatches

```
function projectMatches(project, query) {  
  return !query || flattenProject(project).includes(query);  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `flattenProject`. Those calls show that `projectMatches` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `projectMatches`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: Inside `projectMatches`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function setActiveFilter(filter = "all")

Role: Read `setActiveFilter` first as a named idea, not as syntax. `setActiveFilter` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function setActiveFilter(filter = "all") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `filter` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `setActiveFilter`

```
function setActiveFilter(filter = "all") {  
  activeFilter = filter;  
  if (activeFilter !== "all" && !categories.some((category) => category.id === activeFilter)) {  
    activeFilter = "all";  
  }  
  filterButtons.forEach((item) => item.classList.toggle("active", item.dataset.filter === activeFilter));  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `setActiveFilter` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `setActiveFilter`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: Inside `setActiveFilter`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

`function bindFilterButtons()`

Role: Read `bindFilterButtons` first as a named idea, not as syntax. `bindFilterButtons` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function bindFilterButtons() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `bindFilterButtons`

```
function bindFilterButtons() {  
  filterButtons = [...document.querySelectorAll(".filter-button")];  
  filterButtons.forEach((button) => {  
    button.addEventListener("click", () => {  
      setActiveFilter(button.dataset.filter || "all");  
      renderProjects();  
      updateSearchDropdown();  
    });  
  });  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: attaches event handlers that turn user actions into behavior.

Collaborators: The function also has to be read through the helpers it calls. It works with `setActiveFilter`, `renderProjects`, `updateSearchDropdown`. Those calls show that `bindFilterButtons` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `bindFilterButtons`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: Inside `bindFilterButtons`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderCategoryFilters()

Role: The best entry point for `renderCategoryFilters` is its responsibility in the surrounding workflow. `renderCategoryFilters` turns saved data for `CategoryFilters` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Function entry point:

```
function renderCategoryFilters() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for renderCategoryFilters

```
function renderCategoryFilters() {  
  if (projectTrackCount) {  
    projectTrackCount.textContent = `${categories.length} Track${categories.length === 1 ? "" : "s"}`;  
  }  
  if (projectTrackLabels) {  
    projectTrackLabels.textContent = categories.length  
      ? categories.map((category) => category.label).join(", ")  
      : "project categories";  
  }  
  if (projectFilters) {  
    projectFilters.innerHTML = `  
      <button class="filter-button active" type="button" data-filter="all">All</button>  
      ${categories.map((category) => `  
        <button class="filter-button" type="button" data-filter="${escapeHtml(category.id)}">  
{escapeHtml(category.label)}</button>  
      `).join("")}  
    `;  
  }  
  bindFilterButtons();  
  setActiveFilter(activeFilter);  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `escapeHtml`, `bindFilterButtons`, `setActiveFilter`. Those calls show that `renderCategoryFilters` is not isolated; it is

one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `renderCategoryFilters`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: Inside `renderCategoryFilters`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function flattenSearchText(values = [])

Role: To understand `flattenSearchText`, start with the problem it owns. `flattenSearchText` gathers scattered nested data into a shape that another function can loop over. This is important because portfolio data is hierarchical while rendering, search, and AI context often need a flat list.

Function entry point:

```
function flattenSearchText(values = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `values` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for flattenSearchText

```
function flattenSearchText(values = []) {  
  return values  
    .flat(Infinity)  
    .filter((value) => value !== null && value !== undefined)  
    .map((value) => typeof value === "object" ? JSON.stringify(value) : String(value))  
    .join(" ")  
    .replace(/\s+/g, " ")  
    .trim();  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: serializes structured data before sending or saving it.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means `flattenSearchText` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `flattenSearchText`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: Inside `flattenSearchText`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function addSearchEntry(entries, entry)

Role: To understand `addSearchEntry`, start with the problem it owns. `addSearchEntry` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function addSearchEntry(entries, entry) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `entries` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `entry` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `addSearchEntry`

```
function addSearchEntry(entries, entry) {  
  const text = flattenSearchText([entry.title, entry.type, entry.context, entry.text, entry.url]);  
  if (!entry.title && !text) return null;  
  const storedEntry = {  
    ...entry,  
    normalizedText: normalize(text),  
    text  
  };  
  entries.push(storedEntry);  
  return storedEntry;  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `flattenSearchText`, `normalize`. Those calls show that `addSearchEntry` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `addSearchEntry`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: After the main flow, the working values inside `addSearchEntry` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

text: `text` stores a computed value for temporary calculation. It is initialized from `flattenSearchText([entry.title, entry.type, entry.context, entry.text, entry.url])`. Within the function it is passed into `flattenSearchText`, `normalize`; assigned or updated later in the function.

storedEntry: `storedEntry` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is returned to the caller; passed into `push`; assigned or updated later in the function.

`function searchSnippet(text = "", query = "")`

Role: To understand `searchSnippet`, start with the problem it owns. `searchSnippet` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function searchSnippet(text = "", query = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `text` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller

omits that field. query is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for searchSnippet

```
function searchSnippet(text = "", query = "") {  
  const clean = String(text || "").replace(/\s+/g, " ").trim();  
  if (!clean) return "";  
  const index = normalize(clean).indexOf(normalize(query));  
  if (index < 0) return clean.slice(0, 150);  
  const start = Math.max(0, index - 54);  
  const end = Math.min(clean.length, index + query.length + 80);  
  return `${start ? "... " : ""}${clean.slice(start, end)}${end < clean.length ? "... " : ""}`;  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Next, trace the helper calls that surround the function. It works with normalize. Those calls show that searchSnippet is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without searchSnippet, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: After the main flow, the working values inside searchSnippet reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from `String(text || "").replace(/\s+/g, " ").trim()`. Within the function it is returned to the caller; accesses properties/methods `length`, `slice`; passed into `min`, `normalize`; assigned or updated later in the function.

index: index stores a computed value for temporary calculation. It is initialized from `normalize(clean).indexOf(normalize(query))`. Within the function it is passed into `max`, `min`; assigned or updated later in the function.

start: start stores a computed value for temporary calculation. It is initialized from `Math.max(0, index - 54)`. Within the function it is passed into `slice`; assigned or updated later in the function.

end: end stores a computed value for temporary calculation. It is initialized from `Math.min(clean.length, index + query.length + 80)`. Within the function it is passed into `slice`; assigned or updated later in the function.

function entryScore(entry, query)

Role: Read entryScore first as a named idea, not as syntax. entryScore belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function entryScore(entry, query) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. entry is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more

predictable data before returning or rendering anything. query is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for entryScore

```
function entryScore(entry, query) {
  const normalizedQuery = normalize(query).trim();
  if (!normalizedQuery) return 0;
  const title = normalize(entry.title);
  const type = normalize(entry.type);
  const context = normalize(entry.context);
  const text = entry.normalizedText || "";
  const words = normalizedQuery.split(/\s+/).filter(Boolean);
  let score = 0;

  if (title === normalizedQuery) score += 160;
  if (title.startsWith(normalizedQuery)) score += 120;
  if (title.includes(normalizedQuery)) score += 90;
  if (context.includes(normalizedQuery)) score += 55;
  if (type.includes(normalizedQuery)) score += 35;
  if (text.includes(normalizedQuery)) score += 45;
  words.forEach((word) => {
    if (title.includes(word)) score += 16;
    if (context.includes(word)) score += 10;
    if (text.includes(word)) score += 6;
  });

  if (score > 0 && entry.kind === "file") score += 18;
  if (score > 0 && entry.kind === "section") score += 14;
  if (score > 0 && entry.kind === "project") score += 12;
  return score;
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. It works with `normalize`. Those calls show that `entryScore` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `entryScore`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: The local objects and variables inside `entryScore` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

normalizedQuery: `normalizedQuery` stores a computed value for temporary calculation. It is initialized from `normalize(query).trim()`. Within the function it is accesses properties/methods `split`; assigned or updated later in the function.

title: `title` stores a computed value for temporary calculation. It is initialized from `normalize(entry.title)`. Within the function it is accesses properties/methods `includes`, `startsWith`; passed into `normalize`; assigned or updated later in the function.

type: `type` stores a computed value for temporary calculation. It is initialized from `normalize(entry.type)`. Within the function it is accesses properties/methods `includes`; passed into `normalize`; assigned or updated later in the function.

context: context stores a computed value for temporary calculation. It is initialized from `normalize(entry.context)`. Within the function it is accesses properties/methods includes; passed into `normalize`; assigned or updated later in the function.

text: text stores a computed value for temporary calculation. It is initialized from `entry.normalizedText || ""`. Within the function it is accesses properties/methods includes; assigned or updated later in the function.

words: words stores a computed value for temporary calculation. It is initialized from `normalizedQuery.split(/\s+/).filter(Boolean)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

score: score stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is returned to the caller; assigned or updated later in the function.

function entryMatches(entry, query)

Role: The best entry point for `entryMatches` is its responsibility in the surrounding workflow. `entryMatches` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function entryMatches(entry, query) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `entry` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for entryMatches

```
function entryMatches(entry, query) {  
  return entryScore(entry, query) > 0;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `entryScore`. Those calls show that `entryMatches` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `entryMatches`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: Inside `entryMatches`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function fileIsBrowserSearchable(url = "")

Role: Read `fileIsBrowserSearchable` first as a named idea, not as syntax. `fileIsBrowserSearchable` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function fileIsBrowserSearchable(url = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `fileIsBrowserSearchable`

```
function fileIsBrowserSearchable(url = "") {  
  const clean = String(url || "").split(/[?#]/)[0].toLowerCase();  
  return /\.(txt|md|markdown|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|v|sv|vhd|?|spice|cir|net|asc|sch|kicad_sch|  
  kicad_pcb)$/i.test(clean);  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `fileIsBrowserSearchable` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `fileIsBrowserSearchable`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: The local objects and variables inside `fileIsBrowserSearchable` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(url || "").split(/[?#]/)[0].toLowerCase()`. Within the function it is passed into `test`; assigned or updated later in the function.

async function `indexSearchableFileText(entry)`

Role: The best entry point for `indexSearchableFileText` is its responsibility in the surrounding workflow. `indexSearchableFileText` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function indexSearchableFileText(entry) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `entry` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `indexSearchableFileText`

```
async function indexSearchableFileText(entry) {
```

```

if (!entry.url || !fileIsBrowserSearchable(entry.url)) return;
if (/^(https?:)?\/\//i.test(entry.url) && !entry.url.startsWith(window.location.origin)) return;
try {
  const response = await fetch(entry.url);
  if (!response.ok) return;
  const text = await response.text();
  entry.text = flattenSearchText([entry.text, text.slice(0, 60000)]);
  entry.normalizedText = normalize(entry.text);
  updateSearchDropdown();
} catch {
  // Files remain searchable by title, path, caption, and catalog text if direct text fetch is unavailable.
}
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it waits for asynchronous work and calls a network resource. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: performs a fetch and therefore must handle network delay and failure.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `fileIsBrowserSearchable`, `flattenSearchText`, `normalize`, `updateSearchDropdown`. Those calls show that `indexSearchableFileText` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `indexSearchableFileText`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: The easiest way to follow the body of `indexSearchableFileText` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

response: `response` stores data returned from a network call. It is initialized from `await fetch(entry.url)`. Within the function it is accesses properties/methods `ok`, `text`; assigned or updated later in the function.

text: `text` stores the completed result of an awaited operation. It is initialized from `await response.text()`. Within the function it is accesses properties/methods `slice`; passed into `flattenSearchText`, `normalize`; assigned or updated later in the function.

function addProjectSearchEntries(entries, project)

Role: Begin with the role of `addProjectSearchEntries`. `addProjectSearchEntries` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function addProjectSearchEntries(entries, project) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `entries` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for addProjectSearchEntries

```

function addProjectSearchEntries(entries, project) {
  const categoryLabel = slugLabel(project.category);
  const baseContext = `${categoryLabel} / ${project.title}`;
  const projectTerms = [

```

```

    project.id,
    project.summary,
    richTextTerms(project.summaryRich),
    project.focus,
    project.highlights,
    project.tools,
    project.languages,
    project.links,
    window.electronicsSearchKeywords ? window.electronicsSearchKeywords(project, categoryLabel) : []
  ];

  addSearchEntry(entries, {
    context: categoryLabel,
    kind: "project",
    projectId: project.id,
    title: project.title,
    type: "Project",
    text: projectTerms
  });

  const sections = (project.portfolioView?.sections || []).filter((section) => section.id !== "brief" &&
sectionHasRenderableContent(section));
  sections.forEach((section, sectionIndex) => {
    addSearchEntry(entries, {
      context: baseContext,
      kind: "section",
      projectId: project.id,
      sectionIndex: String(sectionIndex),
      title: displayTitle(section.title, "Section"),
      type: "Project section",
      text: [section.description, richTextTerms(section.rich), parsedItemTerms(section)]
    });

    const walkItems = (items = [], path = [], parentTitles = []) => {
      items.forEach((item, index) => {
        const nextPath = [...path, index];
        const title = displayTitle(item.title || item.name || item.label || fileNameFromUrl(item.url || ""),
"Project item");
        const url = item.url(item);
        const context = [baseContext, displayTitle(section.title, "Section"),
...parentTitles].filter(Boolean).join(" / ");
        const entry = {
          context,
          kind: url ? "file" : "subsection",
          projectId: project.id,
          resourcePath: pathToString(nextPath),
          sectionIndex: String(sectionIndex),
          title,
          type: url ? "File or uploaded asset" : "Subsection",
          url,
          text: [item.description, item.meta, item.kind, item.type, url, richTextTerms(item.rich),
parsedItemTerms(item)]
        };
        const storedEntry = addSearchEntry(entries, entry);
        if (url && storedEntry) indexSearchableFileText(storedEntry);
        walkItems(nodeChildren(item), nextPath, [...parentTitles, title]);
      });
    };
    walkItems(section.items || []);
  });

  uniqueDownloads(collectDownloadsFromValue(project, "Project file", [])).forEach((download) => {
    const entry = {
      context: baseContext,
      kind: "file",
      projectId: project.id,
      title: download.title || fileNameFromUrl(download.url),
      type: download.type || "Uploaded file",
      url: download.url,
      text: [download.description, download.status, download.url]
    };
    const storedEntry = addSearchEntry(entries, entry);
    if (storedEntry) indexSearchableFileText(storedEntry);
  });
}

```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `slugLabel`, `richTextTerms`, `addSearchEntry`, `sectionHasRenderableContent`, `displayTitle`, `parsedItemTerms`, `fileNameFromUrl`, `itemUrl`. Those calls show that `addProjectSearchEntries` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `addProjectSearchEntries`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: The local state inside `addProjectSearchEntries` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

categoryLabel: `categoryLabel` stores a computed value for portfolio data state. It is initialized from `slugLabel(project.category)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `addSearchEntry`, `electronicsSearchKeywords`; assigned or updated later in the function.

baseContext: `baseContext` stores a string value for temporary calculation. It is initialized from ``${categoryLabel} / ${project.title}``. Within the function it is passed into `addSearchEntry`; assigned or updated later in the function.

projectTerms: `projectTerms` stores a list for portfolio data state. It is initialized from `[]`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `addSearchEntry`; assigned or updated later in the function.

sections: `sections` stores a function or callback value for portfolio data state. It is initialized from `(project.portfolioView?.sections || []).filter((section) => section.id !== "brief" && sectionHasRenderableContent(section))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

walkItems: `walkItems` stores a function or callback value for temporary calculation. It is initialized from `(items = [], path = [], parentTitles = []) => {}`. Within the function it is assigned or updated later in the function.

nextPath: `nextPath` stores a list for filesystem location. It is initialized from `[...path, index]`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is passed into `pathToString`; assigned or updated later in the function.

title: `title` stores a computed value for temporary calculation. It is initialized from `displayTitle(item.title || item.name || item.label || fileNameFromUrl(item.url || ""), "Project item")`. Within the function it is passed into `addSearchEntry`, `displayTitle`; assigned or updated later in the function.

url: `url` stores a computed value for network or routing value. It is initialized from `itemUrl(item)`. Within the function it is passed into `displayTitle`, `fileNameFromUrl`; assigned or updated later in the function.

context: `context` stores a list for temporary calculation. It is initialized from `[baseContext, displayTitle(section.title, "Section"), ...parentTitles].filter(Boolean).join(" / ")`. Within the function it is passed into `addSearchEntry`; assigned or updated later in the function.

entry: `entry` stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is passed into `addSearchEntry`; assigned or updated later in the function.

storedEntry: `storedEntry` stores a computed value for temporary calculation. It is initialized from `addSearchEntry(entries, entry)`. Within the function it is passed into `indexSearchableFileText`; assigned or updated later in the function.

entry: entry stores a structured object for temporary calculation. It is initialized from `{}`. Within the function it is passed into `addSearchEntry`; assigned or updated later in the function.

storedEntry: `storedEntry` stores a computed value for temporary calculation. It is initialized from `addSearchEntry(entries, entry)`. Within the function it is passed into `indexSearchableFileText`; assigned or updated later in the function.

function addSiteSectionSearchEntries(entries)

Role: The best entry point for `addSiteSectionSearchEntries` is its responsibility in the surrounding workflow. `addSiteSectionSearchEntries` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function addSiteSectionSearchEntries(entries) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `entries` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for addSiteSectionSearchEntries

```
function addSiteSectionSearchEntries(entries) {
  const current = normalizeProfile(profile || {});
  const pageSections = [
    { id: "top", title: heroTitle?.textContent || "Front page", type: "Page section", text:
[heroEyebrow?.textContent, heroCopy?.textContent] },
    { id: "projects", title: "Engineering Projects", type: "Directory", text: "project directory categories files
sections" },
    { id: "process", title: "Process", type: "Page section", text: "engineering process design testing
documentation" },
    { id: "contact", title: "Contact", type: "Page section", text: [current.displayName, current.email,
current.phone, current.githubUrl, current.linkedinUrl, current.websiteUrl, current.contactIntro] }
  ];
  if (current.resumeUrl) {
    pageSections.splice(2, 0, { id: "resume", title: "Resume", type: "Page section", text: "resume professional
profile document" });
  }
  pageSections.forEach((section) => addSearchEntry(entries, {
    domTarget: section.id,
    kind: "page",
    title: section.title,
    type: section.type,
    text: section.text
  }));

  if (current.resumeUrl) {
    const resumeTextEntry = addSearchEntry(entries, {
      context: "Professional Profile / Resume",
      kind: "file",
      title: "Resume",
      type: "Resume",
      url: current.resumeUrl,
      text: [current.displayName, current.portfolioLabel, current.contactIntro, current.resumeUrl]
    });
    if (resumeTextEntry) indexSearchableFileText(resumeTextEntry);
  }

  (siteSections || []).filter(siteSectionRenderable).forEach((section) => {
    addSearchEntry(entries, {
      domTarget: section.id,
      kind: "page",
      title: section.title || "Portfolio section",
      type: "Portfolio section",
```

```

    text: [section.description, richTextTerms(section.richDescription), section.links, section.assets]
  });
  (section.subsections || []).forEach((item) => addSearchEntry(entries, {
    domTarget: section.id,
    kind: "page",
    title: item.title || "Portfolio subsection",
    type: "Portfolio subsection",
    text: [item.description, richTextTerms(item.richDescription), item.links]
  }));
});
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `normalizeProfile`, `addSearchEntry`, `indexSearchableFileText`, `richTextTerms`. Those calls show that `addSiteSectionSearchEntries` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `addSiteSectionSearchEntries`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: The easiest way to follow the body of `addSiteSectionSearchEntries` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

current: `current` stores a computed value for temporary calculation. It is initialized from `normalizeProfile(profile || {})`. Within the function it accesses properties/methods `contactIntro`, `displayName`, `email`, `githubUrl`, `linkedinUrl`, `phone`, `portfolioLabel`, `resumeUrl`; passed into `addSearchEntry`; assigned or updated later in the function.

pageSections: `pageSections` stores a list for portfolio data state. It is initialized from `.`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`, `splice`; assigned or updated later in the function.

resumeTextEntry: `resumeTextEntry` stores a computed value for temporary calculation. It is initialized from `addSearchEntry(entries, {`. Within the function it is passed into `indexSearchableFileText`; assigned or updated later in the function.

function rebuildSearchIndex()

Role: To understand `rebuildSearchIndex`, start with the problem it owns. `rebuildSearchIndex` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function rebuildSearchIndex() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for rebuildSearchIndex

```

function rebuildSearchIndex() {
  const entries = [];
  addSiteSectionSearchEntries(entries);
  projects.forEach((project) => addProjectSearchEntries(entries, project));
  searchableEntries = entries;
}

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: Next, trace the helper calls that surround the function. It works with `addSiteSectionSearchEntries`, `addProjectSearchEntries`. Those calls show that `rebuildSearchIndex` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `rebuildSearchIndex`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: After the main flow, the working values inside `rebuildSearchIndex` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

entries: `entries` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is passed into `addProjectSearchEntries`, `addSiteSectionSearchEntries`; assigned or updated later in the function.

function `searchResultsFor(query)`

Role: The best entry point for `searchResultsFor` is its responsibility in the surrounding workflow. `searchResultsFor` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function searchResultsFor(query) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `searchResultsFor`

```
function searchResultsFor(query) {  
  const normalizedQuery = normalize(query).trim();  
  if (!normalizedQuery) return [];  
  const seen = new Set();  
  return searchableEntries  
    .map((entry) => ({ ...entry, score: entryScore(entry, normalizedQuery) })))  
    .filter((entry) => entry.score > 0)  
    .sort((a, b) => b.score - a.score || a.title.localeCompare(b.title))  
    .filter((entry) => {  
      const key = [entry.kind, entry.projectId, entry.sectionIndex, entry.resourcePath, entry.domTarget, entry.url,  
entry.title].join("|");  
      if (seen.has(key)) return false;  
      seen.add(key);  
      return true;  
    }));  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `normalize`, `entryScore`. Those calls show that `searchResultsFor` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `searchResultsFor`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: The easiest way to follow the body of `searchResultsFor` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

normalizedQuery: `normalizedQuery` stores a computed value for temporary calculation. It is initialized from `normalize(query).trim()`. Within the function it is passed into `entryScore`; assigned or updated later in the function.

seen: `seen` stores a constructed object for lookup collection. It is initialized from `new Set()`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `add`, `has`; assigned or updated later in the function.

key: `key` stores a list for temporary calculation. It is initialized from `[entry.kind, entry.projectId, entry.sectionIndex, entry.resourcePath, entry.domTarget, entry.url, entry.title].join("|")`. Within the function it is passed into `add`; assigned or updated later in the function.

function highlightQueryHtml(value = "", query = "")

Role: The best entry point for `highlightQueryHtml` is its responsibility in the surrounding workflow. `highlightQueryHtml` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function highlightQueryHtml(value = "", query = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for highlightQueryHtml

```
function highlightQueryHtml(value = "", query = "") {  
  const escaped = escapeHtml(value);  
  const cleanQuery = String(query || "").trim();  
  if (!cleanQuery) return escaped;  
  const pattern = new RegExp(`(${escapeRegExp(escapeHtml(cleanQuery))})`, "ig");  
  return escaped.replace(pattern, '<mark class="search-result-mark">$1</mark>');  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `escapeHtml`, `escapeRegExp`. Those calls show that `highlightQueryHtml` is not isolated; it is one piece of a larger

workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `highlightQueryHtml`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: The easiest way to follow the body of `highlightQueryHtml` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

escaped: `escaped` stores a computed value for temporary calculation. It is initialized from `escapeHtml(value)`. Within the function it is returned to the caller; accesses properties/methods `replace`; assigned or updated later in the function.

cleanQuery: `cleanQuery` stores a computed value for temporary calculation. It is initialized from `String(query || "").trim()`. Within the function it is passed into `RegExp`; assigned or updated later in the function.

pattern: `pattern` stores a constructed object for temporary calculation. It is initialized from `new RegExp(`${escapeRegExp(escapeHtml(cleanQuery))}`, "ig")`. Within the function it is passed into `replace`; assigned or updated later in the function.

function clearSearchHighlights()

Role: Begin with the role of `clearSearchHighlights`. `clearSearchHighlights` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function clearSearchHighlights() { ... }
```

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for clearSearchHighlights

```
function clearSearchHighlights() {  
  window.CSS?.highlights?.delete?.(searchHighlightName);  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. No other named helper from this file is detected inside the body. That means `clearSearchHighlights` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `clearSearchHighlights`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: Inside `clearSearchHighlights`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function textNodeSearchTargets()

Role: The best entry point for `textNodeSearchTargets` is its responsibility in the surrounding workflow. `textNodeSearchTargets` belongs to search and filtering. This function builds, scores, filters, highlights, or

navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function textNodeSearchTargets() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for textNodeSearchTargets

```
function textNodeSearchTargets() {  
  return [  
    document.querySelector("main"),  
    document.querySelector("#section-view-dialog[open] .section-view-content")  
  ].filter(Boolean);  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it updates browser DOM. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means textNodeSearchTargets performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without textNodeSearchTargets, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: Inside textNodeSearchTargets, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function applySearchHighlights(query)

Role: The best entry point for applySearchHighlights is its responsibility in the surrounding workflow. applySearchHighlights belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function applySearchHighlights(query) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. query is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for applySearchHighlights

```
function applySearchHighlights(query) {  
  clearSearchHighlights();  
  const cleanQuery = String(query || "").trim();  
  if (!cleanQuery || !window.CSS?.highlights || typeof window.Highlight !== "function") return 0;  
  const ranges = [];  
  const needle = normalize(cleanQuery);  
  
  textNodeSearchTargets().forEach((root) => {  
    const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {
```

```

    acceptNode(node) {
      if (!node.textContent || !normalize(node.textContent).includes(needle)) return NodeFilter.FILTER_REJECT;
      if (node.parentElement?.closest("script, style, input, textarea, select, button, .search-engine, .site-
header")) return NodeFilter.FILTER_REJECT;
      return NodeFilter.FILTER_ACCEPT;
    }
  });
  let node = walker.nextNode();
  while (node) {
    const haystack = normalize(node.textContent);
    let index = haystack.indexOf(needle);
    while (index >= 0) {
      const range = document.createRange();
      range.setStart(node, index);
      range.setEnd(node, index + cleanQuery.length);
      ranges.push(range);
      index = haystack.indexOf(needle, index + Math.max(1, cleanQuery.length));
    }
    node = walker.nextNode();
  }
});

if (ranges.length) window.CSS.highlights.set(searchHighlightName, new window.Highlight(...ranges));
return ranges.length;
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `clearSearchHighlights`, `normalize`, `textNodeSearchTargets`. Those calls show that `applySearchHighlights` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `applySearchHighlights`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: The easiest way to follow the body of `applySearchHighlights` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

cleanQuery: `cleanQuery` stores a computed value for temporary calculation. It is initialized from `String(query || "").trim()`. Within the function it is accesses properties/methods `length`; passed into `indexOf`, `normalize`, `setEnd`; assigned or updated later in the function.

ranges: `ranges` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `length`, `push`; passed into `set`; assigned or updated later in the function.

needle: `needle` stores a computed value for temporary calculation. It is initialized from `normalize(cleanQuery)`. Within the function it is passed into `includes`, `indexOf`; assigned or updated later in the function.

walker: `walker` stores a DOM element reference. It is initialized from `document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {`. Within the function it is accesses properties/methods `nextNode`; assigned or updated later in the function.

node: `node` stores a computed value for temporary calculation. It is initialized from `walker.nextNode()`. Within the function it is accesses properties/methods `parentElement`, `textContent`; passed into `createTreeWalker`, `normalize`, `setEnd`, `setStart`; assigned or updated later in the function.

haystack: `haystack` stores a computed value for runtime state or cache. It is initialized from `normalize(node.textContent)`. Within the function it is accesses properties/methods `indexOf`; assigned or updated later in the function.

index: index stores a computed value for temporary calculation. It is initialized from `haystack.indexOf(needle)`. Within the function it is passed into `indexOf`, `setEnd`, `setStart`; assigned or updated later in the function.

range: range stores a DOM element reference. It is initialized from `document.createRange()`. Within the function it accesses properties/methods `setEnd`, `setStart`; passed into `push`; assigned or updated later in the function.

function queueSearchHighlights()

Role: Read `queueSearchHighlights` first as a named idea, not as syntax. `queueSearchHighlights` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function queueSearchHighlights() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for queueSearchHighlights

```
function queueSearchHighlights() {  
  clearTimeout(searchHighlightTimer);  
  searchHighlightTimer = setTimeout(() => applySearchHighlights(searchInput?.value || ""), 30);  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it uses a timer to avoid blocking or to wait for another process. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The function also has to be read through the helpers it calls. It works with `applySearchHighlights`. Those calls show that `queueSearchHighlights` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `queueSearchHighlights`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: Inside `queueSearchHighlights`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function ensureSearchPanel()

Role: Begin with the role of `ensureSearchPanel`. `ensureSearchPanel` creates or repairs something only when it is needed. This pattern avoids duplicate DOM nodes, duplicate repositories, duplicate handlers, or missing folders while still letting later code assume the resource exists.

Function entry point:

```
function ensureSearchPanel() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for ensureSearchPanel

```
function ensureSearchPanel() {
```

```

if (!searchInput || !searchWrap || searchPanel) return;
const shell = document.createElement("div");
shell.className = "search-engine";
searchWrap.replaceWith(shell);
shell.append(searchWrap);
searchPanel = document.createElement("div");
searchPanel.className = "search-results-panel";
searchPanel.hidden = true;
searchPanel.innerHTML = `
  <div class="search-results-tools">
    <span id="search-status">Type to search projects, files, sections, and phrases.</span>
    <label>
      <span>Show</span>
      <select id="search-result-limit" aria-label="Number of search results">
        <option value="5">5</option>
        <option value="10" selected>10</option>
        <option value="15">15</option>
        <option value="20">20</option>
      </select>
    </label>
  </div>
  <div class="search-results-list" role="listbox" aria-label="Search results"></div>
`;
shell.append(searchPanel);
searchStatus = searchPanel.querySelector("#search-status");
searchLimitSelect = searchPanel.querySelector("#search-result-limit");
searchLimitSelect.addEventListener("change", () => {
  searchResultLimit = Number(searchLimitSelect.value) || 10;
  updateSearchDropdown();
});
searchPanel.addEventListener("mousedown", (event) => event.preventDefault());
searchPanel.addEventListener("click", (event) => {
  const button = event.target.closest("[data-search-result-index]");
  if (!button) return;
  const result = currentSearchResults[Number(button.dataset.searchResultIndex)];
  if (result) goToSearchResult(result);
});
}

```

Implementation: The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: creates DOM elements instead of relying only on static HTML; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter; attaches event handlers that turn user actions into behavior.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `updateSearchDropdown`, `goToSearchResult`. Those calls show that `ensureSearchPanel` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `ensureSearchPanel`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: The local state inside `ensureSearchPanel` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

shell: `shell` stores a DOM element reference. It is initialized from `document.createElement("div")`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `append`, `className`; passed into `replaceWith`; assigned or updated later in the function.

button: `button` stores a computed value for DOM reference or UI state. It is initialized from `event.target.closest("[data-search-result-index]")`. It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods `dataset`; passed into `Number`; assigned or updated later in the function.

result: result stores a computed value for temporary calculation. It is initialized from `currentSearchResults[Number(button.dataset.searchResultIndex)]`. Within the function it is passed into `closest, goToSearchResult, querySelector`; assigned or updated later in the function.

function setSearchMessage(message, state = "neutral")

Role: The best entry point for `setSearchMessage` is its responsibility in the surrounding workflow. `setSearchMessage` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function setSearchMessage(message, state = "neutral") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `message` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `state` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for setSearchMessage

```
function setSearchMessage(message, state = "neutral") {  
  if (!searchStatus) return;  
  searchStatus.textContent = message;  
  searchStatus.dataset.state = state;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `setSearchMessage` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `setSearchMessage`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: Inside `setSearchMessage`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderSearchResults(query, visibleCount)

Role: Begin with the role of `renderSearchResults`. `renderSearchResults` turns saved data for `SearchResults` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Function entry point:

```
function renderSearchResults(query, visibleCount) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `query` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response

behavior. `visibleCount` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `renderSearchResults`

```
function renderSearchResults(query, visibleCount) {
  ensureSearchPanel();
  if (!searchPanel) return;
  const list = searchPanel.querySelector(".search-results-list");
  const cleanQuery = String(query || "").trim();
  currentSearchResults = searchResultsFor(cleanQuery);
  const limitedResults = currentSearchResults.slice(0, searchResultLimit);
  searchPanel.hidden = !cleanQuery;
  searchInput.setAttribute("aria-expanded", String(Boolean(cleanQuery)));

  if (!cleanQuery) {
    list.innerHTML = "";
    setSearchMessage("Type to search projects, files, sections, and phrases.");
    searchInput.removeAttribute("aria-invalid");
    return;
  }

  if (!currentSearchResults.length) {
    list.innerHTML = "";
    setSearchMessage(`${cleanQuery} was not found on this portfolio.`, "not-found");
    searchInput.setAttribute("aria-invalid", "true");
    return;
  }

  searchInput.removeAttribute("aria-invalid");
  const visibleText = visibleCount
    ? `${visibleCount} visible match${visibleCount === 1 ? "" : "es"} highlighted.`
    : "Matches found elsewhere. Choose a result below.";
  setSearchMessage(`${visibleText} Showing ${limitedResults.length} of ${currentSearchResults.length}.`, "found");
  list.innerHTML = limitedResults.map((result, index) => `
    <button class="search-result-item" type="button" role="option" data-search-result-index="${index}">
      <span class="search-result-title">${highlightQueryHtml(result.title || "Untitled result", cleanQuery)}</span>
      <span class="search-result-meta">${escapeHtml([result.type, result.context].filter(Boolean).join(" / "))}</span>
      ${searchSnippet(result.text, cleanQuery)} ? <span class="search-result-snippet">${
        highlightQueryHtml(searchSnippet(result.text, cleanQuery), cleanQuery)}</span>` : ""`
    </button>
  `).join("");
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `ensureSearchPanel`, `searchResultsFor`, `setSearchMessage`, `highlightQueryHtml`, `escapeHtml`, `searchSnippet`. Those calls show that `renderSearchResults` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `renderSearchResults`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: The local state inside `renderSearchResults` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

list: list stores a DOM element reference. It is initialized from `searchPanel.querySelector(".search-results-list")`. Within the function it is accesses properties/methods `innerHTML`; passed into `querySelector`; assigned or updated later in the function.

cleanQuery: `cleanQuery` stores a computed value for temporary calculation. It is initialized from `String(query || "").trim()`. Within the function it is passed into `highlightQueryHtml`, `searchResultsFor`, `searchSnippet`, `setAttribute`, `setSearchMessage`; assigned or updated later in the function.

limitedResults: `limitedResults` stores a computed value for temporary calculation. It is initialized from `currentSearchResults.slice(0, searchResultLimit)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; passed into `setSearchMessage`; assigned or updated later in the function.

visibleText: `visibleText` stores a computed value for temporary calculation. It is initialized from `visibleCount`. Within the function it is passed into `setSearchMessage`; assigned or updated later in the function.

function updateSearchDropdown()

Role: Read `updateSearchDropdown` first as a named idea, not as syntax. `updateSearchDropdown` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function updateSearchDropdown() { ... }
```

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for updateSearchDropdown

```
function updateSearchDropdown() {  
  const query = searchInput?.value || "";  
  const visibleCount = applySearchHighlights(query);  
  renderSearchResults(query, visibleCount);  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The function also has to be read through the helpers it calls. It works with `applySearchHighlights`, `renderSearchResults`. Those calls show that `updateSearchDropdown` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `updateSearchDropdown`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: The local objects and variables inside `updateSearchDropdown` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

query: `query` stores a computed value for temporary calculation. It is initialized from `searchInput?.value || ""`. Within the function it is passed into `applySearchHighlights`, `renderSearchResults`; assigned or updated later in the function.

visibleCount: visibleCount stores a computed value for temporary calculation. It is initialized from applySearchHighlights(query). Within the function it is passed into renderSearchResults; assigned or updated later in the function.

function showSearchPanelIfNeeded()

Role: The best entry point for showSearchPanelIfNeeded is its responsibility in the surrounding workflow. showSearchPanelIfNeeded belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function showSearchPanelIfNeeded() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for showSearchPanelIfNeeded

```
function showSearchPanelIfNeeded() {  
  if (!searchInput?.value?.trim()) return;  
  updateSearchDropdown();  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with updateSearchDropdown. Those calls show that showSearchPanelIfNeeded is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without showSearchPanelIfNeeded, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: Inside showSearchPanelIfNeeded, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function goToSearchResult(result)

Role: Read goToSearchResult first as a named idea, not as syntax. goToSearchResult turns a search dropdown item into navigation. Depending on what the result represents, it may open a project window, jump to a main page section, highlight text on the current page, or open a file/link. This makes search feel like a search engine instead of a passive list. The function is necessary because search results can point to many different kinds of things: project titles, section overviews, files, custom sections, contact links, or page text. One navigation rule would not work for all of them.

Function entry point:

```
function goToSearchResult(result) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. result is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string

data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for goToSearchResult

```
function goToSearchResult(result) {
  if (!result) return;
  searchPanel.hidden = true;
  if (result.domTarget && scrollToDomTarget(result.domTarget)) return;

  if (result.projectId && result.sectionIndex !== undefined) {
    openParsedSection(result.projectId, result.sectionIndex, result.resourcePath || "");
    queueSearchHighlights();
    return;
  }

  if (result.projectId) {
    if (!document.getElementById(result.projectId)) {
      setActiveFilter("all");
      renderProjects();
    }
    const target = document.getElementById(result.projectId);
    target?.scrollIntoView({ behavior: "smooth", block: "center" });
    queueSearchHighlights();
    return;
  }

  if (result.url) window.open(result.url, "_blank", "noopener");
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The function also has to be read through the helpers it calls. It works with `scrollToDomTarget`, `openParsedSection`, `queueSearchHighlights`, `setActiveFilter`, `renderProjects`. Those calls show that `goToSearchResult` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `goToSearchResult`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: The local objects and variables inside `goToSearchResult` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

target: `target` stores a DOM element reference. It is initialized from `document.getElementById(result.projectId)`. Within the function it is assigned or updated later in the function.

function handleSearchInput()

Role: Read `handleSearchInput` first as a named idea, not as syntax. `handleSearchInput` belongs to search and filtering. This function builds, scores, filters, highlights, or navigates search results so the public website behaves like a real searchable portfolio. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function handleSearchInput() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for handleSearchInput

```
function handleSearchInput() {
  renderProjects();
  updateSearchDropdown();
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The function also has to be read through the helpers it calls. It works with `renderProjects`, `updateSearchDropdown`. Those calls show that `handleSearchInput` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `handleSearchInput`, search would become less like a search engine. Results could be missing, poorly ranked, not highlighted, or unable to navigate the visitor to the exact project, section, file, or phrase.

Local values: Inside `handleSearchInput`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

Portfolio AI frontend

The functions in this group all support portfolio ai frontend. Read them as a small subsystem: the smaller helpers prepare data and the larger functions coordinate side effects or rendering.

function assistantEndpoint()

Role: The best entry point for `assistantEndpoint` is its responsibility in the surrounding workflow. `assistantEndpoint` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantEndpoint() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantEndpoint

```
function assistantEndpoint() {
  if (typeof window.OMB_AI_ENDPOINT === "string") return window.OMB_AI_ENDPOINT.trim();
  const metaEndpoint = document.querySelector('meta[name="portfolio-ai-endpoint"]')?.content || "";
  if (metaEndpoint.trim()) return String(metaEndpoint).trim();
  if (["localhost", "127.0.0.1"].includes(window.location.hostname)) return "/api/portfolio-ai";
  return "/api/portfolio-ai";
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it updates browser DOM. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `assistantEndpoint` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantEndpoint`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The easiest way to follow the body of `assistantEndpoint` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

metaEndpoint: `metaEndpoint` stores a DOM element reference. It is initialized from `document.querySelector('meta[name="portfolio-ai-endpoint"]')?.content || ""`. Within the function it is accessed properties/methods `trim`; passed into `String`; assigned or updated later in the function.

function assistantSourceLabel(result = {})

Role: The best entry point for `assistantSourceLabel` is its responsibility in the surrounding workflow. `assistantSourceLabel` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantSourceLabel(result = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `result` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for assistantSourceLabel

```
function assistantSourceLabel(result = {}) {  
  const title = result.title || "Portfolio item";  
  if (result.kind === "project" || result.kind === "page") return title;  
  const contextParts = String(result.context || "")  
    .split("/")  
    .map((item) => item.trim())  
    .filter(Boolean);  
  const specificContext = contextParts.slice(-2).join(" / ");  
  return [title, specificContext].filter(Boolean).join(" - ");  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `assistantSourceLabel` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantSourceLabel`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The easiest way to follow the body of `assistantSourceLabel` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

title: `title` stores a computed value for temporary calculation. It is initialized from `result.title || "Portfolio item"`. Within the function it is returned to the caller; assigned or updated later in the function.

contextParts: `contextParts` stores a computed value for temporary calculation. It is initialized from `String(result.context || "")`. Within the function it is accessed properties/methods `slice`; assigned or updated later in the function.

specificContext: specificContext stores a resolved filesystem or route value. It is initialized from contextParts.slice(-2).join(" / "). Within the function it is assigned or updated later in the function.

function assistantProjectSummary(project = {})

Role: The best entry point for assistantProjectSummary is its responsibility in the surrounding workflow. assistantProjectSummary is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantProjectSummary(project = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantProjectSummary

```
function assistantProjectSummary(project = {}) {  
  const categoryLabel = slugLabel(project.category);  
  const sections = (project.portfolioView?.sections || []).  
    .filter((section) => section.id !== "brief" && sectionHasRenderableContent(section))  
    .map((section) => displayTitle(section.title, "Section"));  
  return {  
    category: categoryLabel,  
    focus: project.focus || [],  
    id: project.id,  
    links: project.links || [],  
    sections,  
    summary: project.summary || "",  
    title: project.portfolioView?.title || project.title,  
    tools: project.tools || []  
  };  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with slugLabel, sectionHasRenderableContent, displayTitle. Those calls show that assistantProjectSummary is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without assistantProjectSummary, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The easiest way to follow the body of assistantProjectSummary is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

categoryLabel: categoryLabel stores a computed value for portfolio data state. It is initialized from slugLabel(project.category). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

sections: sections stores a function or callback value for portfolio data state. It is initialized from (project.portfolioView?.sections || []). Within the function it is assigned or updated later in the function.

function assistantProjectTitleAcronyms(title = "")

Role: The best entry point for assistantProjectTitleAcronyms is its responsibility in the surrounding workflow. assistantProjectTitleAcronyms is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantProjectTitleAcronyms(title = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. title is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantProjectTitleAcronyms

```
function assistantProjectTitleAcronyms(title = "") {  
  const words = normalize(title)  
    .split(/\s+/)  
    .filter((word) => word.length > 2);  
  const acronyms = new Set();  
  for (let start = 0; start < words.length; start += 1) {  
    for (let length = 2; length <= Math.min(5, words.length - start); length += 1) {  
      acronyms.add(words.slice(start, start + length).map((word) => word[0]).join(""));  
    }  
  }  
  return [...acronyms].filter((item) => item.length >= 2);  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with normalize. Those calls show that assistantProjectTitleAcronyms is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without assistantProjectTitleAcronyms, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The easiest way to follow the body of assistantProjectTitleAcronyms is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

words: words stores a computed value for temporary calculation. It is initialized from normalize(title). Within the function it is accesses properties/methods length, slice; passed into add; assigned or updated later in the function.

acronyms: acronyms stores a constructed object for lookup collection. It is initialized from new Set(). Within the function it is mutated as a collection or DOM container; accesses properties/methods add; assigned or updated later in the function.

start: start stores a numeric value for temporary calculation. It is initialized from 0. Within the function it is passed into add; assigned or updated later in the function.

length: length stores a numeric value for temporary calculation. It is initialized from 2. Within the function it is passed into add; assigned or updated later in the function.

function assistantProjectIsNamedInQuestion(question = "", project = {})

Role: To understand assistantProjectIsNamedInQuestion, start with the problem it owns.

assistantProjectIsNamedInQuestion is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantProjectIsNamedInQuestion(question = "", project = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. project identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantProjectIsNamedInQuestion

```
function assistantProjectIsNamedInQuestion(question = "", project = {}) {  
  const clean = normalize(question);  
  if (!clean) return false;  
  const fillerWords = new Set(["with", "from", "into", "using", "project", "design", "system", "systems"]);  
  const rawTitle = project.portfolioView?.title || project.title || project.id;  
  const title = normalize(rawTitle);  
  if (!title) return false;  
  if (clean.includes(title)) return true;  
  const cleanWords = new Set(clean.split(/\s+/));  
  const acronymMatch = assistantProjectTitleAcronyms(rawTitle)  
    .some((acronym) => cleanWords.has(acronym));  
  if (acronymMatch) return true;  
  const words = title.split(/\s+/).filter((word) => word.length > 3 && !fillerWords.has(word));  
  if (!words.length) return false;  
  const matches = words.filter((word) => clean.includes(word)).length;  
  return words.length <= 2 ? matches === words.length : matches >= Math.min(3, words.length);  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with normalize, assistantProjectTitleAcronyms. Those calls show that assistantProjectIsNamedInQuestion is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without assistantProjectIsNamedInQuestion, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: After the main flow, the working values inside assistantProjectIsNamedInQuestion reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: clean stores a computed value for temporary calculation. It is initialized from normalize(question). Within the function it is accesses properties/methods includes, split; passed into Set; assigned or updated later in the function.

fillerWords: fillerWords stores a constructed object for lookup collection. It is initialized from new Set(["with", "from", "into", "using", "project", "design", "system", "systems"]). Within the function it is accesses properties/methods has; assigned or updated later in the function.

rawTitle: rawTitle stores a computed value for temporary calculation. It is initialized from project.portfolioView?.title || project.title || project.id. Within the function it is passed into assistantProjectTitleAcronyms, normalize; assigned or updated later in the function.

title: title stores a computed value for temporary calculation. It is initialized from normalize(rawTitle). Within the function it is accesses properties/methods split; assigned or updated later in the function.

cleanWords: cleanWords stores a constructed object for lookup collection. It is initialized from new Set(clean.split(/\s+/)). Within the function it is accesses properties/methods has; assigned or updated later in the function.

acronymMatch: acronymMatch stores a computed value for temporary calculation. It is initialized from assistantProjectTitleAcronyms(rawTitle). Within the function it is assigned or updated later in the function.

words: words stores a function or callback value for temporary calculation. It is initialized from title.split(/\s+/).filter((word) => word.length > 3 && !fillerWords.has(word)). Within the function it is returned to the caller; iterated or transformed as a collection; accesses properties/methods filter, length; passed into min; assigned or updated later in the function.

matches: matches stores a function or callback value for temporary calculation. It is initialized from words.filter((word) => clean.includes(word)).length. Within the function it is assigned or updated later in the function.

function assistantNamedProjectIds(question = "")

Role: Read assistantNamedProjectIds first as a named idea, not as syntax. assistantNamedProjectIds is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantNamedProjectIds(question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantNamedProjectIds

```
function assistantNamedProjectIds(question = "") {  
  return projects  
    .filter((project) => assistantProjectIsNamedInQuestion(question, project))  
    .map((project) => project.id);  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. It works with assistantProjectIsNamedInQuestion. Those calls show that assistantNamedProjectIds is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `assistantNamedProjectIds`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: Inside `assistantNamedProjectIds`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function assistantProjectTitleMatches(question = "")

Role: Begin with the role of `assistantProjectTitleMatches`. `assistantProjectTitleMatches` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantProjectTitleMatches(question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for assistantProjectTitleMatches

```
function assistantProjectTitleMatches(question = "") {  
  return assistantNamedProjectIds(question).length > 0;  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `assistantNamedProjectIds`. Those calls show that `assistantProjectTitleMatches` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `assistantProjectTitleMatches`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: Inside `assistantProjectTitleMatches`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function assistantUrlKind(url = "")

Role: The best entry point for `assistantUrlKind` is its responsibility in the surrounding workflow. `assistantUrlKind` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantUrlKind(url = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the

app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantUrlKind

```
function assistantUrlKind(url = "") {  
  const clean = String(url || "").split(/[?#]/)[0].toLowerCase();  
  if (!clean) return "unknown";  
  if (/github\.com|raw\.githubusercontent\.com|gist\.githubusercontent\.com/i.test(clean)) return "github";  
  if (/linkedin\.com/i.test(clean)) return "linkedin";  
  if (/\. (png|jpe?g|gif|webp|svg)$/i.test(clean)) return "image";  
  if (/\.pdf$/i.test(clean)) return "pdf";  
  if (/\. (zip|7z|rar)$/i.test(clean)) return "archive";  
  if (/\. (xlsx?|csv)$/i.test(clean)) return "spreadsheet";  
  if (fileIsBrowserSearchable(clean)) return "text";  
  if (/^https?:\/\//i.test(clean)) return "webpage";  
  return "file";  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `fileIsBrowserSearchable`. Those calls show that `assistantUrlKind` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantUrlKind`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The easiest way to follow the body of `assistantUrlKind` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `String(url || "").split(/[?#]/)[0].toLowerCase()`. Within the function it is passed into `test`; assigned or updated later in the function.

function assistantAbsoluteUrl(url = "")

Role: Begin with the role of `assistantAbsoluteUrl`. `assistantAbsoluteUrl` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantAbsoluteUrl(url = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `url` points at an external resource, local file, route, Git remote, or public link. The function normally normalizes or validates it before using it so the app does not treat unsafe or malformed paths as trusted input. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantAbsoluteUrl

```
function assistantAbsoluteUrl(url = "") {
  const target = normalizeLinkTarget(url, { assumeWeb: looksLikeBareWebAddress(url) || /^https?:\/\//i.test(url) });
  if (!target) return "";
  try {
    return new URL(target, window.location.href).href;
  } catch {
    return target;
  }
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalizeLinkTarget`, `looksLikeBareWebAddress`. Those calls show that `assistantAbsoluteUrl` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `assistantAbsoluteUrl`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local state inside `assistantAbsoluteUrl` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

target: `target` stores a computed value for temporary calculation. It is initialized from `normalizeLinkTarget(url, { assumeWeb: looksLikeBareWebAddress(url) || /^https?:\/\//i.test(url) })`. Within the function it is returned to the caller; passed into `URL`; assigned or updated later in the function.

function assistantLinkRecordsFromValue(value, context = "")

Role: The best entry point for `assistantLinkRecordsFromValue` is its responsibility in the surrounding workflow. `assistantLinkRecordsFromValue` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantLinkRecordsFromValue(value, context = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `context` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for assistantLinkRecordsFromValue

```
function assistantLinkRecordsFromValue(value, context = "") {
  const records = [];
  const downloads = uniqueDownloads(collectDownloadsFromValue(value, "Portfolio asset", []));
  downloads.forEach((download) => {
    const rawUrl = download.url || download.href || download.path || "";
    if (!rawUrl) return;
    records.push({
      context,
      description: download.description || "",
      kind: download.kind || assistantUrlKind(rawUrl),
      label: download.title || download.label || fileNameFromUrl(rawUrl),
      url: assistantAbsoluteUrl(rawUrl)
    });
  });
}
```

```
    return records;
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `uniqueDownloads`, `collectDownloadsFromValue`, `assistantUrlKind`, `fileNameFromUrl`, `assistantAbsoluteUrl`. Those calls show that `assistantLinkRecordsFromValue` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantLinkRecordsFromValue`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The easiest way to follow the body of `assistantLinkRecordsFromValue` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

records: `records` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods `push`; assigned or updated later in the function.

downloads: `downloads` stores a computed value for temporary calculation. It is initialized from `uniqueDownloads(collectDownloadsFromValue(value, "Portfolio asset", []))`. Within the function it is iterated or transformed as a collection; accesses properties/methods `forEach`; assigned or updated later in the function.

rawUrl: `rawUrl` stores a computed value for network or routing value. It is initialized from `download.url || download.href || download.path || ""`. Within the function it is passed into `assistantAbsoluteUrl`, `fileNameFromUrl`, `push`; assigned or updated later in the function.

function assistantUrlsFromTextValue(value, context = "")

Role: Read `assistantUrlsFromTextValue` first as a named idea, not as syntax. `assistantUrlsFromTextValue` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantUrlsFromTextValue(value, context = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. `context` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantUrlsFromTextValue

```
function assistantUrlsFromTextValue(value, context = "") {
  const text = flattenSearchText([value]);
  const matches = text.match(/\b(?:https?:\/\/|www\.)[^<"]+\/gi) || [];
  return [...new Set(matches.map(url => url.replace(/[.,;:!?]+$/g, "")))]
    .slice(0, 16)
    .map(url => ({
      context,
      description: "",
```

```

    kind: assistantUrlKind(url),
    label: /github\.com/i.test(url) ? "GitHub repository or profile" : fileNameFromUrl(url),
    type: "Public link",
    url: assistantAbsoluteUrl(url)
  }));
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: The function also has to be read through the helpers it calls. It works with `flattenSearchText`, `assistantUrlKind`, `fileNameFromUrl`, `assistantAbsoluteUrl`. Those calls show that `assistantUrlsFromTextValue` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `assistantUrlsFromTextValue`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local objects and variables inside `assistantUrlsFromTextValue` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

text: `text` stores a computed value for temporary calculation. It is initialized from `flattenSearchText([value])`. Within the function it is accessed via `properties/methods match`; assigned or updated later in the function.

matches: `matches` stores a computed value for temporary calculation. It is initialized from `text.match(/\b(?:https?:\V|www\.)[^s<>"])+/gi) || []`. Within the function it is iterated or transformed as a collection; accesses `properties/methods map`; passed into `Set`; assigned or updated later in the function.

function assistantPublicProfileLinks()

Role: The best entry point for `assistantPublicProfileLinks` is its responsibility in the surrounding workflow. `assistantPublicProfileLinks` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantPublicProfileLinks() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantPublicProfileLinks

```

function assistantPublicProfileLinks() {
  const records = [];
  const add = (item = {}, context = "") => {
    const rawUrl = item.url || item.href || item.path || "";
    if (!rawUrl) return;
    const label = item.label || item.title || fileNameFromUrl(rawUrl);
    records.push({
      context,
      kind: item.kind || assistantUrlKind(rawUrl),
      label,
      text: item.description || "",
      url: assistantAbsoluteUrl(rawUrl)
    });
  };

  const current = normalizeProfile(profile || {});
  if (current.resumeUrl) add({ label: "Resume", kind: "resume", url: current.resumeUrl }, "Professional Profile");
}

```

```

    if (current.githubUrl) add({ label: "GitHub", kind: "github", url: current.githubUrl }, "Professional Profile");
    if (current.linkedinUrl) add({ label: "LinkedIn", kind: "linkedin", url: current.linkedinUrl }, "Professional Profile");
    if (current.websiteUrl) add({ label: "Website", kind: "webpage", url: current.websiteUrl }, "Professional Profile");

    (siteSections || []).forEach((section) => {
      (section.links || []).forEach((link) => add(link, section.title || "Portfolio section"));
    });

    return records.filter((record, index, array) => (
      record.url && array.findIndex((item) => item.url === record.url && item.label === record.label) === index
    ));
  }
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `fileNameFromUrl`, `assistantUrlKind`, `assistantAbsoluteUrl`, `normalizeProfile`. Those calls show that `assistantPublicProfileLinks` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantPublicProfileLinks`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The easiest way to follow the body of `assistantPublicProfileLinks` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

records: `records` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods `filter`, `push`; assigned or updated later in the function.

add: `add` stores a function or callback value for temporary calculation. It is initialized from `(item = {}, context = "") => {}`. Within the function it is assigned or updated later in the function.

rawUrl: `rawUrl` stores a computed value for network or routing value. It is initialized from `item.url || item.href || item.path || ""`. Within the function it is passed into `assistantAbsoluteUrl`, `fileNameFromUrl`, `push`; assigned or updated later in the function.

label: `label` stores a computed value for temporary calculation. It is initialized from `item.label || item.title || fileNameFromUrl(rawUrl)`. Within the function it is passed into `add`; assigned or updated later in the function.

current: `current` stores a computed value for temporary calculation. It is initialized from `normalizeProfile(profile || {})`. Within the function it accesses properties/methods `githubUrl`, `linkedinUrl`, `resumeUrl`, `websiteUrl`; passed into `add`; assigned or updated later in the function.

function assistantProjectEvidence(project = {})

Role: Begin with the role of `assistantProjectEvidence`. `assistantProjectEvidence` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantProjectEvidence(project = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `project` identifies or carries the project being built, rendered, compiled, searched, or published. It links this function back to one portfolio project instead of letting work spill across unrelated projects. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantProjectEvidence

```
function assistantProjectEvidence(project = {}) {
  const downloads = uniqueDownloads(collectDownloadsFromValue(project, "Project asset", []));
  const downloadRecords = downloads.slice(0, 26).map((download) => {
    const rawUrl = download.url || "";
    return {
      description: download.description || "",
      kind: assistantUrlKind(rawUrl),
      label: download.title || download.label || fileNameFromUrl(rawUrl),
      type: download.type || "",
      url: assistantAbsoluteUrl(rawUrl)
    };
  });
  const textLinkRecords = assistantUrlsFromTextValue([project.summary, project.links, project.portfolioView],
project.portfolioView?.title || project.title);
  return [...downloadRecords, ...textLinkRecords].filter((record, index, array) => (
    record.url && array.findIndex((item) => item.url === record.url) === index
  ));
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `uniqueDownloads`, `collectDownloadsFromValue`, `assistantUrlKind`, `fileNameFromUrl`, `assistantAbsoluteUrl`, `assistantUrlsFromTextValue`. Those calls show that `assistantProjectEvidence` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `assistantProjectEvidence`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local state inside `assistantProjectEvidence` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

downloads: `downloads` stores a computed value for temporary calculation. It is initialized from `uniqueDownloads(collectDownloadsFromValue(project, "Project asset", []))`. Within the function it is accesses properties/methods `slice`; assigned or updated later in the function.

downloadRecords: `downloadRecords` stores a function or callback value for lookup collection. It is initialized from `downloads.slice(0, 26).map((download) => {`. Within the function it is assigned or updated later in the function.

rawUrl: `rawUrl` stores a computed value for network or routing value. It is initialized from `download.url || ""`. Within the function it is passed into `assistantAbsoluteUrl`, `assistantUrlKind`, `fileNameFromUrl`; assigned or updated later in the function.

textLinkRecords: `textLinkRecords` stores a computed value for temporary calculation. It is initialized from `assistantUrlsFromTextValue([project.summary, project.links, project.portfolioView], project.portfolioView?.title || project.title)`. Within the function it is assigned or updated later in the function.

function assistantQuestionIsCasual(question = "")

Role: The best entry point for `assistantQuestionIsCasual` is its responsibility in the surrounding workflow. `assistantQuestionIsCasual` is one piece of the Ask My Portfolio conversation system. It helps decide what the

user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantQuestionIsCasual(question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `assistantQuestionIsCasual`

```
function assistantQuestionIsCasual(question = "") {  
  const clean = normalize(question);  
  return /^(hi|hello|hey|good\s+(morning|afternoon|evening)|yo|sup|thanks|thank\s+you|ok|okay)\b/.test(clean)  
    || /^(how\s+are\s+you|what\s+can\s+you\s+help.*|what\s+do\s+you\s+do|who\s+are\s+you|help|help\s+me|  
can\s+you\s+help|nice\s+to\s+meet\s+you)\??$/i.test(clean)  
    || /^(what'\s+what\s+is|do\s+you\s+know)\s+my\s+name\??$/i.test(clean)  
    || /^who\s+am\s+i\??$/i.test(clean);  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `normalize`. Those calls show that `assistantQuestionIsCasual` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantQuestionIsCasual`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The easiest way to follow the body of `assistantQuestionIsCasual` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

`function assistantQuestionHasPortfolioIntent(question = "")`

Role: Read `assistantQuestionHasPortfolioIntent` first as a named idea, not as syntax.

`assistantQuestionHasPortfolioIntent` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantQuestionHasPortfolioIntent(question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string

data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantQuestionHasPortfolioIntent

```
function assistantQuestionHasPortfolioIntent(question = "") {  
  const clean = normalize(question);  
  return assistantProjectTitleMatches(question)  
    || /\b(maurice|otieno|portfolio|resume|github|linkedin|contact|email|phone|project|projects|repo|repository|  
repositories|file|files|document|documents|artifact|artifacts|link|links|download|open|show|where|built|build|  
created|designed|implemented)\b/.test(clean)  
    || /\b(your|his|maurice's)\s+(project|projects|resume|github|linkedin|portfolio|work|email|phone|contact|repo|  
repository|files?|documents?|links?)\b/.test(clean);  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. It works with `normalize`, `assistantProjectTitleMatches`. Those calls show that `assistantQuestionHasPortfolioIntent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `assistantQuestionHasPortfolioIntent`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local objects and variables inside `assistantQuestionHasPortfolioIntent` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

function assistantConversationSuggestsPortfolioContext(history = [])

Role: To understand `assistantConversationSuggestsPortfolioContext`, start with the problem it owns. `assistantConversationSuggestsPortfolioContext` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantConversationSuggestsPortfolioContext(history = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `history` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantConversationSuggestsPortfolioContext

```
function assistantConversationSuggestsPortfolioContext(history = []) {  
  return history  
    .slice(-4)  
    .some((item) => /\b(maurice|otieno|portfolio|project|resume|github|linkedin|electronics|engineering)  
\b/.test(normalize(item?.content)));  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalize`. Those calls show that `assistantConversationSuggestsPortfolioContext` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `assistantConversationSuggestsPortfolioContext`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: Inside `assistantConversationSuggestsPortfolioContext`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function assistantQuestionAllowsPublicLookup(question = "", intent = assistantQuestionIntent(question))

Role: To understand `assistantQuestionAllowsPublicLookup`, start with the problem it owns. `assistantQuestionAllowsPublicLookup` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantQuestionAllowsPublicLookup(question = "", intent =
assistantQuestionIntent(question)) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `intent` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for assistantQuestionAllowsPublicLookup

```
function assistantQuestionAllowsPublicLookup(question = "", intent = assistantQuestionIntent(question)) {
  const clean = normalize(question);
  return intent === "general_engineering"
    || intent === "general_knowledge"
    || /\b(github|linkedin|repo|repository|repositories|public\s+link|website|web\s+page|profile|resume|
uploaded\s+file|document|source\s+code)\b/.test(clean);
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalize`. Those calls show that `assistantQuestionAllowsPublicLookup` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `assistantQuestionAllowsPublicLookup`, the assistant would either answer with less context, display unrelated

links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: After the main flow, the working values inside `assistantQuestionAllowsPublicLookup` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

function `assistantQuestionLooksConceptual(question = "")`

Role: Begin with the role of `assistantQuestionLooksConceptual`. `assistantQuestionLooksConceptual` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantQuestionLooksConceptual(question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `assistantQuestionLooksConceptual`

```
function assistantQuestionLooksConceptual(question = "") {  
  const clean = normalize(question);  
  if (!assistantQuestionIsEngineeringRelated(question)) return false;  
  return /^(what|what\s+is|what\s+are|what's|define|explain|describe|how|why|compare|differentiate)\b/.test(clean)  
    || /\b(definition|meaning|basics|overview|introduction|difference\s+between|how\s+does|how\s+do|why\s+does|  
why\s+do)\b/.test(clean);  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalize`, `assistantQuestionIsEngineeringRelated`. Those calls show that `assistantQuestionLooksConceptual` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `assistantQuestionLooksConceptual`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local state inside `assistantQuestionLooksConceptual` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

function assistantQuestionIntent(question = "", history = assistantChatHistory)

Role: Begin with the role of assistantQuestionIntent. assistantQuestionIntent is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantQuestionIntent(question = "", history = assistantChatHistory) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. history is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantQuestionIntent

```
function assistantQuestionIntent(question = "", history = assistantChatHistory) {  
  if (assistantQuestionIsCasual(question)) return "general_conversation";  
  if (/^(what'?s|what\s+is|do\s+you\s+know)\s+my\s+name\??$/i.test(normalize(question)) || /who\s+am\s+i\??$/i.test(normalize(question))) {  
    return "general_conversation";  
  }  
  const hasPortfolioIntent = assistantQuestionHasPortfolioIntent(question);  
  if (!hasPortfolioIntent && assistantConversationSuggestsPortfolioContext(history) && /\b(name|you|your|he|his|him|that|this|it)\b/i.test(normalize(question))) {  
    return "general_conversation";  
  }  
  if (!hasPortfolioIntent && assistantQuestionIsEngineeringRelated(question)) return "general_engineering";  
  if (!hasPortfolioIntent && assistantQuestionLooksConceptual(question)) return "general_engineering";  
  if (hasPortfolioIntent) return "portfolio_specific";  
  return "general_knowledge";  
}
```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with assistantQuestionIsCasual, normalize, assistantQuestionHasPortfolioIntent, assistantConversationSuggestsPortfolioContext, assistantQuestionIsEngineeringRelated, assistantQuestionLooksConceptual. Those calls show that assistantQuestionIntent is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without assistantQuestionIntent, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local state inside assistantQuestionIntent explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

hasPortfolioIntent: hasPortfolioIntent stores a computed value for temporary calculation. It is initialized from assistantQuestionHasPortfolioIntent(question). Within the function it is assigned or updated later in the function.

function assistantQuestionTokens(question = "")

Role: Read assistantQuestionTokens first as a named idea, not as syntax. assistantQuestionTokens is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantQuestionTokens(question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantQuestionTokens

```
function assistantQuestionTokens(question = "") {  
  const stopWords = new Set([  
    "about", "again", "also", "and", "are", "can", "could", "does", "for", "from", "give", "have", "his",  
    "into", "link", "links", "maurice", "me", "my", "open", "otieno", "please", "portfolio", "project",  
    "projects", "show", "tell", "that", "the", "their", "this", "what", "where", "which", "who", "with",  
    "work", "your"  
  ]);  
  return normalize(question)  
    .split(/^[^a-z0-9+#.]+/i)  
    .map((token) => token.trim())  
    .filter((token) => token.length > 2 && !stopWords.has(token));  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. It works with normalize. Those calls show that assistantQuestionTokens is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without assistantQuestionTokens, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local objects and variables inside assistantQuestionTokens show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

stopWords: stopWords stores a constructed object for lookup collection. It is initialized from new Set([. Within the function it is accesses properties/methods has; assigned or updated later in the function.

function assistantSpecificSourceTokens(question = "")

Role: To understand assistantSpecificSourceTokens, start with the problem it owns. assistantSpecificSourceTokens is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantSpecificSourceTokens(question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `assistantSpecificSourceTokens`

```
function assistantSpecificSourceTokens(question = "") {  
  const genericTokens = new Set([  
    "built", "build", "created", "designed", "did", "done", "engineering", "file", "files", "hardware",  
    "item", "items", "section", "sections", "system", "systems", "thing", "things", "tool", "tools",  
    "use", "used", "using", "website"  
  ]);  
  return assistantQuestionTokens(question).filter((token) => !genericTokens.has(token));  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `assistantQuestionTokens`. Those calls show that `assistantSpecificSourceTokens` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `assistantSpecificSourceTokens`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: After the main flow, the working values inside `assistantSpecificSourceTokens` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

genericTokens: `genericTokens` stores a constructed object for lookup collection. It is initialized from `new Set([`. Within the function it is accesses properties/methods `has`; assigned or updated later in the function.

`function assistantPromptTargetsProjectLanding(question = "")`

Role: Read `assistantPromptTargetsProjectLanding` first as a named idea, not as syntax. `assistantPromptTargetsProjectLanding` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantPromptTargetsProjectLanding(question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `assistantPromptTargetsProjectLanding`

```
function assistantPromptTargetsProjectLanding(question = "") {  
  const clean = normalize(question);
```

```

return /\b(open|show|view|go\s+to|take\s+me\s+to)\b/.test(clean)
  && /\b(project|overview|page)\b/.test(clean)
  && !/\b(code|source|repo|repository|github|file|files|document|documents|test|tests|result|results|pcb|
schematic|simulation|tool|tools|resume|linkedin)\b/.test(clean);
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. It works with `normalize`. Those calls show that `assistantPromptTargetsProjectLanding` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `assistantPromptTargetsProjectLanding`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local objects and variables inside `assistantPromptTargetsProjectLanding` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

function assistantEntryRelationScore(result = {}, question = "")

Role: Read `assistantEntryRelationScore` first as a named idea, not as syntax. `assistantEntryRelationScore` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantEntryRelationScore(result = {}, question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `result` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantEntryRelationScore

```

function assistantEntryRelationScore(result = {}, question = "") {
  const tokens = assistantSpecificSourceTokens(question);
  const haystack = normalize([result.title, result.context, result.type, result.text,
result.url].filter(Boolean).join(" "));
  const tokenHits = tokens.filter((token) => haystack.includes(token)).length;
  let relation = tokenHits * 30;

  if (tokens.length && tokenHits === tokens.length) relation += 25;
  if (Number(result.score || 0) >= 120) relation += 15;
  if (Number(result.score || 0) >= 80) relation += 8;
  if (result.kind === "project") relation += 12;
  if (result.kind === "file" && /\b(file|files|download|document|documents|code|source|github|repo|repository|test|
tests|result|results|pcb|schematic|simulation)\b/.test(normalize(question))) {
    relation += 16;
  }
}

```

```
    return relation;
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. It works with `assistantSpecificSourceTokens`, `normalize`. Those calls show that `assistantEntryRelationScore` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `assistantEntryRelationScore`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local objects and variables inside `assistantEntryRelationScore` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

tokens: `tokens` stores a computed value for temporary calculation. It is initialized from `assistantSpecificSourceTokens(question)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `length`; assigned or updated later in the function.

haystack: `haystack` stores a resolved filesystem or route value. It is initialized from `normalize([result.title, result.context, result.type, result.text, result.url].filter(Boolean).join(" "))`. Within the function it is accesses properties/methods `includes`; assigned or updated later in the function.

tokenHits: `tokenHits` stores a function or callback value for temporary calculation. It is initialized from `tokens.filter((token) => haystack.includes(token)).length`. Within the function it is assigned or updated later in the function.

relation: `relation` stores a computed value for temporary calculation. It is initialized from `tokenHits * 30`. Within the function it is returned to the caller; assigned or updated later in the function.

function assistantNamedProjectTopicTokens(question = "", namedIds = assistantNamedProjectIds(question))

Role: To understand `assistantNamedProjectTopicTokens`, start with the problem it owns. `assistantNamedProjectTopicTokens` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantNamedProjectTopicTokens(question = "", namedIds =
assistantNamedProjectIds(question)) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `namedIds` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantNamedProjectTopicTokens

```
function assistantNamedProjectTopicTokens(question = "", namedIds = assistantNamedProjectIds(question)) {
  const projectVocabulary = new Set();
  namedIds
    .map((id) => projects.find((project) => project.id === id))
    .filter(Boolean)
    .forEach((project) => {
      const title = project.portfolioView?.title || project.title || project.id;
      normalize(title).split(/\s+/).filter((word) => word.length > 2).forEach((word) =>
        projectVocabulary.add(word));
      assistantProjectTitleAcronyms(title).forEach((acronym) => projectVocabulary.add(acronym));
      projectVocabulary.add(normalize(project.id));
    });
  return assistantSpecificSourceTokens(question).filter((token) => !projectVocabulary.has(token));
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalize`, `assistantProjectTitleAcronyms`, `assistantSpecificSourceTokens`. Those calls show that `assistantNamedProjectTopicTokens` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `assistantNamedProjectTopicTokens`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: After the main flow, the working values inside `assistantNamedProjectTopicTokens` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

projectVocabulary: `projectVocabulary` stores a constructed object for portfolio data state. It is initialized from `new Set()`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is mutated as a collection or DOM container; accesses properties/methods `add`, `has`; assigned or updated later in the function.

title: `title` stores a computed value for temporary calculation. It is initialized from `project.portfolioView?.title || project.title || project.id`. Within the function it is passed into `assistantProjectTitleAcronyms`, `normalize`; assigned or updated later in the function.

function assistantSourcesForDisplay(question = "", results = [], intent = assistantQuestionIntent(question))

Role: The best entry point for `assistantSourcesForDisplay` is its responsibility in the surrounding workflow. `assistantSourcesForDisplay` chooses which portfolio links appear under an assistant answer. The user complained earlier that related links were generic; this function is the fix-point for that behavior. It scores source relation to the prompt, respects named projects and topic tokens, and avoids showing source buttons that do not belong to the answer context. The function matters because sources are a promise. A displayed link should help verify or explore the answer. If this function is too loose, the assistant looks random even when the text answer is good.

Function entry point:

```
function assistantSourcesForDisplay(question = "", results = [], intent =
  assistantQuestionIntent(question)) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `results` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `intent` is user language: a search query, AI question, or classified assistant

intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantSourcesForDisplay

```
function assistantSourcesForDisplay(question = "", results = [], intent = assistantQuestionIntent(question)) {
  if (intent !== "portfolio_specific") return [];
  const tokens = assistantSpecificSourceTokens(question);
  const namedProjectIdList = assistantNamedProjectIds(question);
  const namedProjectIds = new Set(namedProjectIdList);
  const namedProjectTopicTokens = assistantNamedProjectTopicTokens(question, namedProjectIdList);
  const asksForLinks = /\b(open|show|where|link|links|github|linkedin|resume|download|file|files|repo|repository|
source\s+code)\b/.test(normalize(question));
  const asksForProjectLanding = assistantPromptTargetsProjectLanding(question);
  const pageLinkIntent = /\b(resume|github|linkedin|contact|email|phone)\b/.test(normalize(question));
  const asksForSpecificArtifact = /\b(code|source|repo|repository|github|file|files|document|documents|test|tests|
result|results|pcb|schematic|simulation|tool|tools)\b/.test(normalize(question));

  if (!namedProjectIds.size && !pageLinkIntent && tokens.length === 0) return [];

  const filtered = results
    .map((result) => ({ ...result, relation: assistantEntryRelationScore(result, question) })))
    .filter((result) => {
      if (!result) return false;
      if (namedProjectIds.size && !namedProjectIds.has(result.projectId)) return false;
      if (asksForProjectLanding) return result.kind === "project";
      const haystack = normalize([result.title, result.context, result.type, result.text,
result.url].filter(Boolean).join(" "));
      const tokenHits = tokens.filter((token) => haystack.includes(token)).length;
      const topicHits = namedProjectTopicTokens.filter((token) => haystack.includes(token)).length;
      if (namedProjectIds.size && namedProjectTopicTokens.length && result.kind !== "project" && topicHits === 0)
return false;
      if (namedProjectIds.size && namedProjectTopicTokens.length && result.kind === "project" &&
asksForSpecificArtifact) return false;
      if (namedProjectIds.size && result.kind === "project") return true;
      if (pageLinkIntent && tokenHits >= 1) return true;
      if (!tokens.length) return false;
      if (tokenHits >= Math.min(2, tokens.length)) return true;
      if (asksForLinks && tokenHits >= 1 && result.relation >= 35) return true;
      return result.relation >= 55;
    });

  return filtered
    .sort((a, b) => b.relation - a.relation || Number(b.score || 0) - Number(a.score || 0))
    .slice(0, asksForLinks ? 4 : 2);
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `assistantSpecificSourceTokens`, `assistantNamedProjectIds`, `assistantNamedProjectTopicTokens`, `normalize`, `assistantPromptTargetsProjectLanding`, `assistantEntryRelationScore`. Those calls show that `assistantSourcesForDisplay` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantSourcesForDisplay`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The easiest way to follow the body of `assistantSourcesForDisplay` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

tokens: tokens stores a computed value for temporary calculation. It is initialized from `assistantSpecificSourceTokens(question)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `length`; assigned or updated later in the function.

namedProjectIdList: `namedProjectIdList` stores a computed value for portfolio data state. It is initialized from `assistantNamedProjectIds(question)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is passed into `Set`, `assistantNamedProjectTopicTokens`; assigned or updated later in the function.

namedProjectIds: `namedProjectIds` stores a constructed object for portfolio data state. It is initialized from `new Set(namedProjectIdList)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods `has`, `size`; assigned or updated later in the function.

namedProjectTopicTokens: `namedProjectTopicTokens` stores a computed value for portfolio data state. It is initialized from `assistantNamedProjectTopicTokens(question, namedProjectIdList)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is iterated or transformed as a collection; accesses properties/methods `filter`, `length`; assigned or updated later in the function.

asksForLinks: `asksForLinks` stores a computed value for temporary calculation. It is initialized from `\b(open|show|where|link|links|github|linkedin|resume|download|file|files|repo|repository|source\s+code)\b/.test(normalize(question))`. Within the function it is passed into `slice`; assigned or updated later in the function.

asksForProjectLanding: `asksForProjectLanding` stores a computed value for portfolio data state. It is initialized from `assistantPromptTargetsProjectLanding(question)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

pageLinkIntent: `pageLinkIntent` stores a computed value for temporary calculation. It is initialized from `\b(resume|github|linkedin|contact|email|phone)\b/.test(normalize(question))`. Within the function it is assigned or updated later in the function.

asksForSpecificArtifact: `asksForSpecificArtifact` stores a computed value for temporary calculation. It is initialized from `\b(code|source|repo|repository|github|file|files|document|documents|test|tests|result|results|pcb|schematic|simulation|tool|tools)\b/.test(normalize(question))`. Within the function it is assigned or updated later in the function.

filtered: `filtered` stores a computed value for temporary calculation. It is initialized from `results`. Within the function it is returned to the caller; assigned or updated later in the function.

haystack: `haystack` stores a resolved filesystem or route value. It is initialized from `normalize([result.title, result.context, result.type, result.text, result.url].filter(Boolean).join(" ")). Within the function it is accesses properties/methods includes; assigned or updated later in the function.`

tokenHits: `tokenHits` stores a function or callback value for temporary calculation. It is initialized from `tokens.filter((token) => haystack.includes(token)).length`. Within the function it is assigned or updated later in the function.

topicHits: `topicHits` stores a function or callback value for temporary calculation. It is initialized from `namedProjectTopicTokens.filter((token) => haystack.includes(token)).length`. Within the function it is assigned or updated later in the function.

function assistantContextForQuestion(question = "", intent = assistantQuestionIntent(question), knownResults = null)

Role: Read `assistantContextForQuestion` first as a named idea, not as syntax. `assistantContextForQuestion` builds the packet of portfolio evidence sent to the AI backend. It uses the question intent, search results, named project matches, profile links, project summaries, and available evidence to decide what the model should see. It

is selective by design so a generic greeting does not receive a pile of unrelated project context. Without this function, the assistant would either know too little about the portfolio or always overload the model with the same generic links. This is the function that makes the AI choose context based on the conversation.

Function entry point:

```
function assistantContextForQuestion(question = "", intent = assistantQuestionIntent(question),
knownResults = null) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `intent` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `knownResults` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `assistantContextForQuestion`

```
function assistantContextForQuestion(question = "", intent = assistantQuestionIntent(question), knownResults = null)
{
  const directResults = ["general_engineering", "general_knowledge", "general_conversation"].includes(intent)
    ? []
    : (knownResults || searchResultsFor(question)).slice(0, 8);
  const projectIds = [...new Set([
    ...directResults.map((item) => item.projectId).filter(Boolean),
    ...assistantNamedProjectIds(question)
  ])];
  const matchedProjects = projectIds
    .map((id) => projects.find((project) => project.id === id))
    .filter(Boolean)
    .slice(0, 5);
  const includePortfolioLinkContext = intent === "portfolio_specific";
  const wantsPublicCode = /\b(github|repo|repository|repositories|source\s+code|code|snippet|firmware|verilog|
systemverilog|python|javascript)\b/.test(normalize(question));
  const publicProfileFetches = includePortfolioLinkContext
    ? assistantPublicProfileLinks()
      .map((item) => ({ ...item, question }))
      .sort((a, b) => {
        if (!wantsPublicCode) return 0;
        const aGithub = /github/i.test(`${a.kind} | ${a.url} | `) ? 1 : 0;
        const bGithub = /github/i.test(`${b.kind} | ${b.url} | `) ? 1 : 0;
        return bGithub - aGithub;
      })
    : [];
  const resultFetches = directResults.map((result) => ({
    context: result.context,
    kind: assistantUrlKind(result.url),
    title: result.title,
    type: result.type,
    url: assistantAbsoluteUrl(result.url || ""),
    question
  }));
  const projectFetches = matchedProjects.flatMap((project) => assistantProjectEvidence(project)
    .filter((item) => ["text", "webpage", "github", "linkedin", "resume_text"].includes(item.kind))
    .slice(0, 6)
    .map((item) => ({ ...item, question })));
  const sourceFetches = [
    ...(wantsPublicCode ? publicProfileFetches : []),
    ...resultFetches,
    ...(wantsPublicCode ? [] : publicProfileFetches),
    ...projectFetches
  ].filter((item) => item.url);
  const uniqueSourceFetches = sourceFetches.filter((item, index, array) => (
```

```

    array.findIndex((candidate) => candidate.url === item.url) === index
  )).slice(0, 14);
  return {
    categories: categories.map((category) => ({ id: category.id, label: category.label })),
    intent,
    profile: normalizeProfile(profile || {}),
    knowledgeManifest: {
      publicProfiles: includePortfolioLinkContext ? assistantPublicProfileLinks() : [],
      publicSourcePolicy: "Only public profile and project links shown in the portfolio are used. GitHub repository links can be expanded into repository metadata, README text, and selected public source files when the visitor asks for code.",
      matchedProjectEvidence: matchedProjects.map((project) => ({
        evidence: assistantProjectEvidence(project),
        projectId: project.id,
        title: project.portfolioView?.title || project.title
      })),
      note: "Image records include filenames, captions, descriptions, and neighboring portfolio text. Use a vision-capable backend before claiming visual details not present in captions or text."
    },
    portfolioContextPolicy: intent !== "portfolio_specific"
      ? "Answer from general knowledge first. Do not lead with the portfolio owner's project context unless the visitor explicitly asks to connect the concept to the portfolio."
      : "Answer from the configured portfolio context first. Do not invent project details outside the supplied context.",
    projects: matchedProjects.map(assistantProjectSummary),
    question,
    results: directResults.map((result) => ({
      context: result.context,
      kind: result.kind,
      text: searchSnippet(result.text, question),
      title: result.title,
      type: result.type,
      url: result.url
    })),
    sourceFetches: uniqueSourceFetches,
    siteSections: siteSections.filter(siteSectionRenderable).map((section) => ({ id: section.id, title: section.title })),
  };
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a structured object. That object is the contract the next layer uses instead of trying to interpret raw strings or side effects.

Collaborators: The function also has to be read through the helpers it calls. It works with `searchResultsFor`, `assistantNamedProjectIds`, `normalize`, `assistantPublicProfileLinks`, `assistantUrlKind`, `assistantAbsoluteUrl`, `assistantProjectEvidence`, `normalizeProfile`. Those calls show that `assistantContextForQuestion` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `assistantContextForQuestion`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local objects and variables inside `assistantContextForQuestion` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

directResults: `directResults` stores a list for filesystem location. It is initialized from `["general_engineering", "general_knowledge", "general_conversation"].includes(intent)`. It controls a boundary or destination for file work, so an incorrect value can redirect uploads, previews, compiler artifacts, or published assets. Within the function it is iterated or transformed as a collection; accesses properties/methods map; passed into `Set`; assigned or updated later in the function.

projectIds: `projectIds` stores a list for portfolio data state. It is initialized from `[...new Set(`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

matchedProjects: matchedProjects stores a computed value for portfolio data state. It is initialized from projectIds. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is iterated or transformed as a collection; accesses properties/methods flatMap, map; assigned or updated later in the function.

includePortfolioLinkContext: includePortfolioLinkContext stores a computed value for temporary calculation. It is initialized from intent === "portfolio_specific". Within the function it is assigned or updated later in the function.

wantsPublicCode: wantsPublicCode stores a computed value for temporary calculation. It is initialized from /b(github|repo|repository|repositories|source\s+code|code|snippet|firmware|verilog|systemverilog|python|javascript)b/.test(normalize(question)). Within the function it is assigned or updated later in the function.

publicProfileFetches: publicProfileFetches stores a computed value for portfolio data state. It is initialized from includePortfolioLinkContext. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

aGithub: aGithub stores a computed value for Git publishing and authorization state. It is initialized from /github/i.test(`\${a.kind || ""} \${a.url || ""}`) ? 1 : 0. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

bGithub: bGithub stores a computed value for Git publishing and authorization state. It is initialized from /github/i.test(`\${b.kind || ""} \${b.url || ""}`) ? 1 : 0. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is returned to the caller; assigned or updated later in the function.

resultFetches: resultFetches stores a function or callback value for lookup collection. It is initialized from directResults.map((result) => ({). Within the function it is assigned or updated later in the function.

projectFetches: projectFetches stores a function or callback value for portfolio data state. It is initialized from matchedProjects.flatMap((project) => assistantProjectEvidence(project). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

sourceFetches: sourceFetches stores a list for temporary calculation. It is initialized from [. Within the function it is iterated or transformed as a collection; accesses properties/methods filter; assigned or updated later in the function.

uniqueSourceFetches: uniqueSourceFetches stores a function or callback value for temporary calculation. It is initialized from sourceFetches.filter((item, index, array) => (. Within the function it is assigned or updated later in the function.

function assistantRemember(role, content = "")

Role: To understand assistantRemember, start with the problem it owns. assistantRemember is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantRemember(role, content = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. role is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. content is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string

data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantRemember

```
function assistantRemember(role, content = "") {  
  const cleanRole = role === "assistant" ? "assistant" : "user";  
  const cleanContent = String(content || "").trim();  
  if (!cleanContent) return;  
  assistantChatHistory.push({  
    role: cleanRole,  
    content: cleanContent.slice(0, 1400)  
  });  
  assistantChatHistory = assistantChatHistory.slice(-10);  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means assistantRemember performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without assistantRemember, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: After the main flow, the working values inside assistantRemember reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

cleanRole: cleanRole stores a computed value for temporary calculation. It is initialized from `role === "assistant" ? "assistant" : "user"`. Within the function it is passed into push; assigned or updated later in the function.

cleanContent: cleanContent stores a computed value for temporary calculation. It is initialized from `String(content || "").trim()`. Within the function it is accesses properties/methods slice; passed into push; assigned or updated later in the function.

function assistantConversationForRequest(currentQuestion = "", priorConversation = assistantChatHistory)

Role: To understand assistantConversationForRequest, start with the problem it owns. assistantConversationForRequest is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantConversationForRequest(currentQuestion = "", priorConversation =  
assistantChatHistory) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. currentQuestion is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. priorConversation is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantConversationForRequest

```
function assistantConversationForRequest(currentQuestion = "", priorConversation = assistantChatHistory) {  
  const history = priorConversation.slice(-8).map((item) => ({  
    role: item.role === "assistant" ? "assistant" : "user",  
    content: String(item.content || "").slice(0, 1200)  
  }));  
  if (currentQuestion) {  
    history.push({ role: "user", content: String(currentQuestion).slice(0, 1200) });  
  }  
  return history;  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser route/history. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means assistantConversationForRequest performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without assistantConversationForRequest, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: After the main flow, the working values inside assistantConversationForRequest reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

history: history stores a function or callback value for runtime state or cache. It is initialized from priorConversation.slice(-8).map((item) => ({. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods push; assigned or updated later in the function.

function assistantFallbackResults(question = "", intent = assistantQuestionIntent(question))

Role: Begin with the role of assistantFallbackResults. assistantFallbackResults is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantFallbackResults(question = "", intent = assistantQuestionIntent(question))  
{ ... }
```

Inputs: The function-specific inputs are worth reading before the body. question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. intent is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantFallbackResults

```
function assistantFallbackResults(question = "", intent = assistantQuestionIntent(question)) {  
  if (intent !== "portfolio_specific") return [];  
  const clean = normalize(question).trim();  
  if (!clean) return [];  
  const namedProjectIds = assistantNamedProjectIds(question);  
  let results = searchResultsFor(clean);  
  if (namedProjectIds.length && results.length) {  
    const focusedResults = results.filter((result) => namedProjectIds.includes(result.projectId));  
    if (focusedResults.length) return focusedResults.slice(0, 6);  
  }  
  if (results.length) return results.slice(0, 6);  
}
```

```

    if (namedProjectIds.length) {
      return namedProjectIds
        .map((id) => projects.find((project) => project.id === id))
        .filter(Boolean)
        .map((project) => ({
          context: slugLabel(project.category),
          kind: "project",
          projectId: project.id,
          text: flattenSearchText([project.summary, project.focus, project.tools, project.languages]),
          title: project.portfolioView?.title || project.title,
          type: "Project"
        })))
        .slice(0, 6);
    }

    const category = categories.find((item) => clean.includes(normalize(item.label)) ||
clean.includes(normalize(item.id)));
    if (category) {
      return projects
        .filter((project) => project.category === category.id)
        .map((project) => ({
          context: category.label,
          kind: "project",
          projectId: project.id,
          text: flattenSearchText([project.summary, project.focus, project.tools, project.languages]),
          title: project.portfolioView?.title || project.title,
          type: "Project"
        })))
        .slice(0, 6);
    }

    if (/resume|professional|contact|email|phone|github|linkedin|profile/.test(clean)) {
      return searchResultsFor("resume contact github professional profile").slice(0, 6);
    }

    if (/project|work|portfolio|hardware|design/.test(clean)) {
      return projects.slice(0, 6).map((project) => ({
        context: slugLabel(project.category),
        kind: "project",
        projectId: project.id,
        text: project.summary || "",
        title: project.portfolioView?.title || project.title,
        type: "Project"
      })));
    }

    return [];
  }
}

```

Implementation: The body is where the contract above becomes behavior. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `normalize`, `assistantNamedProjectIds`, `searchResultsFor`, `slugLabel`, `flattenSearchText`. Those calls show that `assistantFallbackResults` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `assistantFallbackResults`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local state inside `assistantFallbackResults` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question).trim()`. Within the function it is accessed by `includes`; passed into `searchResultsFor`; assigned or updated later in the function.

namedProjectIds: namedProjectIds stores a computed value for portfolio data state. It is initialized from assistantNamedProjectIds(question). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is returned to the caller; accesses properties/methods includes, length; assigned or updated later in the function.

results: results stores a computed value for temporary calculation. It is initialized from searchResultsFor(clean). Within the function it is returned to the caller; iterated or transformed as a collection; accesses properties/methods filter, length, slice; assigned or updated later in the function.

focusedResults: focusedResults stores a function or callback value for temporary calculation. It is initialized from results.filter((result) => namedProjectIds.includes(result.projectId)). Within the function it is returned to the caller; accesses properties/methods length, slice; assigned or updated later in the function.

category: category stores a function or callback value for portfolio data state. It is initialized from categories.find((item) => clean.includes(normalize(item.label)) || clean.includes(normalize(item.id))). It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods id, label; passed into slugLabel; assigned or updated later in the function.

function assistantQuestionIsEngineeringRelated(question = "")

Role: Read assistantQuestionIsEngineeringRelated first as a named idea, not as syntax.

assistantQuestionIsEngineeringRelated is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantQuestionIsEngineeringRelated(question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. question is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantQuestionIsEngineeringRelated

```
function assistantQuestionIsEngineeringRelated(question = "") {  
  const clean = normalize(question);  
  return /\b(op\s*amp|amplifier|analog|mixed\s*signal|adc|dac|filter|vco|oscillator|pwm|charger|rectifier|regulator|  
buck|boost|ldo|mosfet|bjt|transistor|diode|pcb|schematic|layout|ground|noise|frequency|gain|phase|ltspice|kicad|  
vivado|verilog|vhdl|fpga|asic|stm32|mcu|microcontroller|embedded|firmware|rtos|i2c|spi|uart|sensor|control|signal|  
circuit|electronics|hardware|power)\b/.test(clean);  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: The function also has to be read through the helpers it calls. It works with normalize. Those calls show that assistantQuestionIsEngineeringRelated is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without assistantQuestionIsEngineeringRelated, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local objects and variables inside `assistantQuestionIsEngineeringRelated` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

function `assistantGeneralEngineeringAnswer(question = "")`

Role: The best entry point for `assistantGeneralEngineeringAnswer` is its responsibility in the surrounding workflow. `assistantGeneralEngineeringAnswer` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantGeneralEngineeringAnswer(question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `assistantGeneralEngineeringAnswer`

```
function assistantGeneralEngineeringAnswer(question = "") {
  const clean = normalize(question);
  if (!assistantQuestionIsEngineeringRelated(question)) return "";

  if (/\\bembedded\\s+systems?\\b/.test(clean)) {
    return [
      "Embedded systems is a field of engineering focused on computers that are built into larger products, machines, instruments, or control systems.",
      "",
      "Unlike a laptop or desktop computer, an embedded system is usually designed for a specific job. It combines hardware, firmware, sensors, actuators, communication interfaces, and timing constraints so the product can make measurements, control outputs, or respond to events reliably.",
      "",
      "Common examples include:",
      "- A microcontroller controlling a motor drive.",
      "- A sensor node collecting temperature, pressure, or motion data.",
      "- A medical device running real-time measurement firmware.",
      "- An automotive controller reading sensors and driving actuators.",
      "",
      "The important engineering ideas are timing, interrupts, power use, peripheral interfaces, reliability, and how the firmware proves the hardware is behaving correctly."
    ].join("\\n");
  }

  if (/\\b(op\\s*amp|amplifier|fully differential|differential)\\b/.test(clean)) {
    return [
      "A useful way to think about an op amp is as a high-gain error-correcting block. With negative feedback, the circuit around it sets the useful behavior: gain, filtering, buffering, summing, integration, or differential conversion.",
      "",
      "For a fully differential op amp:",
      "- The signal is carried on two opposite-phase outputs instead of one output referenced to ground.",
      "- The differential output improves noise immunity and dynamic range.",
      "- A common-mode feedback loop is normally needed to hold the average output voltage at the desired common-mode level.",
      "",
      "In portfolio terms, this connects most naturally to analog and mixed-signal design, especially filters, VCO control paths, ADC front ends, and precision measurement circuits."
    ].join("\\n");
  }
}
```

```

if (/b(pwm|vco|oscillator|frequency)\b/.test(clean)) {
    return [
        "PWM-driven VCO work is about converting a digital timing signal into an analog control quantity and then into a frequency.",
        "",
        "The usual signal chain is:",
        "- PWM duty cycle encodes the control value.",
        "- A low-pass filter turns the PWM waveform into an average voltage.",
        "- The VCO maps that voltage to oscillation frequency.",
        "- Tests compare duty cycle, control voltage, frequency range, ripple, and stability.",
        "",
        "That is a mixed-signal problem because digital timing, analog filtering, oscillator behavior, and measurement all interact."
    ].join("\n");
}

if (/b(fpga|asic|verilog|vhdl|vivado|digital)\b/.test(clean)) {
    return [
        "FPGA and ASIC work both implement digital hardware, but the engineering tradeoffs are different.",
        "",
        "- FPGA design is reconfigurable, fast to test, and useful for prototyping or deployment when flexibility matters.",
        "- ASIC design targets a fixed silicon implementation, so verification, timing closure, power, area, and design-for-test become much more permanent decisions.",
        "- Verilog or VHDL describes hardware structure and behavior; it is not software running line by line.",
        "",
        "For a recruiter, strong evidence includes block diagrams, RTL, simulation waveforms, timing reports, verification plans, and notes explaining design tradeoffs."
    ].join("\n");
}

if (/b(stm32|mcu|microcontroller|embedded|firmware|rtos|i2c|spi|uart)\b/.test(clean)) {
    return [
        "Embedded work is strongest when the firmware is tied clearly to hardware behavior.",
        "",
        "A good explanation usually covers:",
        "- The MCU role in the system.",
        "- The peripherals used, such as ADC, PWM, timers, I2C, SPI, UART, or GPIO.",
        "- How timing, interrupts, sampling, power, and fault handling were managed.",
        "- What was measured to prove the system worked.",
        "",
        "STM32CubeIDE is useful because it helps configure peripherals, generate startup code, debug firmware, and manage the build flow around STM32 devices."
    ].join("\n");
}

if (/b(pcb|layout|kicad|schematic|ground|noise)\b/.test(clean)) {
    return [
        "PCB design is where the schematic becomes a physical electrical system.",
        "",
        "The important ideas are:",
        "- Current returns matter as much as current paths.",
        "- Grounding, decoupling, trace width, component placement, and loop area affect noise and stability.",
        "- Analog, power, and switching sections should be arranged so noisy currents do not corrupt sensitive measurements.",
        "- A good portfolio PCB section should show schematic intent, layout decisions, board files, bring-up notes, and test results.",
        "",
        "KiCad evidence is strongest when the design files are paired with clear measurements or bring-up photos."
    ].join("\n");
}

if (/b(charger|rectifier|ac\s*to\s*dc|regulator|buck|boost|ldo|power)\b/.test(clean)) {
    return [
        "An AC-to-DC charger or power supply usually has a few major stages.",
        "",
        "- Input protection and filtering handle transients and noise.",
        "- Rectification converts AC into pulsating DC.",
        "- Bulk capacitance smooths the rectified waveform.",
        "- Regulation sets the desired output voltage or current.",
        "- Load testing checks ripple, thermal behavior, regulation, and fault response.",
        "",
        "For portfolio presentation, the best proof is a clear schematic, expected waveforms, measured output behavior, and a short explanation of component choices."
    ].join("\n");
}

```

```

return [
  "I can answer this as a general electronics or hardware-engineering question, even if it is not directly tied to one saved project yet.",
  "",
  "The best way to approach it is:",
  "- Define the signal or energy flow.",
  "- Identify the active devices, passive networks, and control points.",
  "- State what should be measured to prove the design works.",
  "- Connect the explanation back to a project artifact: schematic, code, test data, PCB, or simulation.",
  "",
  "If you ask about a specific circuit, tool, waveform, or failure mode, I can make the explanation more concrete."
].join("\n");
}

```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `normalize`, `assistantQuestionIsEngineeringRelated`. Those calls show that `assistantGeneralEngineeringAnswer` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantGeneralEngineeringAnswer`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The easiest way to follow the body of `assistantGeneralEngineeringAnswer` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

function `assistantLocalAnswer(question = "", results = [], intent = assistantQuestionIntent(question))`

Role: To understand `assistantLocalAnswer`, start with the problem it owns. `assistantLocalAnswer` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```

function assistantLocalAnswer(question = "", results = [], intent =
assistantQuestionIntent(question)) { ... }

```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `results` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `intent` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `assistantLocalAnswer`

```

function assistantLocalAnswer(question = "", results = [], intent = assistantQuestionIntent(question)) {

```

```

const ownerName = normalizeProfile(profile || {}).displayName || "the portfolio owner";
if (intent === "general_conversation") {
  const clean = normalize(question);
  if (/^(hi|hello|hey|yo|sup)\b/.test(clean)) {
    return "Hi, what can I do for you?";
  }
  if (/^(what'?s|what\s+is|do\s+you\s+know)\s+my\s+name\??$/i.test(clean) || /^who\s+am\s+i\??$/i.test(clean)) {
    return `I am an AI agent for ${ownerName}'s portfolio. I know the portfolio owner is ${ownerName}, but I do not know a visitor's personal name unless they tell me.`;
  }
  if (/b(who are you|what are you)\b/.test(clean)) {
    return `I am an AI agent for ${ownerName}'s portfolio. I can help with projects, resume links, public links, files, and related engineering questions.`;
  }
  return [
    `I am here with you. I can help explore ${ownerName}'s portfolio, explain projects, summarize files, open relevant sections, or answer related engineering questions.`,
    "",
    "You can ask things like:",
    "- What is embedded systems?",
    "- Show me a project.",
    "- What tools were used?",
    "- Explain the difference between FPGA and ASIC design."
  ].join("\n");
}

if (intent === "general_engineering") {
  const generalAnswer = assistantGeneralEngineeringAnswer(question);
  if (generalAnswer) return generalAnswer;
  return [
    "I can answer that as a general engineering question, but I need a little more detail to make the answer useful.",
    "",
    "Try asking about a specific circuit, device, tool, protocol, waveform, or failure mode."
  ].join("\n");
}

if (intent === "general_knowledge") {
  const generalAnswer = assistantGeneralEngineeringAnswer(question);
  if (generalAnswer) return generalAnswer;
  return [
    "I can answer that as a general question from the local fallback, but the live model will usually give a deeper answer when it is reachable.",
    "",
    "Short answer:",
    ` - ${question.replace(/\?+$/, "")} is a topic I can discuss by breaking it into definition, purpose, main parts, and practical examples.`,
    "- If the question connects to Maurice's portfolio, I will use the saved project context and links as evidence.",
    "- If it is broader than the portfolio, I will answer it as a general engineering or technology concept first."
  ].join("\n");
}

if (!projects.length) return "The project catalog is still loading. Try again in a moment.";
if (!results.length) {
  return [
    `I could not find a strong portfolio match for "${question}".$`,
    "",
    "Try asking about:",
    "- A project name or category.",
    "- A tool such as LTspice, KiCad, Vivado, STM32CubeIDE, or GitHub.",
    "- A hardware topic such as op amps, PCB testing, VCOs, embedded firmware, FPGA, ASIC, or power circuits."
  ].join("\n");
}

const projectIds = [...new Set(results.map((item) => item.projectId).filter(Boolean))];
const matchedProjects = projectIds
  .map(id => projects.find((project) => project.id === id))
  .filter(Boolean);
const projectName = matchedProjects.map((project) => project.portfolioView?.title || project.title).slice(0, 3);
const fileCount = results.filter((item) => item.kind === "file").length;
const sectionCount = results.filter((item) => item.kind === "section" || item.kind === "subsection").length;
const pageCount = results.filter((item) => item.kind === "page").length;
const parts = [];

```

```

if (projectNames.length) {
  parts.push(`The strongest portfolio match is ${projectNames.join(projectNames.length > 2 ? ", " : " and ")}.`);
} else if (pageCount) {
  parts.push("The strongest matches are in the profile, resume, contact, or portfolio information sections.");
} else {
  parts.push(`I found ${results.length} relevant portfolio item${results.length === 1 ? "" : "s"}.`);
}

parts.push("");
parts.push("What I found:");
if (sectionCount || fileCount) {
  parts.push(`- ${sectionCount} section match${sectionCount === 1 ? "" : "es"}.`);
  if (fileCount) parts.push(`- ${fileCount} file or link match${fileCount === 1 ? "" : "es"}.`);
}
if (pageCount) parts.push(`- ${pageCount} page or profile match${pageCount === 1 ? "" : "es"}.`);
if (projectNames.length) {
  parts.push(`- Best project candidate${projectNames.length === 1 ? "" : "s"}: ${projectNames.join(", ")}.`);
}

const firstSnippet = searchSnippet(results[0].text, question);
if (firstSnippet) {
  parts.push("");
  parts.push("Why it matched:");
  parts.push(firstSnippet);
}
return parts.join("\n");
}

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns an array. The caller usually iterates over that list to render entries, process files, or choose candidates.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalizeProfile`, `normalize`, `assistantGeneralEngineeringAnswer`, `searchSnippet`. Those calls show that `assistantLocalAnswer` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `assistantLocalAnswer`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: After the main flow, the working values inside `assistantLocalAnswer` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

ownerName: `ownerName` stores a computed value for temporary calculation. It is initialized from `normalizeProfile(profile || {}).displayName || "the portfolio owner"`. Within the function it is assigned or updated later in the function.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

generalAnswer: `generalAnswer` stores a computed value for temporary calculation. It is initialized from `assistantGeneralEngineeringAnswer(question)`. Within the function it is returned to the caller; assigned or updated later in the function.

generalAnswer: `generalAnswer` stores a computed value for temporary calculation. It is initialized from `assistantGeneralEngineeringAnswer(question)`. Within the function it is returned to the caller; assigned or updated later in the function.

projectIds: `projectIds` stores a list for portfolio data state. It is initialized from `[...new Set(results.map((item) => item.projectId).filter(Boolean))]`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is assigned or updated later in the function.

matchedProjects: `matchedProjects` stores a computed value for portfolio data state. It is initialized from `projectIds`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to

stay consistent. Within the function it is iterated or transformed as a collection; accesses properties/methods map; assigned or updated later in the function.

projectNames: projectNames stores a function or callback value for portfolio data state. It is initialized from `matchedProjects.map((project) => project.portfolioView?.title || project.title).slice(0, 3)`. It carries portfolio content, so rendering, search, AI context, and publishing expect its shape to stay consistent. Within the function it is accesses properties/methods join, length; passed into push; assigned or updated later in the function.

fileCount: fileCount stores a function or callback value for temporary calculation. It is initialized from `results.filter((item) => item.kind === "file").length`. Within the function it is passed into push; assigned or updated later in the function.

sectionCount: sectionCount stores a function or callback value for portfolio data state. It is initialized from `results.filter((item) => item.kind === "section" || item.kind === "subsection").length`. Within the function it is passed into push; assigned or updated later in the function.

pageCount: pageCount stores a function or callback value for temporary calculation. It is initialized from `results.filter((item) => item.kind === "page").length`. Within the function it is passed into push; assigned or updated later in the function.

parts: parts stores a list for temporary calculation. It is initialized from `[]`. Within the function it is returned to the caller; mutated as a collection or DOM container; accesses properties/methods join, push; assigned or updated later in the function.

firstSnippet: firstSnippet stores a computed value for temporary calculation. It is initialized from `searchSnippet(results[0].text, question)`. Within the function it is passed into push; assigned or updated later in the function.

function assistantSourcesMarkup(results = [])

Role: Read assistantSourcesMarkup first as a named idea, not as syntax. assistantSourcesMarkup is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantSourcesMarkup(results = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. results is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantSourcesMarkup

```
function assistantSourcesMarkup(results = []) {
  const sources = results.slice(0, 5);
  if (!sources.length) return "";
  const sourceButtons = sources.map((source) => {
    const id = String(assistantSourceCounter++);
    assistantSourceMap.set(id, source);
    return `
      <button type="button" data-ai-source-id="${id}">${escapeHtml(assistantSourceLabel(source))}</button>
    `;
  }).join("");
  return `
    <div class="ai-source-panel">
      <p class="ai-source-heading">Relevant portfolio links</p>
      <div class="ai-source-list" aria-label="Assistant sources">
        ${sourceButtons}
      </div>
    </div>
  `;
}
```

```

    </div>
  </div>
};
}

```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: The function also has to be read through the helpers it calls. It works with `escapeHtml`, `assistantSourceLabel`. Those calls show that `assistantSourcesMarkup` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `assistantSourcesMarkup`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local objects and variables inside `assistantSourcesMarkup` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

sources: `sources` stores a computed value for temporary calculation. It is initialized from `results.slice(0, 5)`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `map`; assigned or updated later in the function.

sourceButtons: `sourceButtons` stores a function or callback value for DOM reference or UI state. It is initialized from `sources.map((source) => {`. It anchors UI behavior to a specific browser element or window state. Within the function it is assigned or updated later in the function.

id: `id` stores a computed value for temporary calculation. It is initialized from `String(assistantSourceCounter++)`. Within the function it is passed into `set`; assigned or updated later in the function.

function `setAssistantStatus(message, state = "ready")`

Role: Read `setAssistantStatus` first as a named idea, not as syntax. `setAssistantStatus` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function setAssistantStatus(message, state = "ready") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `message` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `state` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `setAssistantStatus`

```
function setAssistantStatus(message, state = "ready") {
  if (!aiAssistantStatus) return;
  aiAssistantStatus.textContent = message;
  aiAssistantStatus.dataset.state = state;
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `setAssistantStatus` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `setAssistantStatus`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: Inside `setAssistantStatus`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function assistantDesktopDockedHeight()

Role: The best entry point for `assistantDesktopDockedHeight` is its responsibility in the surrounding workflow. `assistantDesktopDockedHeight` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantDesktopDockedHeight() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for assistantDesktopDockedHeight

```
function assistantDesktopDockedHeight() {  
  if (!aiAssistantPanel) return 360;  
  const value = getComputedStyle(aiAssistantPanel).getPropertyValue("--ai-console-rest-height").trim();  
  if (value.endsWith("px")) return Number.parseFloat(value) || 360;  
  return Math.min(Math.max(window.innerHeight * 0.38, 340), 390);  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Its connection to the rest of the file matters as much as its local code. No other named helper from this file is detected inside the body. That means `assistantDesktopDockedHeight` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `assistantDesktopDockedHeight`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The easiest way to follow the body of `assistantDesktopDockedHeight` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

value: `value` stores a computed value for temporary calculation. It is initialized from `getComputedStyle(aiAssistantPanel).getPropertyValue("--ai-console-rest-height").trim()`. Within the function it accesses properties/methods `endsWith`; passed into `parseFloat`; assigned or updated later in the function.

function updateAssistantPanelGrowth()

Role: Begin with the role of updateAssistantPanelGrowth. updateAssistantPanelGrowth belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function updateAssistantPanelGrowth() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for updateAssistantPanelGrowth

```
function updateAssistantPanelGrowth() {  
  if (!aiAssistantPanel || !aiAssistantLog || !aiAssistantForm) return;  
  const isDesktop = window.matchMedia("(min-width: 901px)").matches;  
  if (!isDesktop) {  
    aiAssistantPanel.style.removeProperty("--ai-console-live-height");  
    aiAssistantPanel.classList.remove("ai-assistant-panel-expanded");  
    return;  
  }  
  
  const dockedHeight = assistantDesktopDockedHeight();  
  const formHeight = aiAssistantForm.getBoundingClientRect().height || 46;  
  const clearHeight = aiClearChatButton?.getBoundingClientRect().height || 38;  
  const titleHeight = document.querySelector("#ai-assistant-title")?.getBoundingClientRect().height || 34;  
  const chromeHeight = titleHeight + formHeight + clearHeight + 92;  
  const messages = [...aiAssistantLog.querySelectorAll(".ai-message")];  
  const messageGap = messages.length > 1 ? (messages.length - 1) * 10 : 0;  
  const messageHeight = messages.reduce((sum, message) => sum + message.getBoundingClientRect().height, 0) +  
  messageGap;  
  const contentHeight = messageHeight + chromeHeight;  
  const maxHeight = Math.min(window.innerHeight * 0.76, 720);  
  const nextHeight = Math.round(Math.max(dockedHeight, Math.min(contentHeight, maxHeight)));  
  
  aiAssistantPanel.style.setProperty("--ai-console-live-height", `${nextHeight}px`);  
  aiAssistantPanel.classList.toggle("ai-assistant-panel-expanded", nextHeight > dockedHeight + 12);  
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with assistantDesktopDockedHeight. Those calls show that updateAssistantPanelGrowth is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without updateAssistantPanelGrowth, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local state inside updateAssistantPanelGrowth explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

isDesktop: isDesktop stores a computed value for temporary calculation. It is initialized from window.matchMedia("(min-width: 901px)").matches. Within the function it is assigned or updated later in the function.

dockedHeight: `dockedHeight` stores a computed value for temporary calculation. It is initialized from `assistantDesktopDockedHeight()`. Within the function it is passed into `round`, `toggle`; assigned or updated later in the function.

formHeight: `formHeight` stores a computed value for temporary calculation. It is initialized from `aiAssistantForm.getBoundingClientRect().height || 46`. Within the function it is assigned or updated later in the function.

clearHeight: `clearHeight` stores a computed value for temporary calculation. It is initialized from `aiClearChatButton?.getBoundingClientRect().height || 38`. Within the function it is assigned or updated later in the function.

titleHeight: `titleHeight` stores a DOM element reference. It is initialized from `document.querySelector("#ai-assistant-title")?.getBoundingClientRect().height || 34`. Within the function it is assigned or updated later in the function.

chromeHeight: `chromeHeight` stores a computed value for temporary calculation. It is initialized from `titleHeight + formHeight + clearHeight + 92`. Within the function it is assigned or updated later in the function.

messages: `messages` stores a list for temporary calculation. It is initialized from `[...aiAssistantLog.querySelectorAll(".ai-message")]`. Within the function it is iterated or transformed as a collection; accesses properties/methods `length`, `reduce`; assigned or updated later in the function.

messageGap: `messageGap` stores a computed value for temporary calculation. It is initialized from `messages.length > 1 ? (messages.length - 1) * 10 : 0`. Within the function it is assigned or updated later in the function.

messageHeight: `messageHeight` stores a function or callback value for temporary calculation. It is initialized from `messages.reduce((sum, message) => sum + message.getBoundingClientRect().height, 0) + messageGap`. Within the function it is assigned or updated later in the function.

contentHeight: `contentHeight` stores a computed value for temporary calculation. It is initialized from `messageHeight + chromeHeight`. Within the function it is passed into `round`; assigned or updated later in the function.

maxHeight: `maxHeight` stores a computed value for temporary calculation. It is initialized from `Math.min(window.innerHeight * 0.76, 720)`. Within the function it is passed into `round`; assigned or updated later in the function.

nextHeight: `nextHeight` stores a computed value for temporary calculation. It is initialized from `Math.round(Math.max(dockedHeight, Math.min(contentHeight, maxHeight)))`. Within the function it is passed into `setProperty`, `toggle`; assigned or updated later in the function.

function `renderAssistantInline(value = "")`

Role: To understand `renderAssistantInline`, start with the problem it owns. `renderAssistantInline` turns saved data for `AssistantInline` into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that `styles.css` expects.

Function entry point:

```
function renderAssistantInline(value = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `value` is human-facing text. The function cleans, limits, displays, compares, or turns it into a safe identifier so the UI and generated site remain readable. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string*

data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for renderAssistantInline

```
function renderAssistantInline(value = "") {  
  return renderInlineMath(value).replace(/\s*([^\s]+)\s*/g, "<strong>$1</strong>");  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with renderInlineMath. Those calls show that renderAssistantInline is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without renderAssistantInline, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: Inside renderAssistantInline, there are no local const, let, or var values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function renderAssistantAnswerContent(content = "")

Role: To understand renderAssistantAnswerContent, start with the problem it owns. renderAssistantAnswerContent turns saved data for AssistantAnswerContent into visible UI. It is a presentation function, but it also enforces public-site rules: empty content should not appear, rich content must be converted into safe HTML, and the generated markup must match the class names that styles.css expects.

Function entry point:

```
function renderAssistantAnswerContent(content = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. content is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for renderAssistantAnswerContent

```
function renderAssistantAnswerContent(content = "") {  
  const lines = String(content || "").replace(/\r\n?/g, "\n").split("\n");  
  const html = [];  
  let bulletItems = [];  
  let codeBlock = null;  
  const flushBullets = () => {  
    if (!bulletItems.length) return;  
    html.push(`<ul>${bulletItems.map((item) => `<li>${renderAssistantInline(item)}</li>`).join("")}</ul>`);  
    bulletItems = [];  
  };  
  const flushCode = () => {  
    if (!codeBlock) return;  
    const language = codeBlock.language ? ` data-language="${escapeHtml(codeBlock.language)}"` : "";  
    html.push(`<pre class="ai-code-block"${language}><code>${escapeHtml(codeBlock.lines.join("\n"))}</code></pre>`);  
    codeBlock = null;  
  };  
  
  lines.forEach((line) => {  
    const trimmed = line.trim();  
    const fence = trimmed.match(/^```([a-z0-9_+.#-]*)\s*$/i);  
    if (fence) {
```

```

    if (codeBlock) {
      flushCode();
    } else {
      flushBullets();
      codeBlock = { language: fence[1] || "", lines: [] };
    }
    return;
  }
  if (codeBlock) {
    codeBlock.lines.push(line);
    return;
  }
  if (!trimmed) {
    flushBullets();
    return;
  }
  const bullet = trimmed.match(/^[-*\u2022]\s+(.+)/u);
  if (bullet) {
    bulletItems.push(bullet[1]);
    return;
  }
  flushBullets();
  if (/^[^!?!?]{2,64}:/$.test(trimmed)) {
    html.push(`<p><strong>${renderAssistantInline(trimmed.slice(0, -1))}</strong></p>`);
  } else {
    html.push(`<p>${renderAssistantInline(trimmed)}</p>`);
  }
});
flushCode();
flushBullets();
return `<div class="ai-answer-content">${html.join("")} || "<p>I am ready.</p>"</div>`;
}

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a string that is already normalized for display, command output, a URL, a CSS value, or a file/path label.

Collaborators: Next, trace the helper calls that surround the function. It works with `renderAssistantInline`, `escapeHtml`. Those calls show that `renderAssistantAnswerContent` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `renderAssistantAnswerContent`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: After the main flow, the working values inside `renderAssistantAnswerContent` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

lines: `lines` stores a computed value for temporary calculation. It is initialized from `String(content || "").replace(/r\n?/g, "\n").split("\n")`. Within the function it is iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods `forEach`, `join`, `push`; passed into `push`; assigned or updated later in the function.

html: `html` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is mutated as a collection or DOM container; accesses properties/methods `join`, `push`; assigned or updated later in the function.

bulletItems: `bulletItems` stores a list for temporary calculation. It is initialized from `[]`. Within the function it is iterated or transformed as a collection; mutated as a collection or DOM container; accesses properties/methods `length`, `map`, `push`; passed into `push`; assigned or updated later in the function.

codeBlock: `codeBlock` stores a computed value for temporary calculation. It is initialized from `null`. Within the function it is accesses properties/methods `language`, `lines`; passed into `escapeHtml`, `push`; assigned or updated later in the function.

flushBullets: flushBullets stores a function or callback value for temporary calculation. It is initialized from `() => {}`. Within the function it is assigned or updated later in the function.

flushCode: flushCode stores a function or callback value for temporary calculation. It is initialized from `() => {}`. Within the function it is assigned or updated later in the function.

language: language stores a computed value for compiler or language configuration. It is initialized from `codeBlock.language ? `data-language="${escapeHtml(codeBlock.language)}" : ""`. It supports the compile workspace by describing language rules, executable locations, tool status, or compiler output. Within the function it is passed into `escapeHtml`, `push`; assigned or updated later in the function.

trimmed: trimmed stores a computed value for temporary calculation. It is initialized from `line.trim()`. Within the function it is accesses properties/methods `match`, `slice`; passed into `push`; assigned or updated later in the function.

fence: fence stores a computed value for temporary calculation. It is initialized from `trimmed.match(/^```([a-z0-9_+.#-]*)\s*$/i)`. Within the function it is assigned or updated later in the function.

bullet: bullet stores a computed value for temporary calculation. It is initialized from `trimmed.match(/[-*\u2022]\s+(.+)/u)`. Within the function it is passed into `push`; assigned or updated later in the function.

function appendAssistantMessage(role, content, sources = [])

Role: The best entry point for `appendAssistantMessage` is its responsibility in the surrounding workflow. `appendAssistantMessage` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function appendAssistantMessage(role, content, sources = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `role` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `content` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `sources` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for appendAssistantMessage

```
function appendAssistantMessage(role, content, sources = []) {  
  if (!aiAssistantLog) return;  
  const article = document.createElement("article");  
  article.className = `ai-message ai-message-${role}`;  
  article.innerHTML = role === "assistant"  
    ? `${renderAssistantAnswerContent(content)}${assistantSourcesMarkup(sources)}`  
    : `

${renderMultilineInlineText(content)}

`;  
  aiAssistantLog.append(article);  
  updateAssistantPanelGrowth();  
  aiAssistantLog.scrollTop = aiAssistantLog.scrollHeight;  
  return article;  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these

implementation details: creates DOM elements instead of relying only on static HTML; writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `renderAssistantAnswerContent`, `assistantSourcesMarkup`, `renderMultilineInlineText`, `updateAssistantPanelGrowth`. Those calls show that `appendAssistantMessage` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `appendAssistantMessage`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The easiest way to follow the body of `appendAssistantMessage` is to watch its local values. They are included here because they explain the implementation rather than merely naming syntax.

article: `article` stores a DOM element reference. It is initialized from `document.createElement("article")`. Within the function it is returned to the caller; accesses properties/methods `className`, `innerHTML`; passed into `append`, `createElement`; assigned or updated later in the function.

function scrollAssistantAnswerToStart(article)

Role: Read `scrollAssistantAnswerToStart` first as a named idea, not as syntax. `scrollAssistantAnswerToStart` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function scrollAssistantAnswerToStart(article) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `article` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for scrollAssistantAnswerToStart

```
function scrollAssistantAnswerToStart(article) {  
  if (!aiAssistantLog || !article) return;  
  const scrollToAnswer = () => {  
    aiAssistantLog.scrollTop = Math.max(0, article.offsetTop - aiAssistantLog.offsetTop - 4);  
  };  
  if (typeof window.requestAnimationFrame === "function") {  
    window.requestAnimationFrame(scrollToAnswer);  
  } else {  
    window.setTimeout(scrollToAnswer, 0);  
  }  
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it uses a timer to avoid blocking or to wait for another process. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The function also has to be read through the helpers it calls. No other named helper from this file is detected inside the body. That means `scrollAssistantAnswerToStart` performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `scrollAssistantAnswerToStart`, the assistant would either answer with less context, display

unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local objects and variables inside `scrollAssistantAnswerToStart` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

scrollToAnswer: `scrollToAnswer` stores a function or callback value for temporary calculation. It is initialized from `() => {}`. Within the function it is passed into `requestAnimationFrame`, `setTimeout`; assigned or updated later in the function.

function appendAssistantPendingMessage()

Role: Begin with the role of `appendAssistantPendingMessage`. `appendAssistantPendingMessage` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function appendAssistantPendingMessage() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for appendAssistantPendingMessage

```
function appendAssistantPendingMessage() {  
  const article = appendAssistantMessage("assistant", "Thinking", []);  
  article?.classList.add("ai-message-pending");  
  return article;  
}
```

Implementation: The body is where the contract above becomes behavior. Inside the body, it updates browser DOM. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: toggles CSS classes to express UI state.

Collaborators: The surrounding workflow becomes clearer when you notice its collaborators. It works with `appendAssistantMessage`. Those calls show that `appendAssistantPendingMessage` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This is why the function is worth separating instead of burying the logic somewhere else. Without `appendAssistantPendingMessage`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local state inside `appendAssistantPendingMessage` explains how the larger task is split into smaller steps. They are included here because they explain the implementation rather than merely naming syntax.

article: `article` stores a computed value for temporary calculation. It is initialized from `appendAssistantMessage("assistant", "Thinking", [])`. Within the function it is returned to the caller; assigned or updated later in the function.

function replaceAssistantMessage(article, content, sources = [])

Role: The best entry point for `replaceAssistantMessage` is its responsibility in the surrounding workflow. `replaceAssistantMessage` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function replaceAssistantMessage(article, content, sources = []) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `article` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `content` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything. `sources` is source text supplied by the builder, project catalog, GitHub, or an uploaded file. The function inspects, formats, saves, compiles, highlights, or extracts meaning from this text. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for `replaceAssistantMessage`

```
function replaceAssistantMessage(article, content, sources = []) {  
  if (!article) {  
    appendAssistantMessage("assistant", content, sources);  
    return;  
  }  
  article.className = "ai-message ai-message-assistant";  
  article.innerHTML = `${renderAssistantAnswerContent(content)}${assistantSourcesMarkup(sources)}`;  
  updateAssistantPanelGrowth();  
  scrollAssistantAnswerToStart(article);  
}
```

Implementation: After the syntax, move into the body and follow the work in the order the function performs it. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Collaborators: Its connection to the rest of the file matters as much as its local code. It works with `appendAssistantMessage`, `renderAssistantAnswerContent`, `assistantSourcesMarkup`, `updateAssistantPanelGrowth`, `scrollAssistantAnswerToStart`. Those calls show that `replaceAssistantMessage` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: This function earns its place in the file because it protects one behavior from being rewritten differently elsewhere. Without `replaceAssistantMessage`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: Inside `replaceAssistantMessage`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function `setAssistantBusy(isBusy)`

Role: To understand `setAssistantBusy`, start with the problem it owns. `setAssistantBusy` belongs to portfolio `ai` frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function setAssistantBusy(isBusy) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `isBusy` is the caller-supplied input used by this function. The function interprets it according to its local responsibility, often converting it into safer, more predictable data before returning or rendering anything.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string*

data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for setAssistantBusy

```
function setAssistantBusy(isBusy) {  
  const submitButton = aiAssistantForm?.querySelector('button[type="submit"]');  
  if (aiAssistantInput) aiAssistantInput.disabled = isBusy;  
  if (submitButton) submitButton.disabled = isBusy;  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: Next, trace the helper calls that surround the function. No other named helper from this file is detected inside the body. That means setAssistantBusy performs its rule directly from parameters, module state, standard APIs, or local calculations.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without setAssistantBusy, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: After the main flow, the working values inside setAssistantBusy reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

submitButton: submitButton stores a DOM element reference. It is initialized from aiAssistantForm?.querySelector("button[type='submit']"). It anchors UI behavior to a specific browser element or window state. Within the function it is accesses properties/methods disabled; assigned or updated later in the function.

function clearAssistantChat()

Role: To understand clearAssistantChat, start with the problem it owns. clearAssistantChat belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Function entry point:

```
function clearAssistantChat() { ... }
```

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for clearAssistantChat

```
function clearAssistantChat() {  
  if (!aiAssistantLog) return;  
  assistantSourceCounter = 0;  
  assistantSourceMap = new Map();  
  assistantChatHistory = [];  
  aiAssistantLog.innerHTML = "";  
  setAssistantStatus("Chat cleared");  
  updateAssistantPanelGrowth();  
  aiAssistantInput?.focus();  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it updates browser DOM. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state. A close reading of the body shows these implementation details: writes generated markup into the page, which is why escaping and sanitizing helpers around it matter.

Collaborators: Next, trace the helper calls that surround the function. It works with `setAssistantStatus`, `updateAssistantPanelGrowth`. Those calls show that `clearAssistantChat` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `clearAssistantChat`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: Inside `clearAssistantChat`, there are no local `const`, `let`, or `var` values to discuss. The function mainly works from its parameters, module-level state, direct expressions, or helpers it calls.

function assistantRemoteTimeoutMs(question = "", context = {}, options = {})

Role: To understand `assistantRemoteTimeoutMs`, start with the problem it owns. `assistantRemoteTimeoutMs` is one piece of the Ask My Portfolio conversation system. It helps decide what the user means, what evidence belongs in context, how answers should be displayed, or how the chat panel should behave.

Function entry point:

```
function assistantRemoteTimeoutMs(question = "", context = {}, options = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `context` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for assistantRemoteTimeoutMs

```
function assistantRemoteTimeoutMs(question = "", context = {}, options = {}) {  
  const clean = normalize(question);  
  const sourceFetches = Array.isArray(context.sourceFetches) ? context.sourceFetches : [];  
  const usesGitHub = sourceFetches.some((source) => String(source.kind || "").toLowerCase() === "github" ||  
    /github\.com\/i.test(source.url || ""));  
  if (usesGitHub || /\b(github|repo|repository|repositories|source\s+code|code|snippet|firmware|verilog|  
    systemverilog|python|javascript|circuit\s+file)\b/.test(clean)) {  
    return 45000;  
  }  
  if (options.allowWebSearch) return 30000;  
  return 18000;  
}
```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. The body is mostly deterministic transformation logic: it reads its inputs, normalizes or scores them, and returns a value the caller can trust. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere.

Collaborators: Next, trace the helper calls that surround the function. It works with `normalize`. Those calls show that `assistantRemoteTimeoutMs` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `assistantRemoteTimeoutMs`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: After the main flow, the working values inside `assistantRemoteTimeoutMs` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

clean: `clean` stores a computed value for temporary calculation. It is initialized from `normalize(question)`. Within the function it is passed into `test`; assigned or updated later in the function.

sourceFetches: `sourceFetches` stores a computed value for temporary calculation. It is initialized from `Array.isArray(context.sourceFetches) ? context.sourceFetches : []`. Within the function it is accesses properties/methods some; passed into `isArray`; assigned or updated later in the function.

usesGitHub: `usesGitHub` stores a function or callback value for Git publishing and authorization state. It is initialized from `sourceFetches.some((source) => String(source.kind || "").toLowerCase() === "github" || /github\.com/i.test(source.url || ""))`. It affects publishing, where mistakes can change the public website or expose the wrong files. Within the function it is assigned or updated later in the function.

async function askRemoteAssistant(question, context, options = {})

Role: To understand `askRemoteAssistant`, start with the problem it owns. `askRemoteAssistant` belongs to portfolio ai frontend. This function belongs to the public chatbot. It classifies questions, gathers context, renders answers, or manages the chat UI. In this role, it owns one named operation that later code depends on.

Function entry point:

```
async function askRemoteAssistant(question, context, options = {}) { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. `context` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. `options` is a structured object rather than one small value. It lets the caller pass several related settings or pieces of state together so the function can make one coordinated decision. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: *Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.*

Exact source block for askRemoteAssistant

```
async function askRemoteAssistant(question, context, options = {}) {
  const endpoint = assistantEndpoint();
  if (!endpoint) return null;
  const controller = new AbortController();
  let didTimeout = false;
  const timer = setTimeout(() => {
    didTimeout = true;
    controller.abort();
  }, assistantRemoteTimeoutMs(question, context, options));
  let response;
  try {
    response = await fetch(endpoint, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      signal: controller.signal,
      body: JSON.stringify({
        context,
        conversation: options.conversation || [],
        intent: options.intent || context.intent || "portfolio_specific",
        allowWebSearch: Boolean(options.allowWebSearch),
        question,
        mode: "portfolio-plus-general-knowledge",
        responseType: "chatgpt-like-answer-first-links-second"
      })
    });
  } catch (error) {
    if (didTimeout) throw new Error("AI request timed out while reading public sources.");
  }
}
```

```

    throw error;
  } finally {
    clearTimeout(timer);
  }
  if (!response.ok) throw new Error("AI service unavailable");
  const data = await response.json();
  const answer = String(data.answer || data.message || "").trim();
  return answer ? { answer, model: data.model || "", usedWebSearch: Boolean(data.usedWebSearch) } : null;
}

```

Implementation: Once the inputs are clear, the implementation shows how the function turns those inputs into useful program state. Inside the body, it waits for asynchronous work, calls a network resource and uses a timer to avoid blocking or to wait for another process. It returns a computed value used immediately by a caller. The important point is that the caller does not repeat this rule elsewhere. A close reading of the body shows these implementation details: serializes structured data before sending or saving it; throws explicit errors when a required safety or compatibility condition fails; performs a fetch and therefore must handle network delay and failure.

Collaborators: Next, trace the helper calls that surround the function. It works with `assistantEndpoint`, `assistantRemoteTimeoutMs`. Those calls show that `askRemoteAssistant` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The practical importance of the function is easiest to see by imagining it removed. Without `askRemoteAssistant`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: After the main flow, the working values inside `askRemoteAssistant` reveal what the function must remember while it runs. They are included here because they explain the implementation rather than merely naming syntax.

endpoint: `endpoint` stores a computed value for network or routing value. It is initialized from `assistantEndpoint()`. Within the function it is passed into `fetch`; assigned or updated later in the function.

controller: `controller` stores a constructed object for temporary calculation. It is initialized from `new AbortController()`. Within the function it is accesses properties/methods `abort`, `signal`; passed into `fetch`; assigned or updated later in the function.

didTimeout: `didTimeout` stores a boolean value for temporary calculation. It is initialized from `false`. Within the function it is assigned or updated later in the function.

timer: `timer` stores a function or callback value for temporary calculation. It is initialized from `setTimeout(() => {`. Within the function it is passed into `clearTimeout`; assigned or updated later in the function.

response: `response` starts without an immediate value. In this file that usually means it is filled by a loop, branch, callback, or later assignment inside the same function. Within the function it is accesses properties/methods `json`, `ok`; assigned or updated later in the function.

data: `data` stores the completed result of an awaited operation. It is initialized from `await response.json()`. Within the function it is accesses properties/methods `answer`, `message`, `model`, `usedWebSearch`; passed into `Boolean`, `String`; assigned or updated later in the function.

answer: `answer` stores a computed value for temporary calculation. It is initialized from `String(data.answer || data.message || "").trim()`. Within the function it is returned to the caller; passed into `String`; assigned or updated later in the function.

async function answerAssistantQuestion(question = "")

Role: Read `answerAssistantQuestion` first as a named idea, not as syntax. `answerAssistantQuestion` is the public chatbot orchestrator. It receives the user's typed question, appends it to chat history, classifies the intent, gathers relevant portfolio context, creates a pending assistant answer, calls the backend AI when appropriate, falls back to local answers when needed, renders sources only when relevant, and restores focus to the input. This function is `async` because the AI backend may need network time and because context gathering can

include indexed file text. It is the reason Ask My Portfolio behaves like a conversation instead of a static search form.

Function entry point:

```
async function answerAssistantQuestion(question = "") { ... }
```

Inputs: The function-specific inputs are worth reading before the body. `question` is user language: a search query, AI question, or classified assistant intent. It drives scoring, context selection, routing, or AI response behavior. The default value is intentional: it lets the function fail softly or produce a predictable empty result when the caller omits that field.

Reading the source: Now place that explanation beside the actual source. Read the code slowly: the colored keywords show control flow, the teal function name marks the entry point, the orange sections are string data, and the muted text marks comments. The goal is not to memorize the code, but to see how the written implementation matches the responsibility described above.

Exact source block for `answerAssistantQuestion`

```
async function answerAssistantQuestion(question = "") {
  const cleanQuestion = String(question || "").trim();
  if (!cleanQuestion) return;
  const priorConversation = assistantChatHistory.slice();
  appendAssistantMessage("user", cleanQuestion);
  assistantRemember("user", cleanQuestion);
  const pendingMessage = appendAssistantPendingMessage();
  setAssistantBusy(true);
  try {
    setAssistantStatus(assistantEndpoint() ? "Composing AI answer" : "Composing local answer", "working");

    const intent = assistantQuestionIntent(cleanQuestion, priorConversation);
    const rawSources = assistantFallbackResults(cleanQuestion, intent);
    const sources = assistantSourcesForDisplay(cleanQuestion, rawSources, intent);
    const context = assistantContextForQuestion(cleanQuestion, intent, rawSources);
    const conversation = assistantConversationForRequest(cleanQuestion, priorConversation);
    try {
      const remoteAnswer = await askRemoteAssistant(cleanQuestion, context, {
        allowWebSearch: assistantQuestionAllowsPublicLookup(cleanQuestion, intent),
        conversation,
        intent
      });
      if (remoteAnswer?.answer) {
        replaceAssistantMessage(pendingMessage, remoteAnswer.answer, sources);
        assistantRemember("assistant", remoteAnswer.answer);
        setAssistantStatus(remoteAnswer.model ? `AI answer ready: ${remoteAnswer.model}` : "AI answer ready");
        return;
      }
    } catch (error) {
      setAssistantStatus(error?.message || "Local answer used", "error");
    }

    const fallbackAnswer = assistantLocalAnswer(cleanQuestion, sources, intent);
    replaceAssistantMessage(pendingMessage, fallbackAnswer, sources);
    assistantRemember("assistant", fallbackAnswer);
    if (!assistantEndpoint()) setAssistantStatus("Local portfolio intelligence");
  } catch {
    const errorAnswer = "I had trouble processing that question. Try rephrasing it with a project name, tool name, or electronics topic.";
    replaceAssistantMessage(pendingMessage, errorAnswer, []);
    assistantRemember("assistant", errorAnswer);
    setAssistantStatus("Assistant error", "error");
  } finally {
    setAssistantBusy(false);
    aiAssistantInput?.focus();
  }
}
```

Implementation: With the signature understood, the implementation can be read as a sequence of decisions and side effects. Inside the body, it waits for asynchronous work. It mostly works through side effects, such as updating the DOM, writing files, sending an HTTP response, changing history, or mutating controlled runtime state.

Collaborators: The function also has to be read through the helpers it calls. It works with `appendAssistantMessage`, `assistantRemember`, `appendAssistantPendingMessage`, `setAssistantBusy`, `setAssistantStatus`, `assistantEndpoint`, `assistantQuestionIntent`, `assistantFallbackResults`. Those calls show that `answerAssistantQuestion` is not isolated; it is one piece of a larger workflow where helpers clean inputs, perform side effects, render results, or protect boundaries before the next step runs.

Why it matters: The application keeps this behavior isolated because other features rely on it being consistent. Without `answerAssistantQuestion`, the assistant would either answer with less context, display unrelated links, lose conversation memory, fail to format messages cleanly, or expose model/provider details in the wrong layer.

Local values: The local objects and variables inside `answerAssistantQuestion` show how the implementation holds temporary work while it runs. They are included here because they explain the implementation rather than merely naming syntax.

cleanQuestion: `cleanQuestion` stores a computed value for temporary calculation. It is initialized from `String(question || "").trim()`. Within the function it is passed into `appendAssistantMessage`, `askRemoteAssistant`, `assistantContextForQuestion`, `assistantConversationForRequest`, `assistantFallbackResults`, `assistantLocalAnswer`, `assistantQuestionIntent`, `assistantRemember`; assigned or updated later in the function.

priorConversation: `priorConversation` stores a computed value for temporary calculation. It is initialized from `assistantChatHistory.slice()`. Within the function it is passed into `assistantConversationForRequest`, `assistantQuestionIntent`; assigned or updated later in the function.

pendingMessage: `pendingMessage` stores a computed value for temporary calculation. It is initialized from `appendAssistantPendingMessage()`. Within the function it is passed into `replaceAssistantMessage`; assigned or updated later in the function.

intent: `intent` stores a computed value for temporary calculation. It is initialized from `assistantQuestionIntent(cleanQuestion, priorConversation)`. Within the function it is passed into `askRemoteAssistant`, `assistantContextForQuestion`, `assistantFallbackResults`, `assistantLocalAnswer`, `assistantSourcesForDisplay`; assigned or updated later in the function.

rawSources: `rawSources` stores a computed value for temporary calculation. It is initialized from `assistantFallbackResults(cleanQuestion, intent)`. Within the function it is passed into `assistantContextForQuestion`, `assistantSourcesForDisplay`; assigned or updated later in the function.

sources: `sources` stores a computed value for temporary calculation. It is initialized from `assistantSourcesForDisplay(cleanQuestion, rawSources, intent)`. Within the function it is passed into `assistantLocalAnswer`, `replaceAssistantMessage`; assigned or updated later in the function.

context: `context` stores a computed value for temporary calculation. It is initialized from `assistantContextForQuestion(cleanQuestion, intent, rawSources)`. Within the function it is passed into `askRemoteAssistant`; assigned or updated later in the function.

conversation: `conversation` stores a computed value for temporary calculation. It is initialized from `assistantConversationForRequest(cleanQuestion, priorConversation)`. Within the function it is assigned or updated later in the function.

remoteAnswer: `remoteAnswer` stores the completed result of an awaited operation. It is initialized from `await askRemoteAssistant(cleanQuestion, context, {`. Within the function it is accesses properties/methods `answer`, `model`; passed into `assistantRemember`, `replaceAssistantMessage`, `setAssistantStatus`; assigned or updated later in the function.

fallbackAnswer: `fallbackAnswer` stores a computed value for temporary calculation. It is initialized from `assistantLocalAnswer(cleanQuestion, sources, intent)`. Within the function it is passed into `assistantRemember`, `replaceAssistantMessage`; assigned or updated later in the function.

errorAnswer: errorAnswer stores a string value for temporary calculation. It is initialized from "I had trouble processing that question. Try rephrasing it with a project name, tool name, or electronics topic.". Within the function it is passed into assistantRemember, replaceAssistantMessage; assigned or updated later in the function.